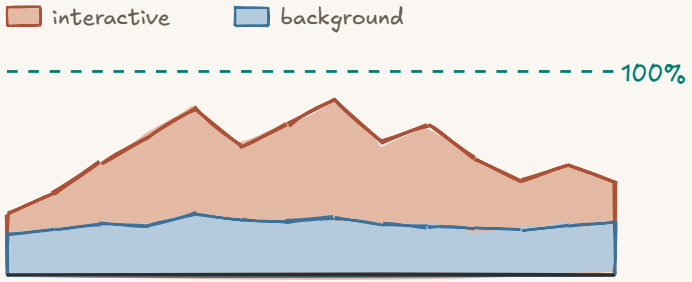
Fabric Capacity Management

From mechanics to strategy

a deep dive

Daan Humblé

dataMinds Connect · Mechelen · 12 October 2026



Fabric Capacity Deep Dive: from mechanics to strategy. A three-hour session at dataMinds Connect 2026, Mechelen, on how Microsoft Fabric capacities work, how to read them, and how to run and pay for them well.

Session scaffold and settled decisions

Before I tell you about your capacity...

01 What's your role?
02 What do you work on day to day?
03 How many capacities do you touch?
04 P or F? Throttled last month?

The session opens by mapping the room: each person's role, their daily work, how many capacities they run or touch, and whether they are on P or F SKUs and have been throttled recently.

Chapter 1 agreed storyline

Why talk about capacities?

The foundation	everything you run in Fabric runs on one
The concept	one of the most fascinating ideas in Fabric
The role	you don't build the solutions, you enable everyone who does

Capacities matter because every Fabric workload runs on one, because a self-balancing shared pool is a distinctive design, and because the capacity administrator enables everyone who builds on the platform rather than building the solutions themselves.

Microsoft Fabric licenses and capacity SKUs / Chapter 1 agreed storyline

Here's me.



Daan Humblé

Cloud Solution Architect · Data & AI

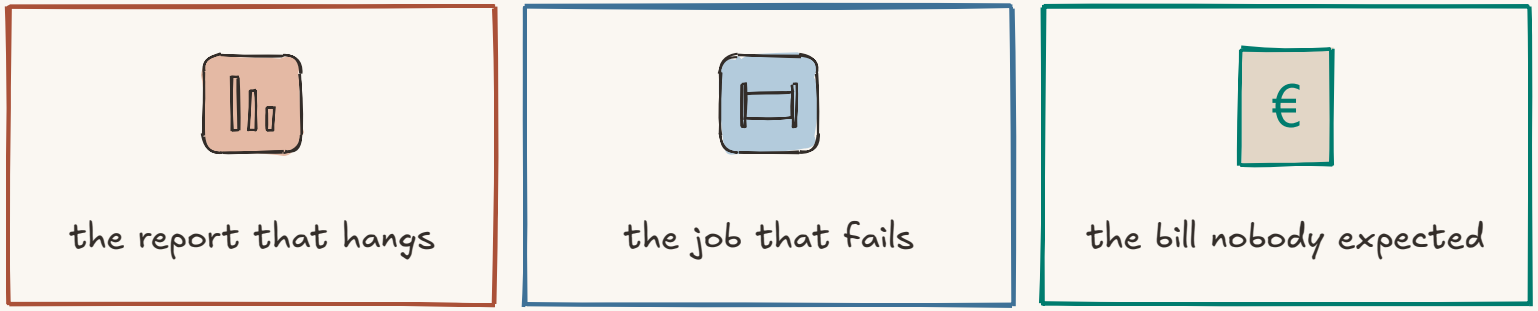
LinkedIn /in/daanhumble
GitHub /HumbleDaan

Every Fabric conversation ends up at the capacity.

Presented by Daan Humblé, Cloud Solution Architect for Data & AI. LinkedIn: /in/daanhumble. GitHub: /HumbleDaan. The session draws on recurring customer conversations about Fabric capacities.

Chapter 1 agreed storyline

You notice a capacity when it hurts.



Capacities usually become visible only when something goes wrong: interactive reports slow down or are rejected, background jobs fail, or costs exceed expectations. The session aims to give administrators and creators what they need to prevent all three.

Session scaffold and settled decisions

Agenda

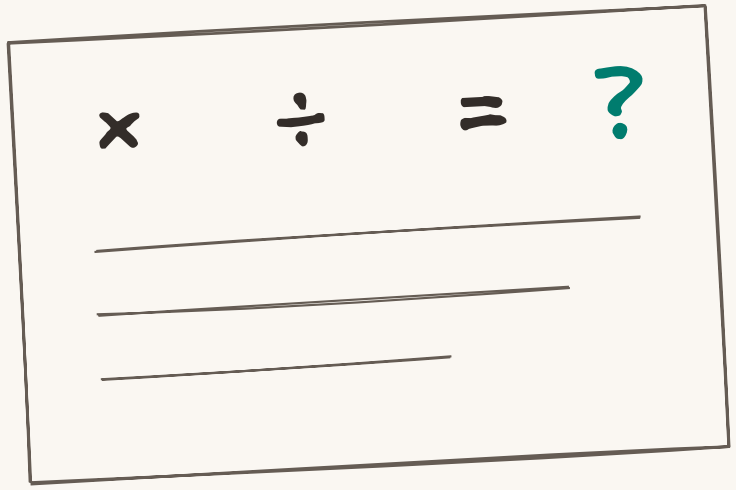
A well-run capacity is invisible.

01	02	03	04	05	06	07	08	09
Intro	Compute	Billed	Monitor	Understand	Protect	Rescue	Pay	Design

The session has nine parts. Before the break: introduction, compute, billing, monitoring and understanding what the monitoring shows. After the break: protecting the capacity, rescuing it when it fails, paying for it, and designing it. Throughline: a well-run capacity is invisible.

Session scaffold and settled decisions

Fair warning: there will be maths.



Brain gymnastics. Napkin-sized.

The session includes some calculation: how capacity usage is measured and smoothed, and breakeven points for billing options. All of it is multiplication and division, built up step by step, and every step is in the handout.

Session scaffold and settled decisions

Every price here has the same basis.

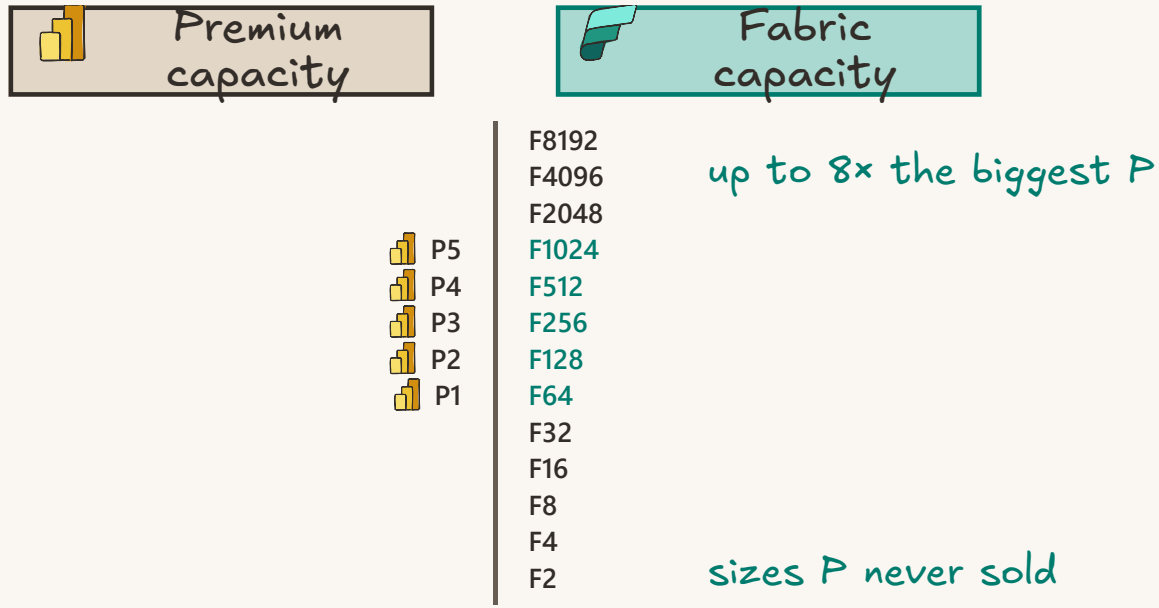
USD	West Europe	2 Sep 2026
list price	region	price pulled

Unless a slide says otherwise.

All prices in this session are USD list prices for West Europe, taken on 2 Sep 2026, unless a slide states otherwise. Prices change; the arithmetic does not, and re-pulling prices moves the totals but none of the ratios.

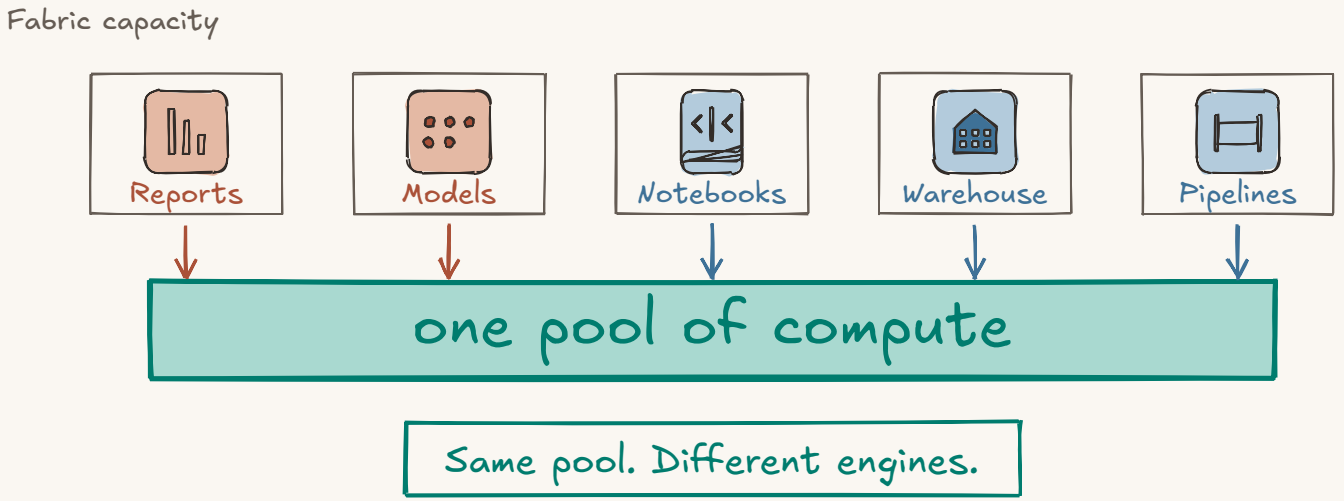
Microsoft Fabric pricing

From Premium capacity to Fabric.



Power BI Premium (P SKUs, generally available June 2017) provided dedicated capacity for one workload family, Power BI. Fabric (public preview May 2023, generally available 15 November 2023) introduced Fabric capacities (F SKUs): one capacity serving many workload families. F64, F128, F256, F512 and F1024 are compute-equivalent to P1 to P5 (one P v-core = 8 CUs), which is a compute comparison, not a feature or licensing equivalence. F SKUs also extend below (F2 to F32) and above (F2048 to F8192) the old range.

One pool. Many engines.



An F capacity provides one pool of compute for every Fabric workload engine. Reports, models, notebooks, warehouses and pipelines share the same purchased allocation even though their execution and accounting behave differently.

Chapter 1 verification trail / Microsoft Fabric licenses and capacity SKUs

What F changed: it's an Azure resource.

P SKU

part of a Microsoft 365 agreement

F SKU

an Azure resource

Pay as you go	billed per second
Reserve	a year ahead, at a discount
Pause · resume · resize	whenever you need to
Private networking	managed private endpoints

F SKUs are Azure resources rather than part of a Microsoft 365 agreement. That brings pay-as-you-go billing per second (one-minute minimum), optional yearly reservations, pause/resume and resize, and networking features such as managed private endpoints, which are supported on Fabric capacities only.

Microsoft Fabric licenses and capacity SKUs / Managed private endpoints for Microsoft Fabric

Shared three ways.

Across workloads	every engine draws on one pool
Across projects	dozens of workspaces, one ceiling
Across people	anyone can run anything

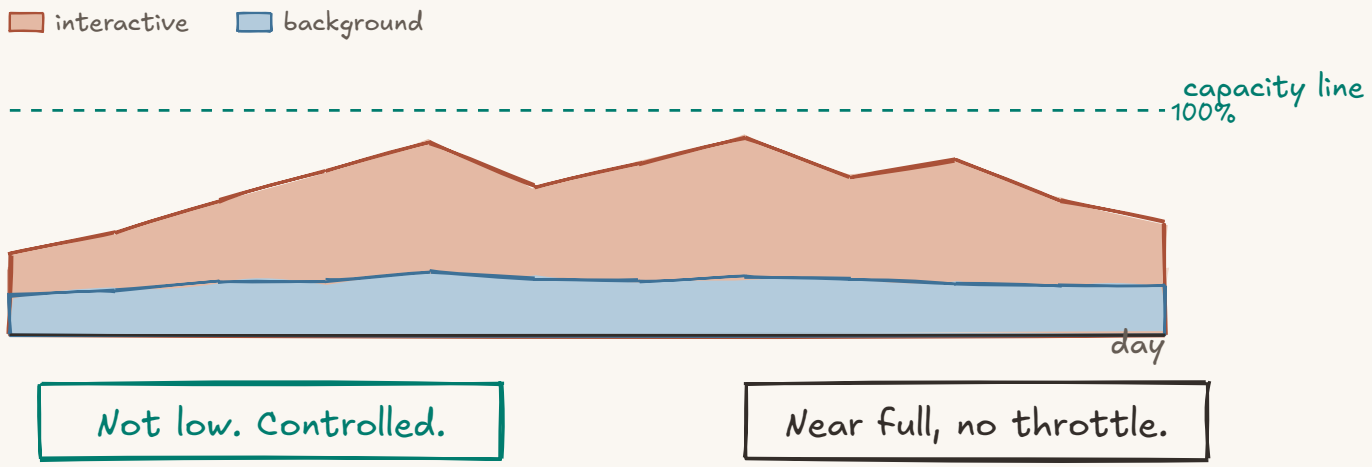
Nobody has to ask you first.*

* without you giving up control

A capacity is shared three ways: across every Fabric workload, across the many projects and workspaces assigned to it, and across every creator in those workspaces. Creators need no approval to run work, though administrators keep the ability to set limits.

Microsoft Fabric licenses and capacity SKUs / Chapter 1 agreed storyline

The admin's job: full, never throttled.



Invisible to users, because the admin did the work.

The capacity administrator's goal is high utilization without throttling. Low utilization indicates purchased capacity sitting idle; high utilization is healthy when users experience no throttling. Done well, the capacity is invisible to its users.

Session scaffold and settled decisions

Invisible does not mean free.

ADMIN

06 Protect
Who can hurt whom?

07 Rescue
What when it breaks?

08 Pay
Who owns the bill?

09 Design
How big, and what goes where?

USER Own the work you submit

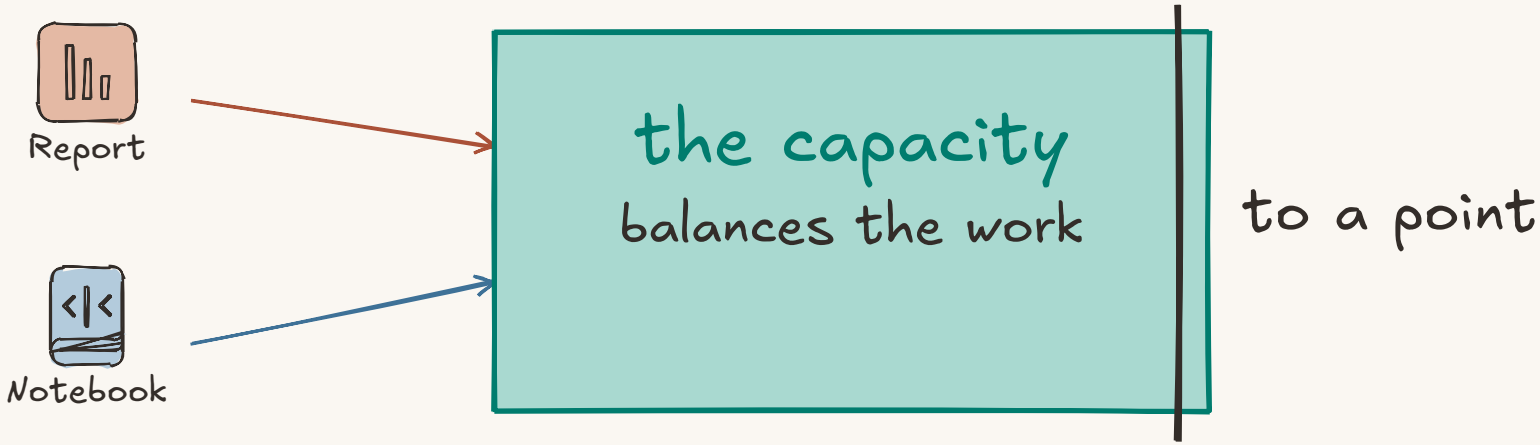
01 Intro · 02 Compute · 03 Billed · 04 Monitor · 05 Understand · 06 Protect · 07 Rescue · 08 Pay · 09 Design

the admin's four: the second half of today

Capacity administration has four responsibilities, which are the four chapters after the break: protecting workloads, rescuing a capacity in trouble, paying for it, and designing it. Creators remain responsible for the efficiency of their own workloads.

Session scaffold and settled decisions / Chapter 1 agreed storyline

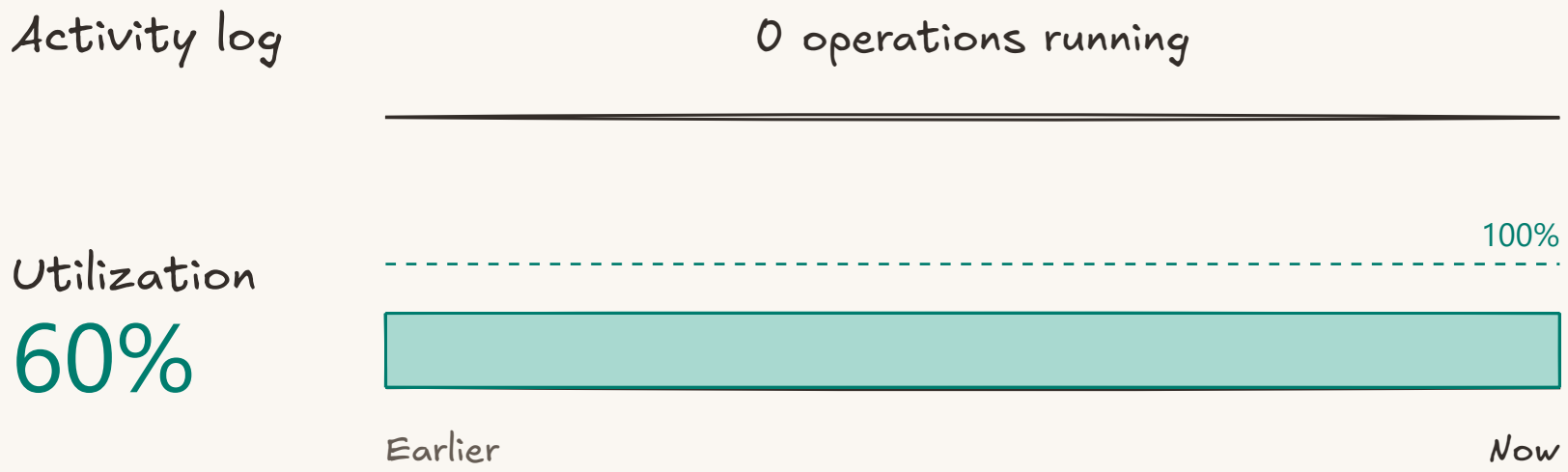
Self-managing. Not hands-off.



A shared capacity absorbs and balances workload variation on its own, but that help is bounded, and it still needs attention. Chapter 2 explains the mechanism and the point where demand becomes an operating problem.

Chapter 1 agreed storyline

Nothing running. Still a load.



The question to hold: the activity log shows no operation running, yet utilization shows a steady load. How can a capacity show consumption when nothing is executing? The same picture returns, explained, once execution and accounting have been separated. It is an illustration, not a tenant measurement.

Fabric throttling, bursting and smoothing / Capacity Metrics app: utilization, throttling and overages

02 · Compute

How does it actually work?

What you run — and how your capacity counts it.

01 Intro
02 Compute
03 Billed
04 Monitor
05 Understand
06 Protect
07 Rescue
08 Pay
09 Design

Compute: how it actually works. This chapter follows one piece of work from the moment it runs to the way a capacity accounts for it: the unit compute is measured in, how much a capacity provides, bursting, smoothing, carried-forward overage and the throttling thresholds. Together they explain why utilization can show a load while nothing is running.

Fabric throttling, bursting and smoothing

capacity unit

noun / CU

The currency everything you do in Microsoft Fabric is expressed in.

Not a CPU core. Not a fixed amount of memory.

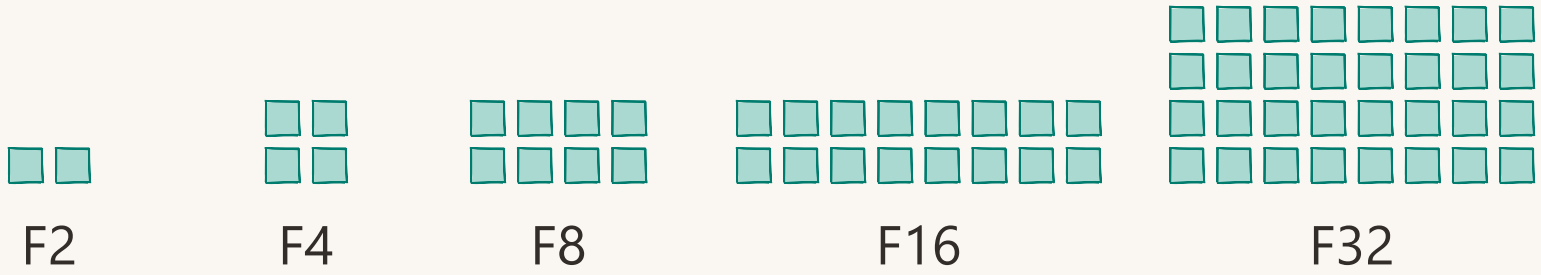


A Capacity Unit (CU) is the currency every Fabric operation is expressed in: the common compute allocation across workloads. Work is measured in CU-seconds: allocation multiplied by time. This abstraction lets different engines share one budget without pretending a CU is a core or a unit of memory.

Fabric licenses and capacity SKUs

How much do you get?

One block = 1 CU



The number in an F-SKU is its baseline CU allocation. F2 = 2 CU; F64 = 64 CU. Doubling the SKU doubles the compute budget for any fixed period. This is a claim about CU, not about every memory, concurrency or per-workload limit.

Fabric licenses and capacity SKUs

Thirty seconds. One budget.

2 CU × 30 s



F2 budget: 60 CU-s

SKU	CU	CU-s / 30 s
F2	2	60
F4	4	120
F8	8	240
F16	16	480
F32	32	960
F64	64	1,920

Fabric timepoints are 30 seconds long. Multiply the SKU allocation by 30: **F2/F4/F8/F16/F32/F64 = 60/120/240/480/960/1,920 CU-s per timepoint**. The animation adds elapsed budget; it does not spread an operation's work. Allocation is measured in CU; each interval's work budget is measured in CU-seconds.

CU is the unit. Percent is the metric. Every utilization figure in the Metrics App is work divided by this per-timepoint budget,

Traditional system. Fixed ceiling.

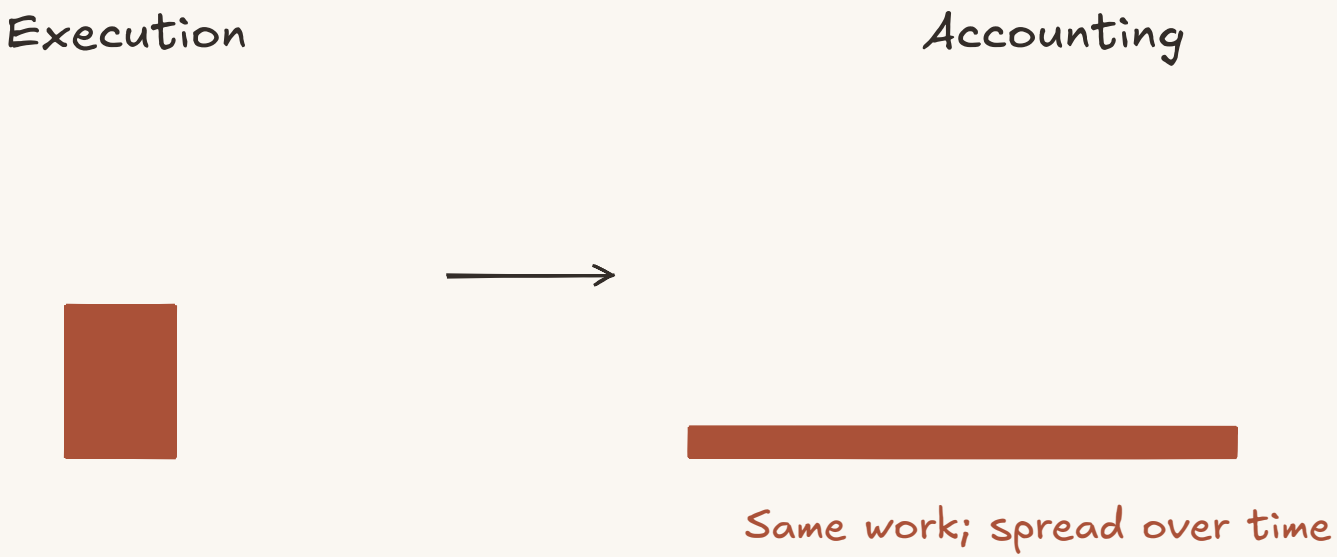


Same work. Longer execution.

A traditional system has a fixed compute ceiling. In this execution comparison, the same work must run below that ceiling, so it takes longer. The red box moves into the smaller graph and spreads horizontally without losing area. This is longer execution, not accounting spread; the next slide contrasts bursting on a Fabric capacity.

Fabric throttling, bursting and smoothing

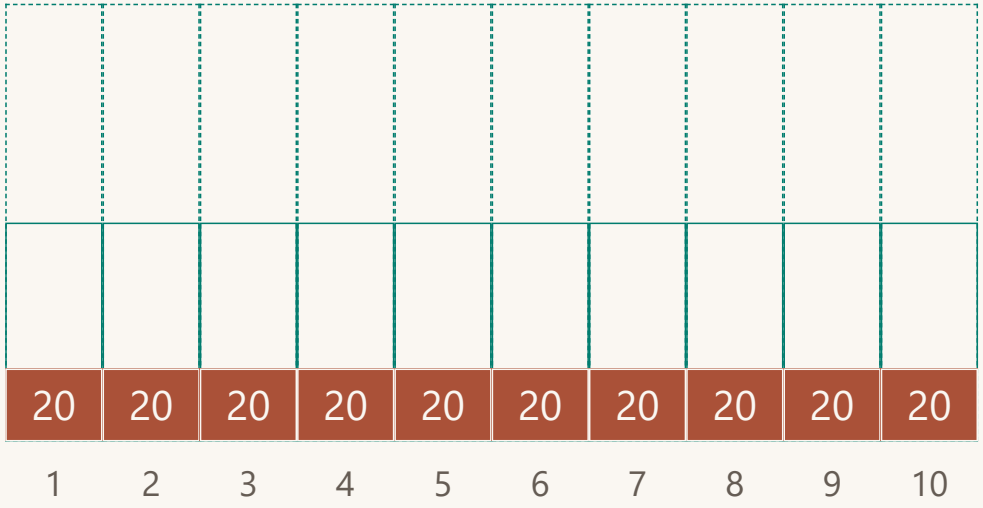
Use now. Account over time.



Bursting is execution; smoothing is accounting. A capacity has no fixed compute ceiling of that kind: through bursting, an operation may use more than the purchased rate while it runs, within workload-specific limits. The same work is then accounted across later timepoints. That is why a capacity can be sized for the average, not the peak.

Fabric throttling, bursting and smoothing

Same work. Ten timepoints.



F4: 120 CU-s

16.7%

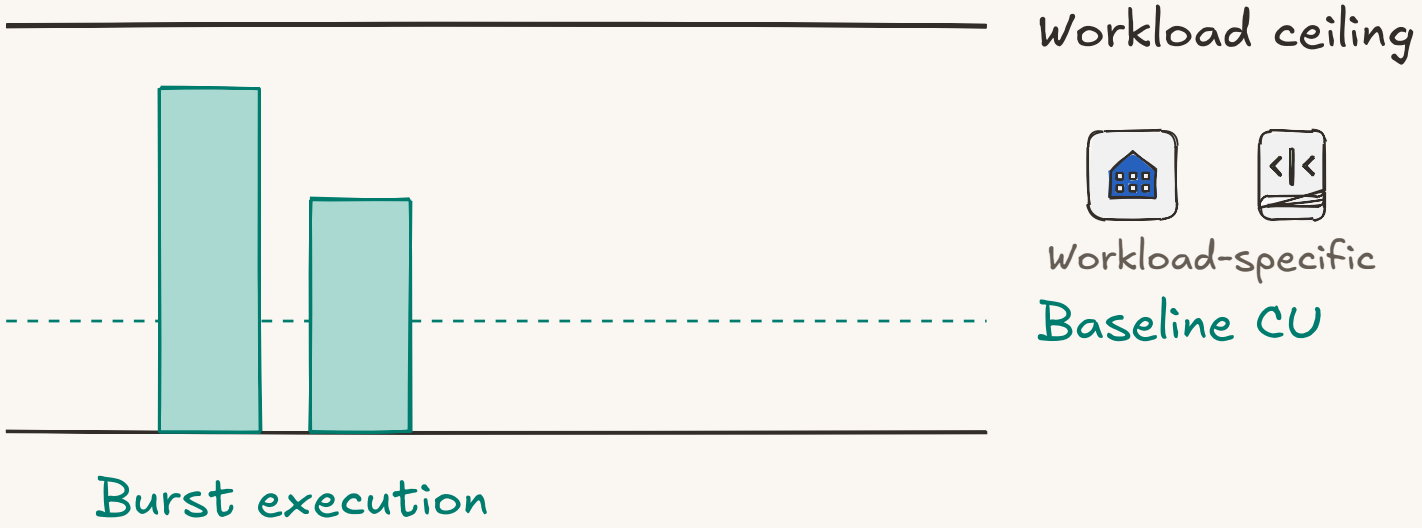
F2: 60 CU-s

33.3%

Using the **5-minute floor**, 200 CU-s / 10 timepoints = 20 CU-s per timepoint. Both alternative budgets stay visible: **solid F2, 60 CU-s, 33.3%**; **dashed F4, 120 CU-s, 16.7%**. They compare the same work, not two charges. Interactive smoothing can extend up to 64 minutes; the example fixes the five-minute floor.

Fabric throttling, bursting and smoothing

Generous. Not infinite.



Bursting is bounded by the SKU and workload. Warehouse and SQL analytics endpoint document F2 up to 32 times baseline and F4 up to 16 times (both 64 CU), and F8 through F2048 up to 12 times. These figures do not transfer to other workloads. Chapter six applies the limits to sizing; chapter nine uses them in the capacity design.

Warehouse / SQL endpoint burst guardrails

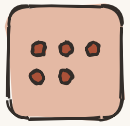
What are the smoothing windows?

Interactive

5-64 min (variable)



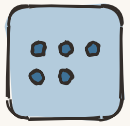
Query



XMLA read

Background

24 hours



Refresh



Notebook

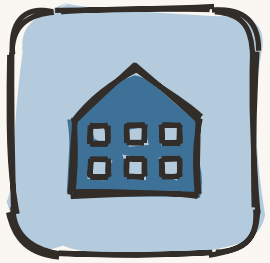


Pipeline

Interactive work is smoothed over a variable **5-minute minimum to 64 minutes**; background over **24 hours**. These are operation classifications, not permanent labels on artifacts or a human-triggered versus unattended test. A model read and a model refresh belong on different sides.

Fabric throttling, bursting and smoothing / Fabric operation classifications

The platform decides.



Warehouse query → 24 h

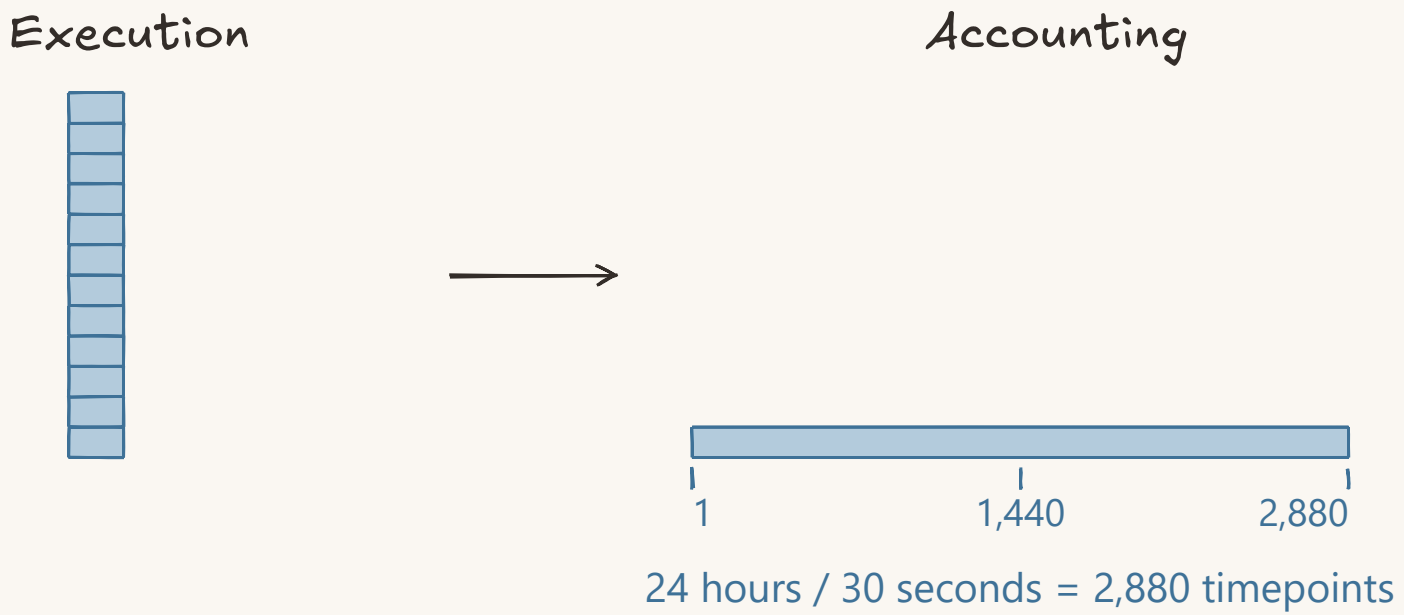
Background

Classification is a platform policy.

Fabric reports almost all Warehouse operations as background to take advantage of 24-hour smoothing. User intent alone cannot classify them. Ambiguous initial throttling decisions favor background, and a later classification can differ when the platform has better information.

Fabric throttling, bursting and smoothing / Fabric operation classifications

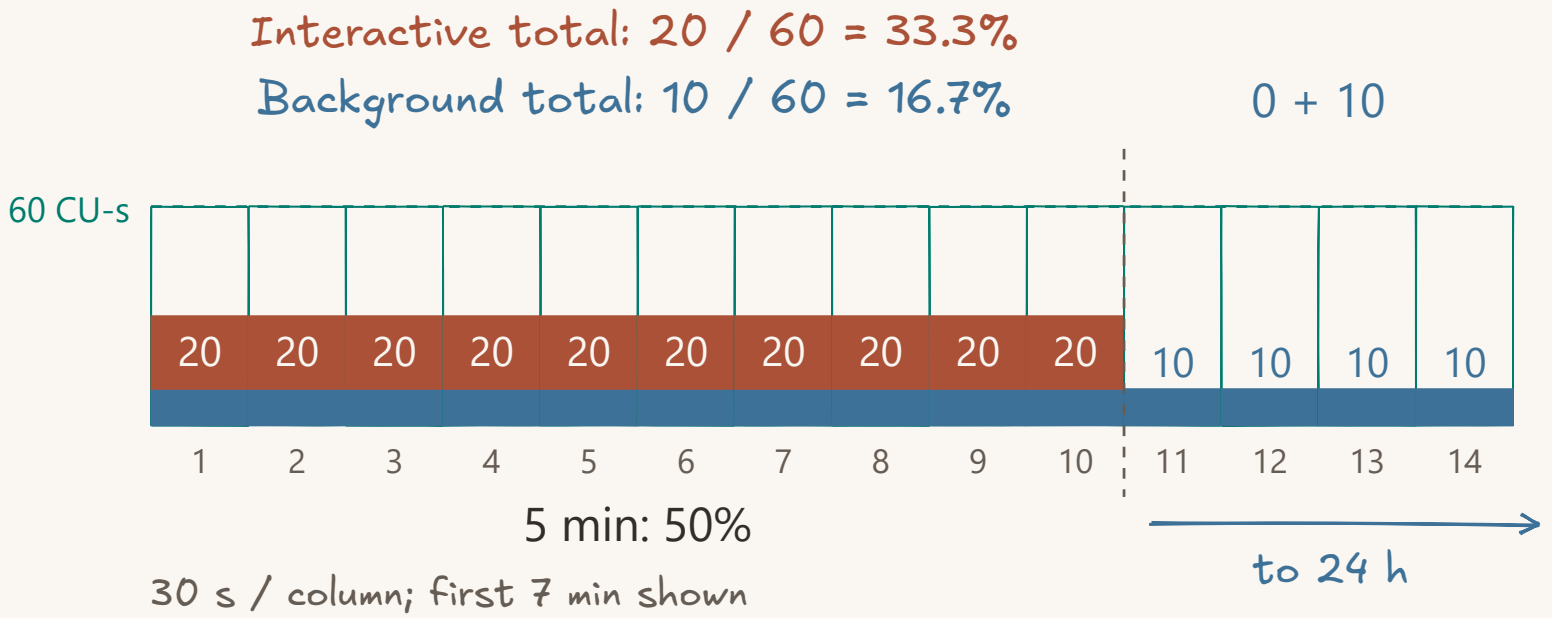
More work. A much longer window.



The illustrative refresh uses **28,800 CU-s** (144 query totals). Its continuous accounting band spans **timepoints 1 through 2,880**: 2,880 x 30 seconds = 24 hours. The axis labels are reference marks, not individual drawn buckets. Work is conserved within the schematic, whose separate scale does not imply a measured runtime or a proportional comparison with the earlier query.

Fabric throttling, bursting and smoothing

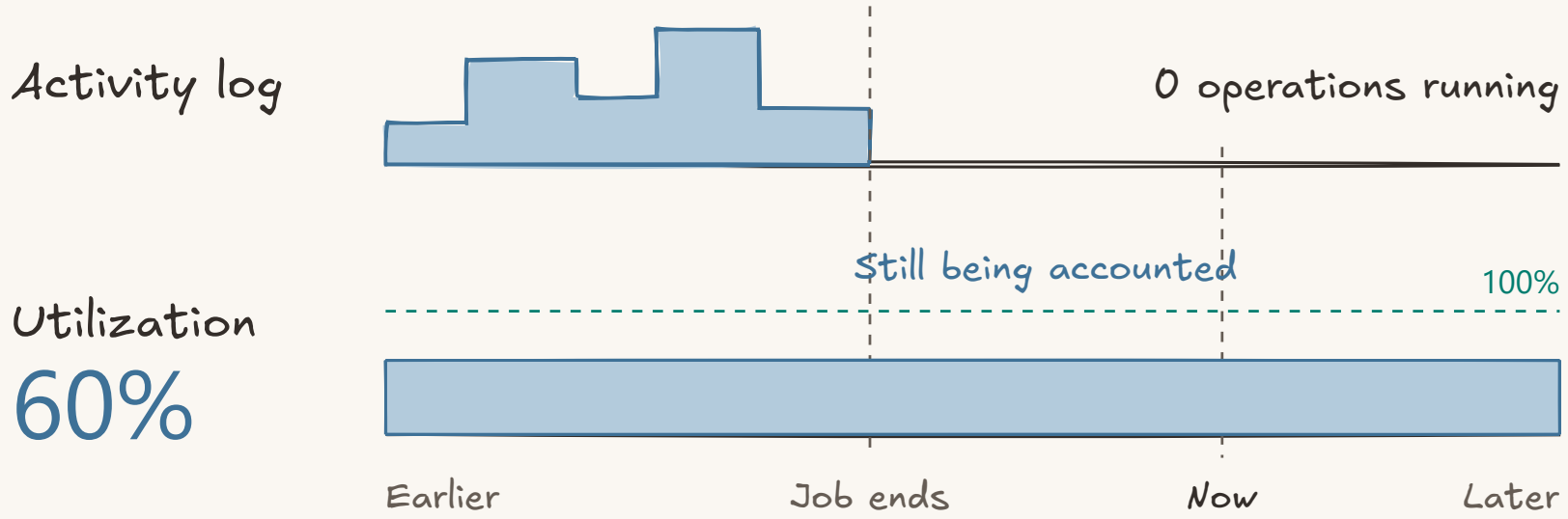
They share the same budget.



28,800 CU-s / 2,880 timepoints = **10 CU-s per timepoint**, 16.7% of F2. The query contributes 20 CU-s (33.3%). Despite 144 times the total work, the refresh has half the per-timepoint footprint.

They land in the same place. Each 30-second column aggregates **interactive 20 / 60 = 33.3%** and **background 10 / 60 = 16.7%**, totaling 50%. After five minutes the interactive installments end; background remains at 16.7%. Only seven minutes are

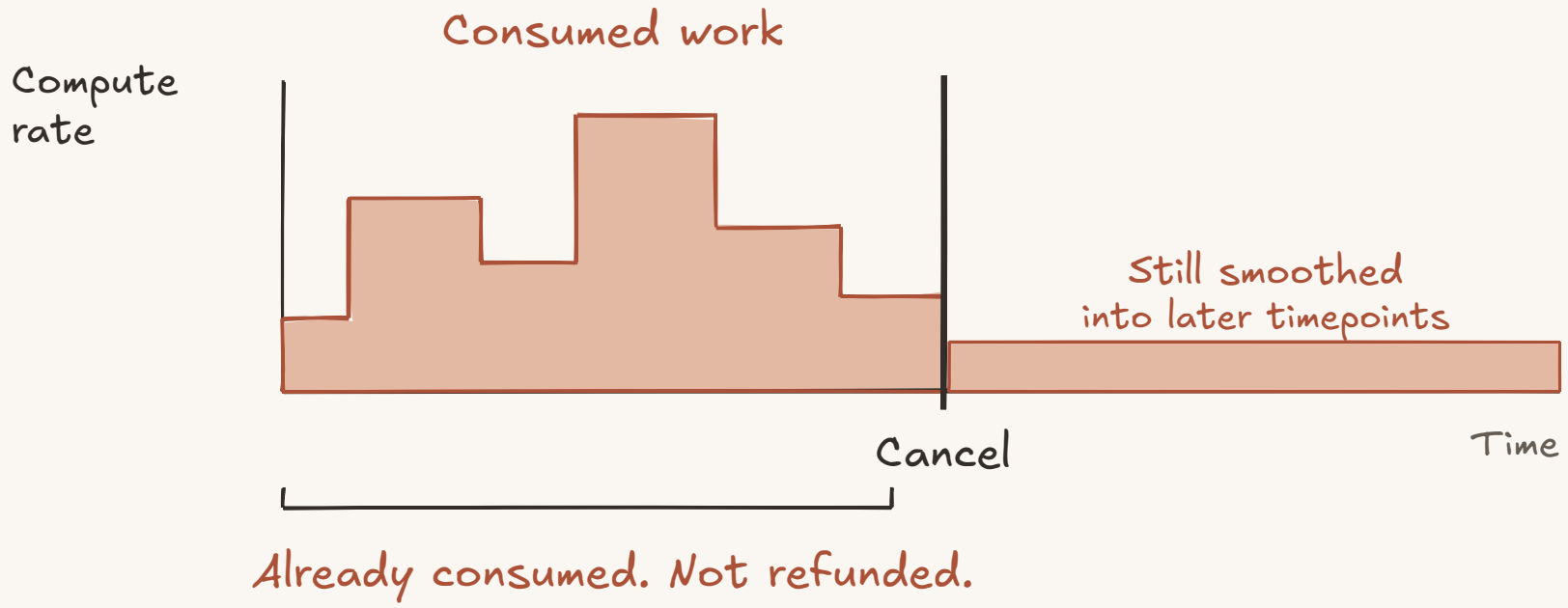
The job ended. The accounting didn't.



Accounting is not activity. The opening picture, zoomed out: the activity log shows a job that ended before now; smoothed utilization continues past it. The rows share time, not a vertical scale. Completed work can remain distributed across later timepoints even with zero operations running. The 60% illustration is an independent puzzle, not a tenant trace or the 16.7% refresh in the running example.

Fabric throttling, bursting and smoothing / Capacity Metrics app: utilization, throttling and overages

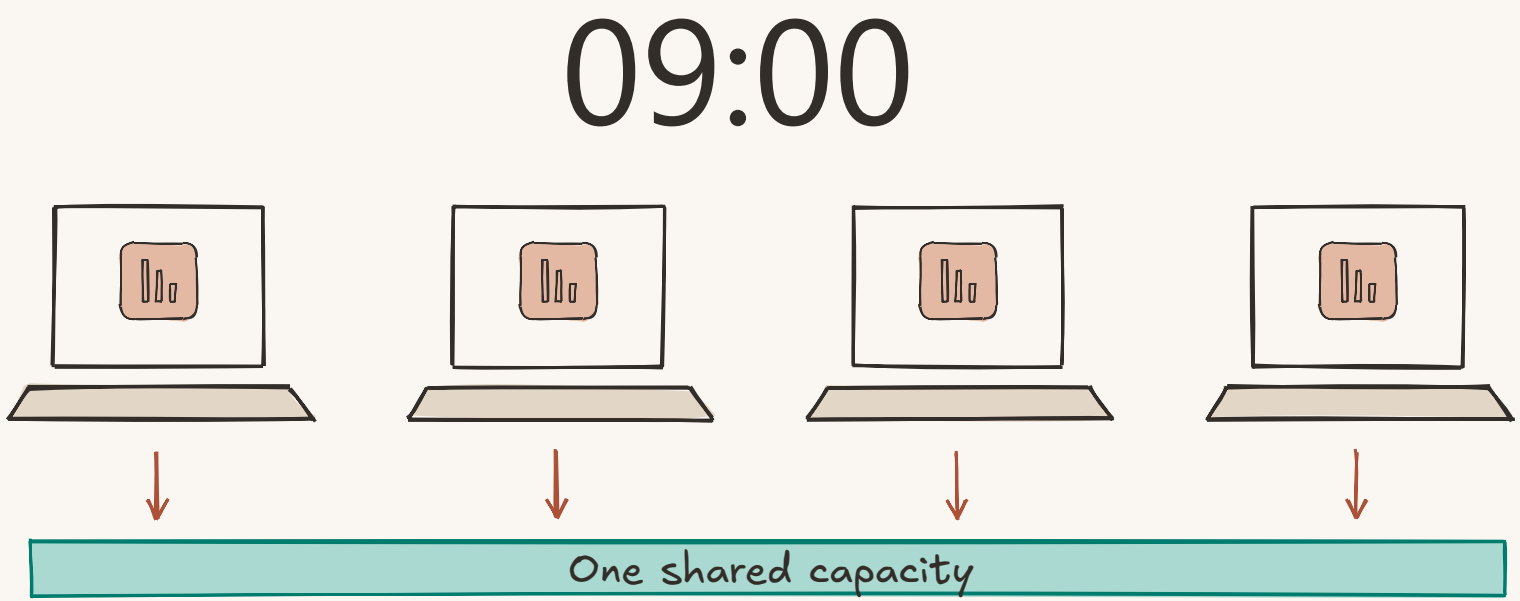
Cancel is not undo.



Cancelling an operation can stop further work. It does **not** erase work already consumed: that work is still accounted for and still smoothed into later timepoints. This is why cancellation may not immediately improve the utilization chart, even though it can still prevent additional consumption.

Fabric throttling, bursting and smoothing / Warehouse compute, smoothing and throttling

Monday morning.

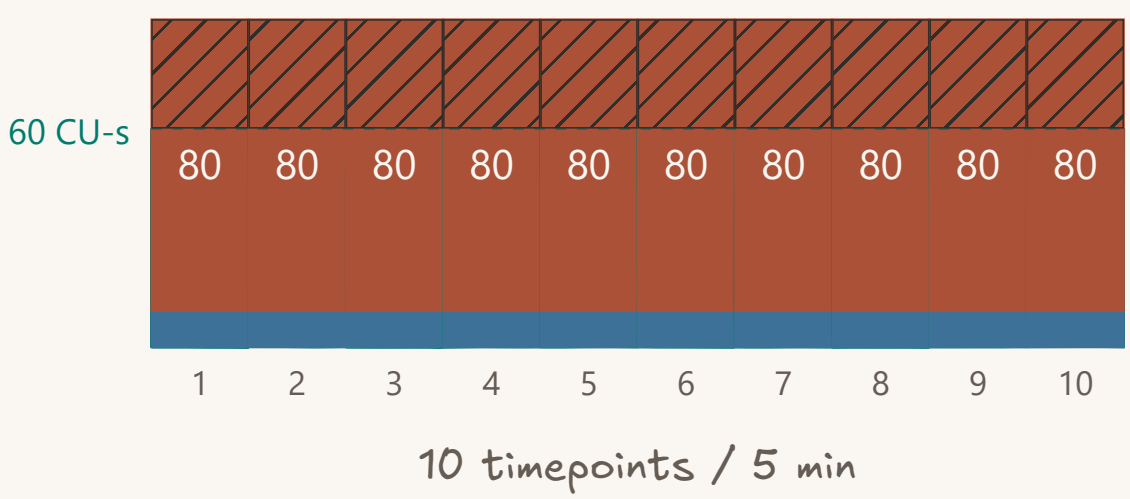


Monday morning creates synchronized demand: many people open reports at once. The worked ledger zooms into **one simplified 09:10-09:15 slice** with four overlapping query contributions and no opening overage. It does not extrapolate four queries to an entire company's fifteen-minute workload.

Fabric throttling, bursting and smoothing

Four queries. One budget.

09:10-09:15 80 interactive + 10 background



150%

90 / 60 CU-s

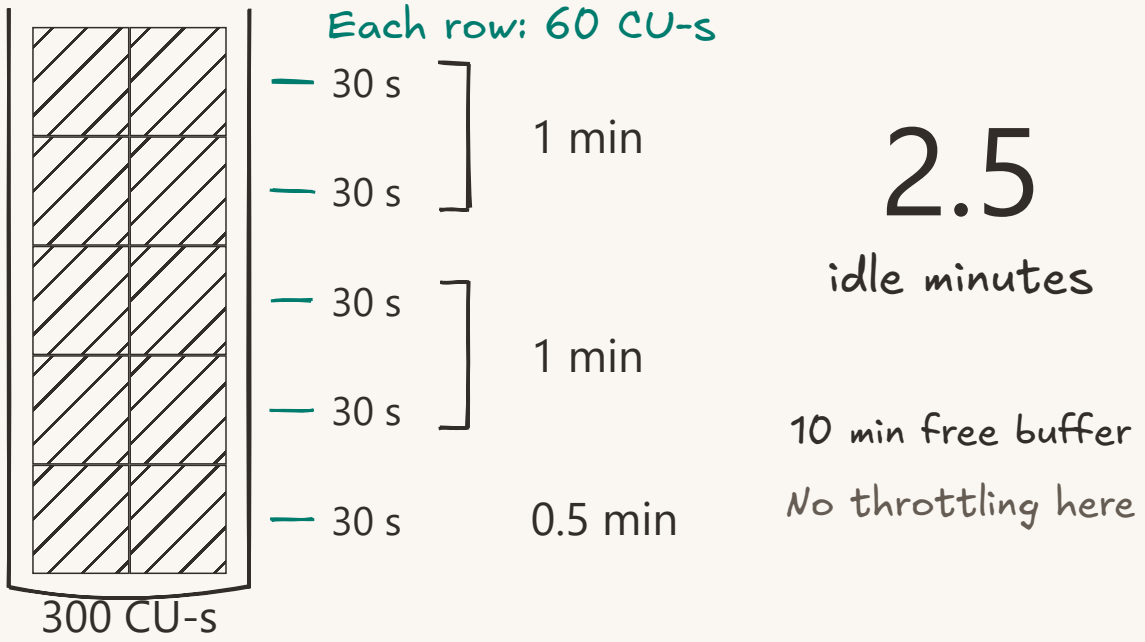
30 excess / tp

During the illustrative 09:10-09:15 slice, the interactive aggregate is **4 x 20 = 80 CU-s** and background is **10**. Total 90 / 60 = **150%**; excess is **30 CU-s** per timepoint. Hatching identifies part of that load, not additional work.

Fabric throttling, bursting and smoothing

Excess becomes minutes of debt.

Average bucket: excess after smoothing



Over ten timepoints, **30 x 10 = 300 CU-s** becomes unpaid usage. The bucket's five rows each hold a full **60 CU-s** budget: **30 seconds** of completely idle F2. Pairing rows gives **2.5 idle minutes**, 25% of the ten-minute free buffer. It is the same balance in different units, not repayment or another charge. Background accounting makes actual recovery longer. Billable overage is off; normal capacity billing continues. The neutral bucket is a teaching schematic; Metrics App Add, Burndown and Cumulative % describe carryforward.

09:15. Coffee.



09:15

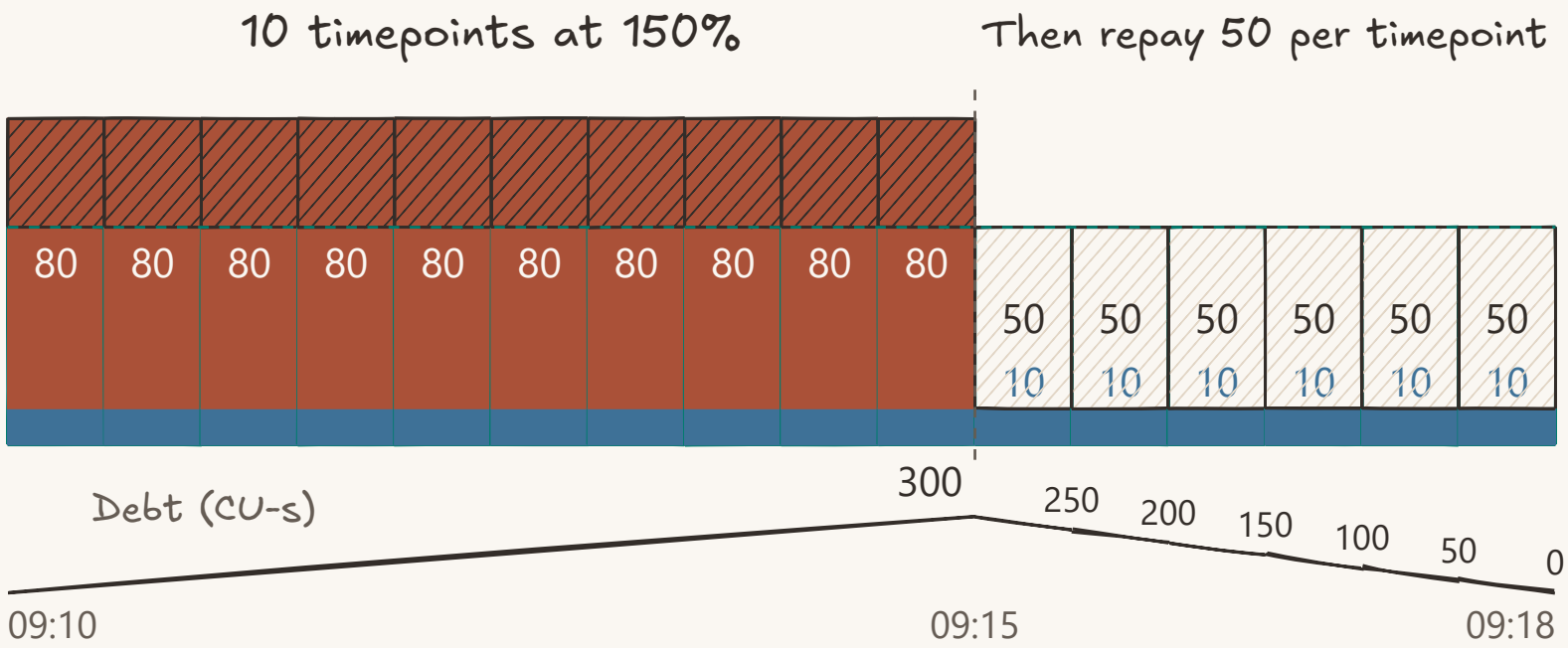
Installments end.

Background stays.

For this model, the last interactive installments finish at 09:15. Background remains **10 CU-s per timepoint**. In real workloads, smoothing tails can outlast the people at their desks; repayment depends on *unused accounting budget*, not merely on operations having stopped.

Fabric throttling, bursting and smoothing

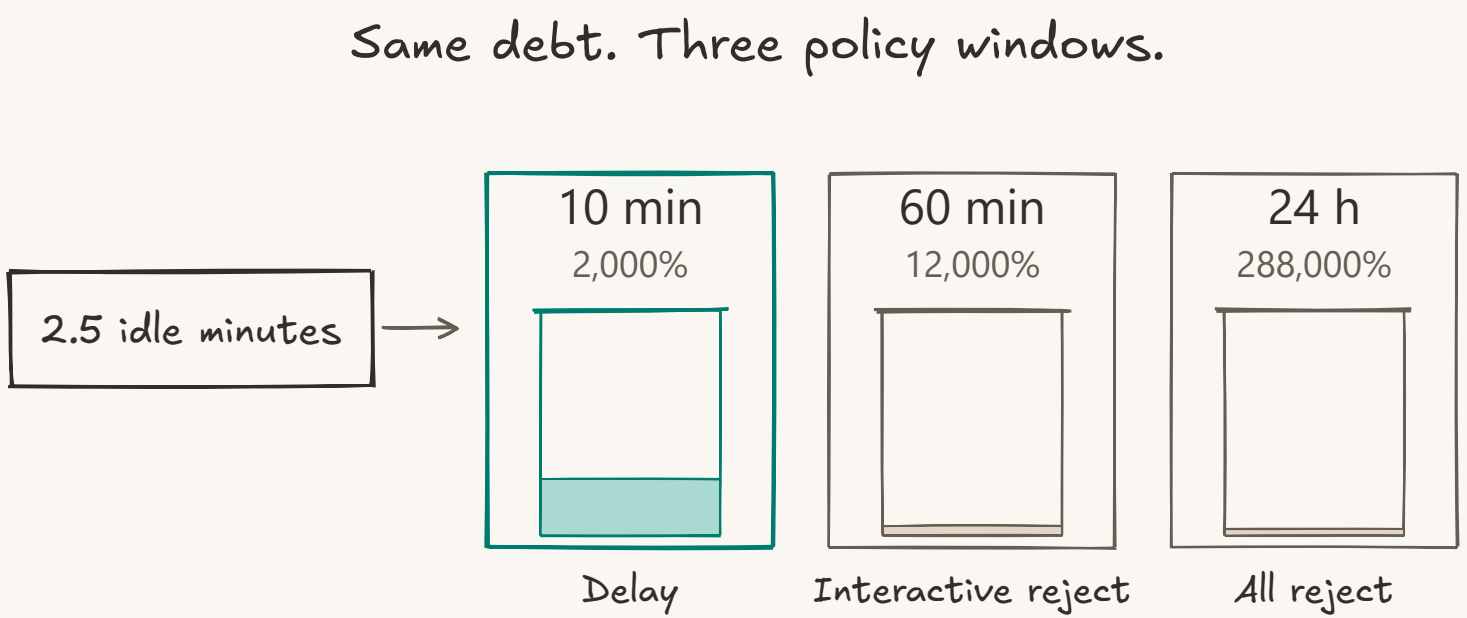
Unused capacity pays it back.



The preceding ten timepoints each account 90 CU-s against 60, accumulating **300 CU-s** of capacity overage. From 09:15, continuing background leaves **60 - 10 = 50 CU-s** per timepoint to repay it. Six timepoints take the balance to zero at modeled 09:18. This three-minute recovery is longer than the 2.5-minute fully idle equivalent.

Fabric throttling, bursting and smoothing

Debt has thresholds.

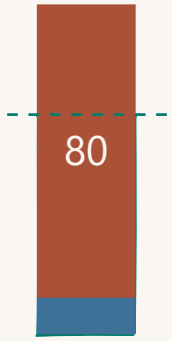


Cumulative overage % = stored overage / one 30-second budget x 100: **10 min = 2,000%; 60 min = 12,000%; 24 h = 288,000%**. The broken axis is not proportional. Baseline policy has no throttling up to 10 minutes; above it, new interactive requests get a 20-second delay; above 60 minutes they are rejected; above 24 hours all new requests are rejected. RTI skips delay; Eventstreams have reduced-resource behavior.

Fabric throttling, bursting and smoothing

Same budget. Different quantities.

Utilization per timepoint

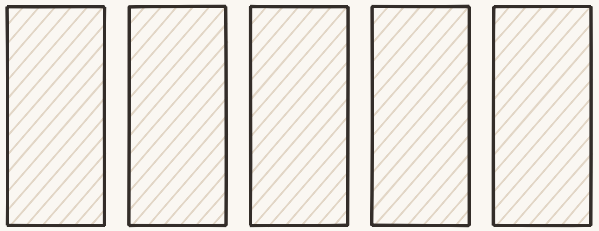


150%

90 used / 60 CU-s

Debt

500%



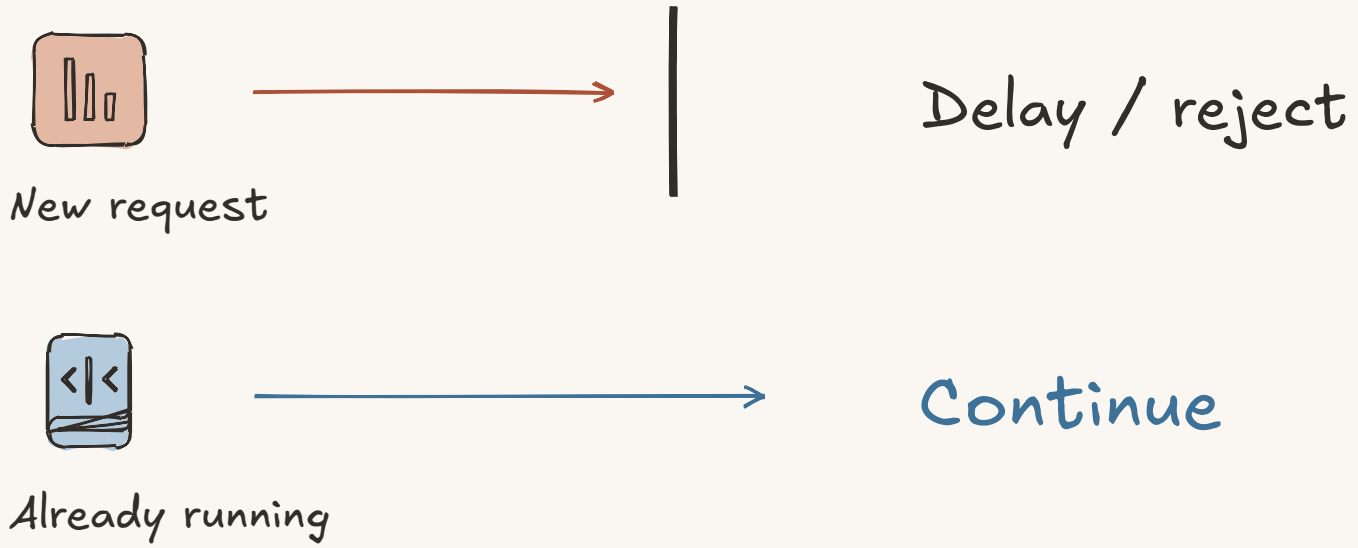
300 owed / 60 CU-s

Both divide by one timepoint's budget.

The example's **150% utilization** is 90 CU-s of work / 60 CU-s in one timepoint. Its **500% cumulative overage** is the stored 300 CU-s / one 60 CU-s budget. As established on the threshold line, 10 minutes is 2,000% on this cumulative scale. Policy-specific Metrics App panels instead normalize their own allowance to 100%; the axes are not interchangeable.

Capacity Metrics app: utilization, throttling and overages / Fabric throttling, bursting and smoothing

Throttle arrivals. Not in-flight work.

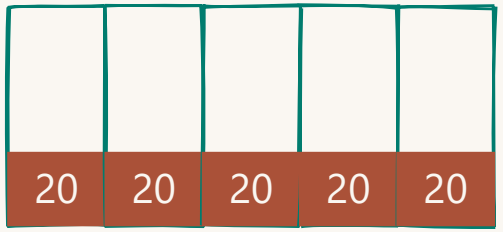


Throttling affects new submissions, not operations already admitted before the capacity began throttling. In-flight work can complete. Only billable operations count toward throttling; visible nonbillable preview usage does not consume the same throttling budget. Workload-specific exceptions still apply.

Fabric throttling, bursting and smoothing

Elastic compute. Hard guardrails.

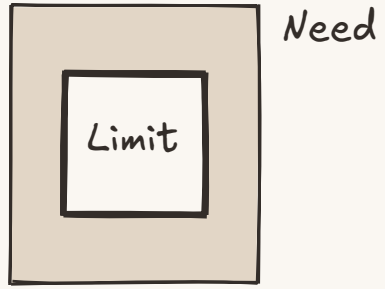
Accounting headroom



Work fits the CU budget

≠

Workload fit



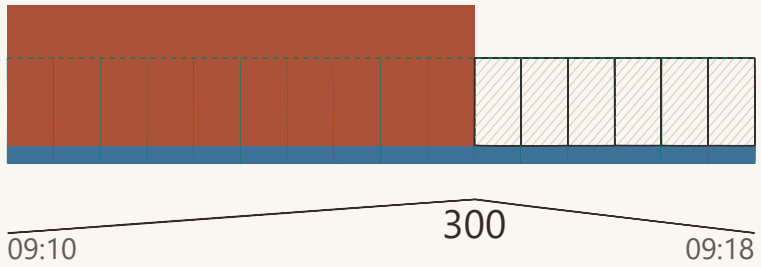
Memory / concurrency / item

Ch09: workload-specific limits

Elastic CU accounting does not bypass applicable workload limits. For example, Power BI documents semantic-model memory limits, per-model concurrent DirectQuery connections and per-item memory constraints. Scope and failure behavior vary by workload; healthy CU utilization does not prove workload fit. Chapter six applies these limits to sizing.

Power BI semantic-model memory, concurrency and item limits / Warehouse / SQL endpoint burst guardrails

Users never noticed. No throttling.



Buffer used. Then repaid.

No throttling.

No extra charge in this example.

You buy the budget — not each burst.

In this worked example, 300 CU-s of carried-forward overage equals 2.5 fully idle minutes, below the ten-minute free buffer. It is repaid without baseline throttling or an extra charge, with billable capacity overage disabled. Normal provisioned-capacity billing still applies. Buying that budget and choosing how to pay for it are the next chapter's questions.

Fabric throttling, bursting and smoothing

03 · How you are billed

You buy the budget. How do you pay?

Running capacity and buying a discount are separate decisions.

This chapter separates the capacity's compute budget from its procurement and licensing. It compares pay-as-you-go with reservations, explains hourly matching and unused reservations, and shows how the SKU also changes Power BI licensing and workload limits.

Fabric capacity reservations — what you buy, scope, exchange and refund

Two ways to pay for a capacity

Pay-as-you-go

$0.22

per CU-hour

no commitment

F64: $14.08 an hour ÷ 64

Reservation

$1,146

per CU-year

1 or 3 years

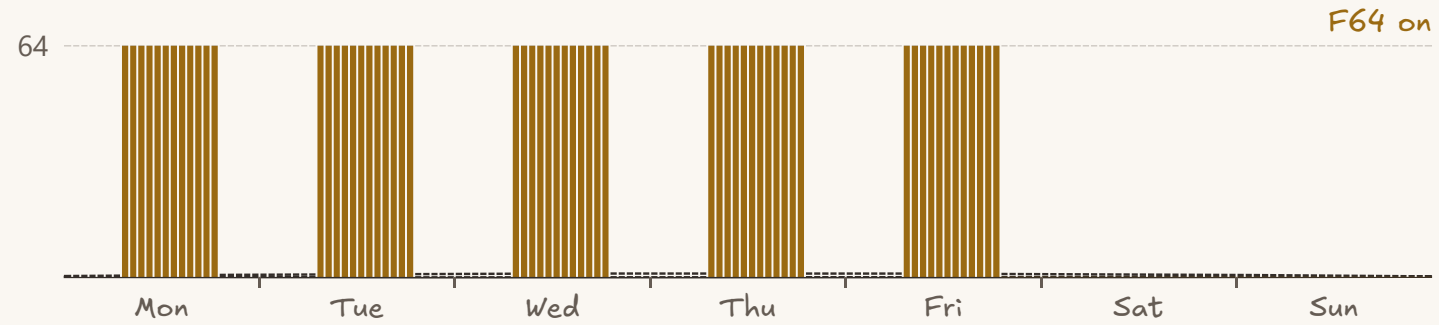
F64: $73,344 a year ÷ 64

$0.22 × 8,760 h = $1,927 a CU-year · reserved: 41% less

A Fabric capacity can be paid for in two ways. Pay-as-you-go charges the list price per CU per hour for as long as the capacity runs, with no commitment: $0.22 in West Europe. A reservation commits to a quantity of CU for one or three years at $1,146 per CU per year, and discounts matching usage. An estate can use both at once.

Both per-CU prices come from the SKU price divided by its CU: an F64 costs $14.08 an hour on pay-as-you-go and $73,344 a

Pay-as-you-go: you pay while it is on



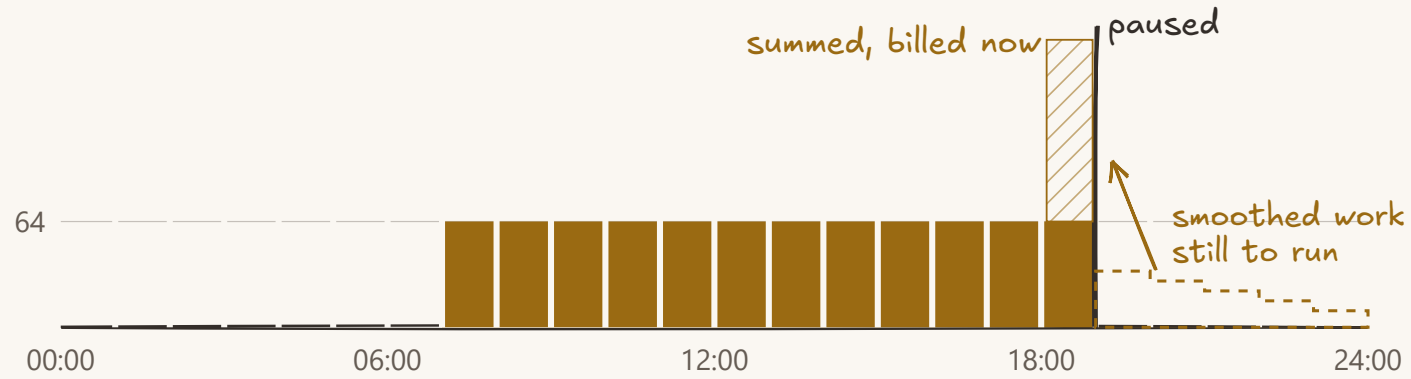
60 h × 64 CU × $0.22 = $845 this week

idle or busy — the same price

On pay-as-you-go a capacity is billed per second for as long as it is switched on, at the full SKU size, whether or not any work runs on it. An F64 switched on twelve hours each weekday is billed 60 hours × 64 CU × $0.22 = $845 a week. When it is off, the meter stops.

It works like a mobile phone plan: a fixed fee buys a fixed allowance, and the fee is the same whether the allowance is used or

Switching off is not where the bill stops



⚠ The catch. What it costs: chapter seven, Rescue.

Pausing a capacity does not end its charges at the moment of the pause. Fabric smooths work over a window of up to twenty-four hours, so at any moment some already-run work is still being accounted for. When a capacity is paused, the remaining cumulative capacity overage and smoothed operations are summed and added to the Azure bill in one charge. A pause schedule's saving is therefore the hours not run minus that settlement at each pause. The cost of the settlement and how to manage it are covered in chapter seven.

Now the other way to pay

		True	False
1	A reservation is attached to a capacity.	True	False
2	One reservation covers one capacity.	True	False
3	Your reservation has to match your SKU.	True	False

Three widespread beliefs about Fabric capacity reservations, all of them false: that a reservation is attached to a capacity, that one reservation covers exactly one capacity, and that a reservation must match the SKU you run. The reservation section takes them in order.

Fabric capacity reservations — what you buy, scope, exchange and refund

A reservation changes the price. Not the capacity.

Purchase a reservation

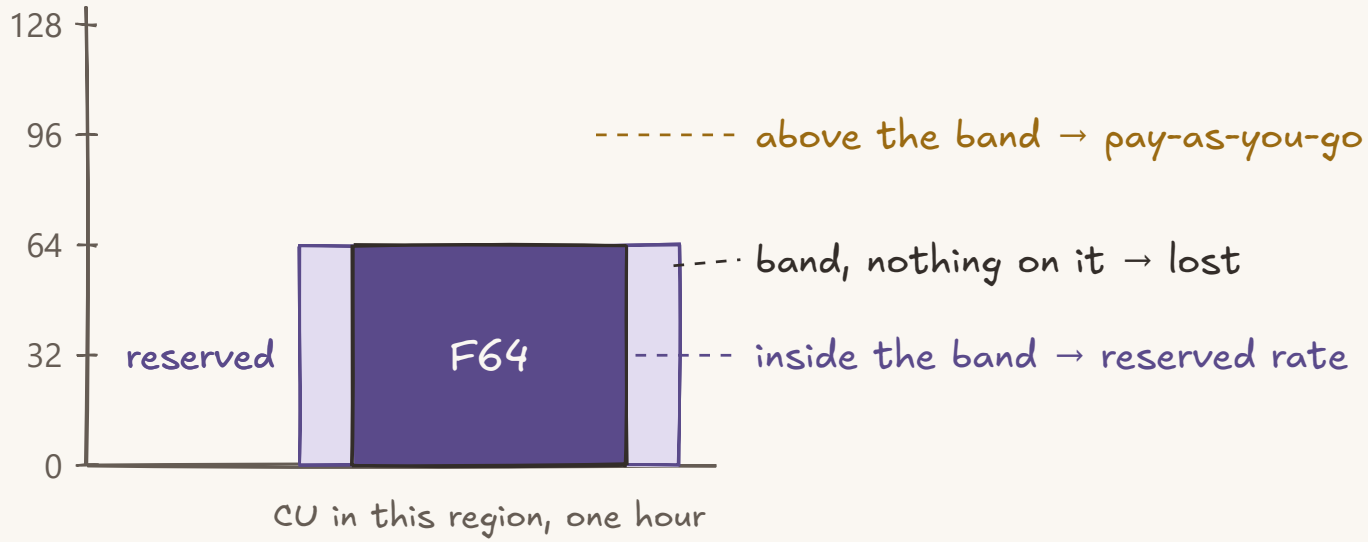
Region	West Europe
Term	1 year or 3 years
Billing frequency	upfront or monthly
Quantity	CU, in 1-CU increments

~~Capacity~~ ~~no such field~~

A Fabric capacity reservation is a purchase of a quantity of CU in a region, for a one- or three-year term, in single-unit increments. It does not reference a capacity and does not provision any compute. Running capacities in the region keep emitting usage as before; usage that matches the reservation is billed at the reserved rate instead of the pay-as-you-go rate, and the reservation itself appears as a separate purchase on the invoice.

Fabric capacity reservations — what you buy, scope, exchange and refund

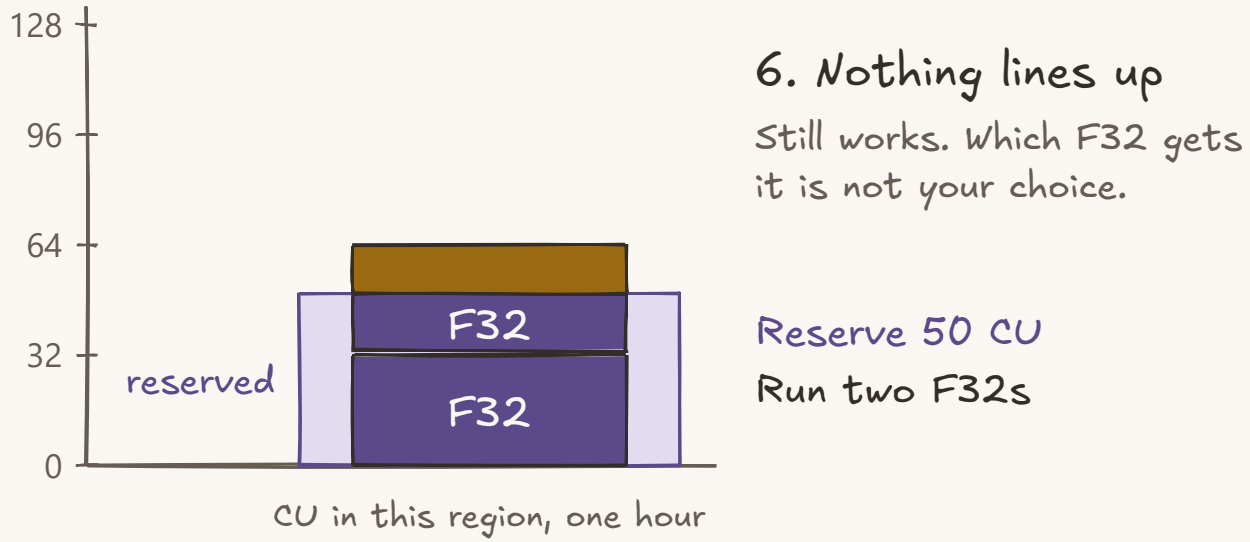
Two pools, laid over each other once an hour



Every hour, Fabric lays two pools over each other: the discount pool, which is every reserved CU in the region, and the consumption pool, which is every running capacity CU in the region. Usage inside the reserved quantity is charged at the reserved rate. Usage above it is charged at pay-as-you-go. Reserved CU with nothing running against it is lost for that hour and does not carry over.

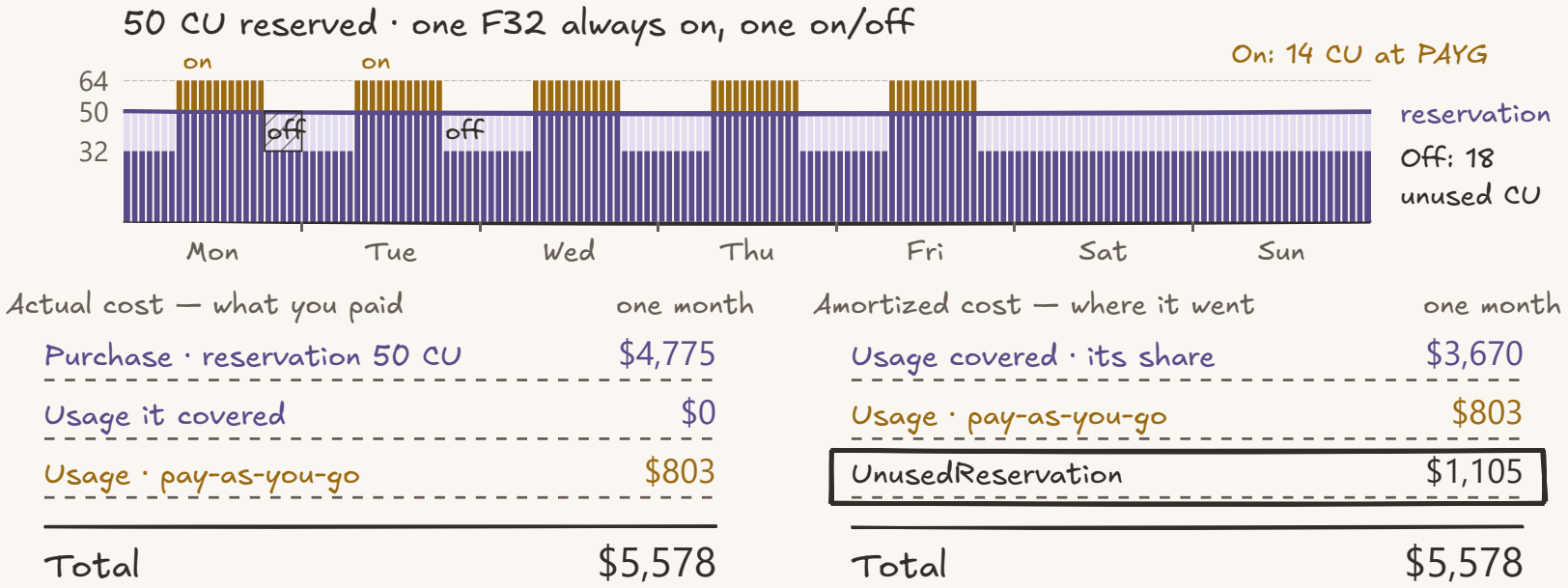
Fabric capacity reservations — what you buy, scope, exchange and refund

Six states of one column



Six states of one reservation column. (1) Reserve 64, run one F64: an exact match. (2) Reserve 64, run two F32s: one reservation covers several capacities. (3) Two reservations of 32 under one F64: reservations add up. (4) Reserve 64, run an F128: usage above the reservation bills at pay-as-you-go, which is usually the cheaper design for a stable baseline with peaks. (5) Reserve 64, run an F32: the unmatched part of the reservation is lost every hour, and scaling down does not reduce the reservation bill. (6) Reserve 50, run two F32s: 50 CU are covered each hour and 14 bill at pay-as-you-go; which capacity receives the discount is not

What the reservation looks like on your bill



A reservation appears in Azure Cost Management in two views of the same invoice. In **Actual cost**, the default and the view that reconciles with the bill, the reservation is a Purchase charge, usage it covers is priced at zero, and usage above it is billed at pay-as-you-go. In **Amortized cost**, the purchase is spread over the usage that consumed it, and a separate UnusedReservation charge type shows the value of reserved hours nothing used. For a 50 CU reservation under one F32 running continuously and a second on twelve hours each weekday, one month reads: Actual — purchase $4,775, covered usage $0, pay-as-you-go $803;

The pools only meet inside a scope

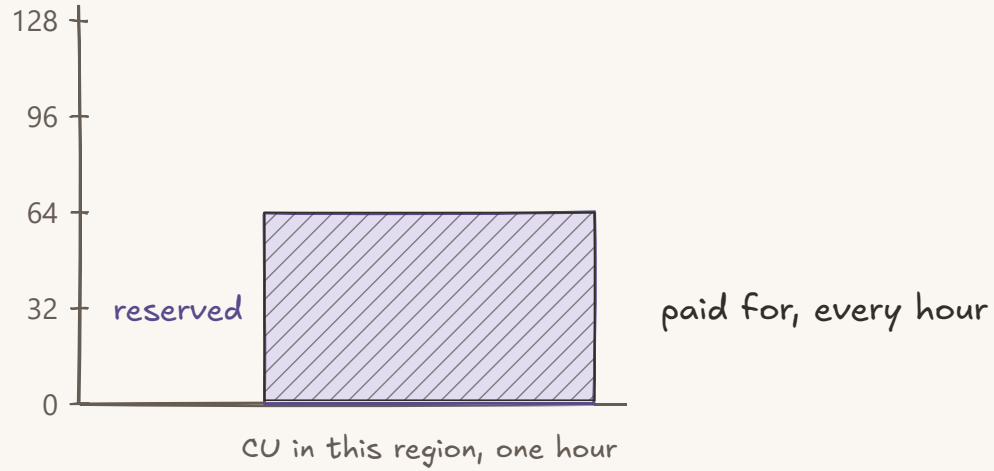
Chosen at purchase, changeable after: **resource group · subscription · shared · management group**.

Left on one subscription by accident, the discount silently under-applies.

A reservation's scope is selected at purchase and can be changed later: a single resource group, a single subscription, shared across the whole billing context, or a management group. Shared scope is what allows one reservation to cover capacities in several subscriptions. Moving a subscription into a different billing context drops the reservation benefit for that subscription.

Fabric capacity reservations — what you buy, scope, exchange and refund

Paid for, whether anything runs on it

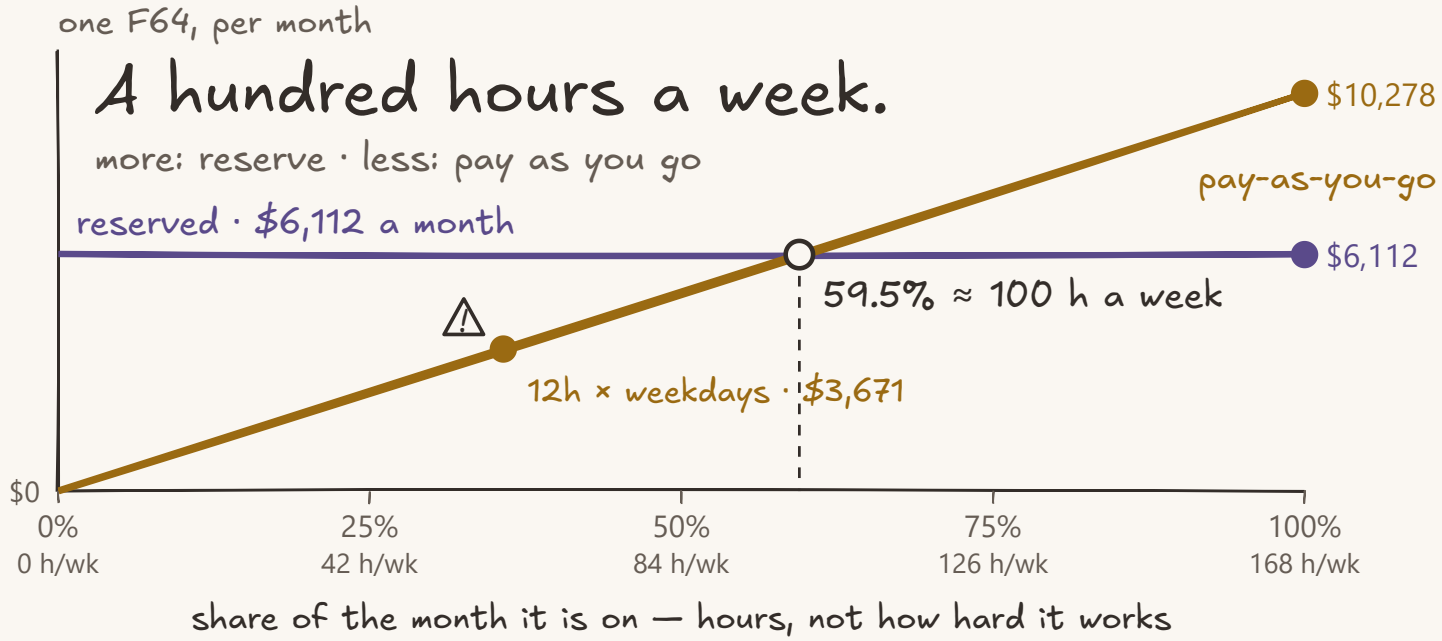


Deploying below your reservation never makes sense.

A reservation is billed for its full term whether or not anything runs against it. The discount is applied hour by hour and does not carry over: an hour in which no running CU matches the reservation is lost permanently. Running less than the reserved quantity therefore saves nothing, and a reservation is better read as a commitment to keep at least that much compute deployed for the term.

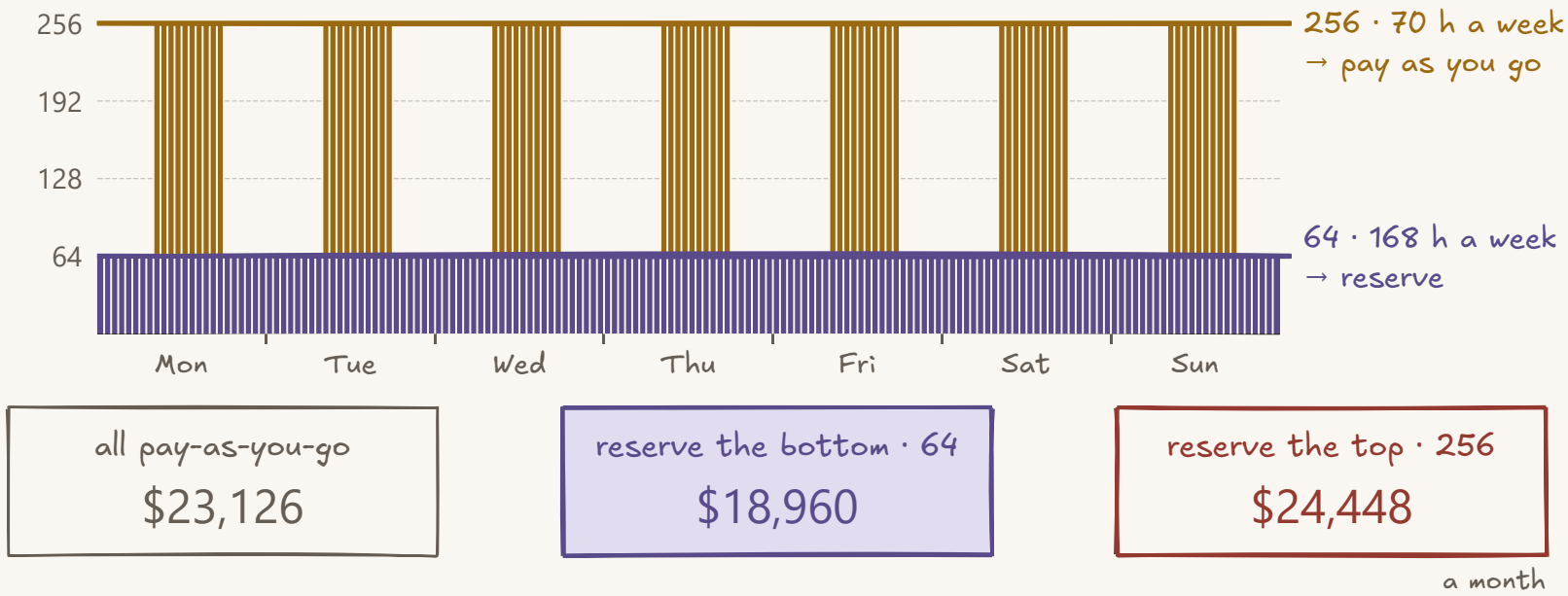
Fabric capacity reservations — what you buy, scope, exchange and refund

When is what cheaper?



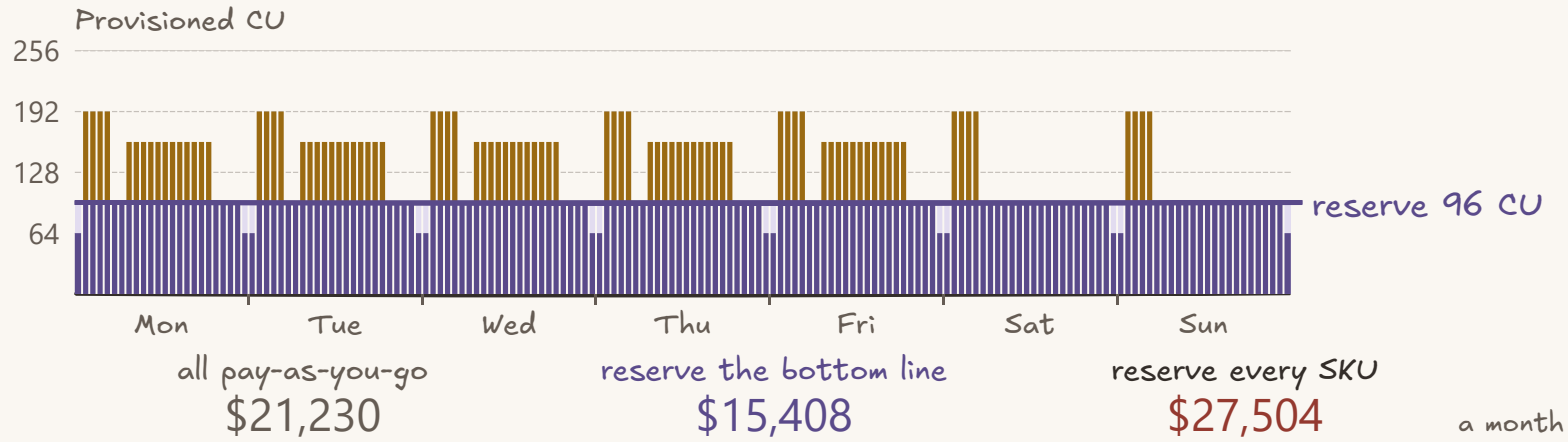
The monthly cost of one F64 in West Europe: reserved, a flat $6,112; on pay-as-you-go, rising in proportion to the share of the month it is switched on, up to $10,278 if it never stops. The lines cross at 59.46% of the month, or 99.9 hours a week: a capacity on for longer than that is cheaper reserved, one on for less is cheaper on pay-as-you-go. The measure is hours switched on, not utilisation; an idle running capacity counts in full. Paused to twelve hours on weekdays without a reservation, an F64 costs $3,671 a month in running hours, before the settlement charged at each pause.

The third way: reserve the bottom



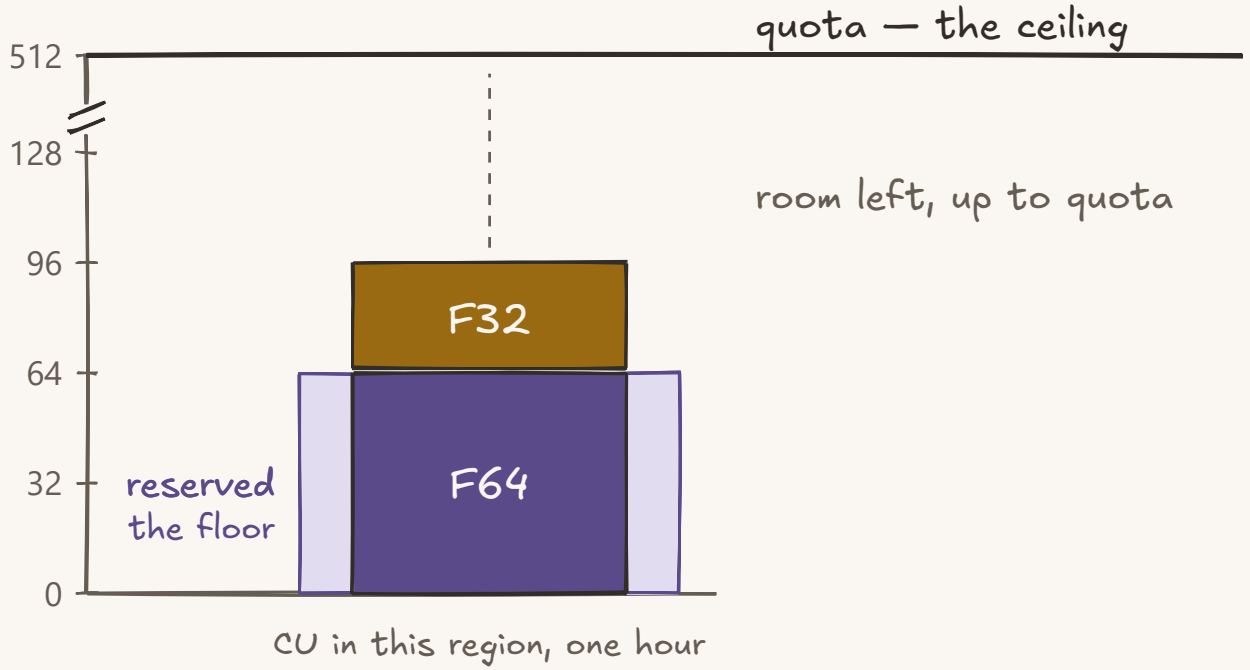
The practical answer is neither pure pay-as-you-go nor a reservation matching the SKU, but a mix: reserve the level of CU the estate holds for more than about a hundred hours a week, and pay as you go for everything above it. For an F64 that scales to F256 from 08:00 to 18:00 every day, the first 64 CU run all week and the next 192 run seventy hours a week. Per month: all pay-as-you-go $23,126; reserving 64 CU $18,960; reserving all 256 CU $24,448, more than reserving nothing. An intermediate reservation is never cheapest, because cost is linear between the levels an estate actually runs at. The peak layer only becomes

Your estate: where is the bottom line?



The desk combines five provisioned-capacity schedules: a production F64 always on, a BI F64 on office hours, nightly ETL F128, a dev F32 on eighteen hours daily, and a second production F64 always on. With the first four selected, the cheapest reservation is 96 CU at $15,408 a month; adding the second production capacity moves it to 160 CU. The part-time F32 moves the reservation because it exists for 126 hours a week, above breakeven. All 31 nonempty combinations compare all pay-as-you-go, the cheapest reservation and reserving every SKU. These are provisioned CU-hour prices, before pause settlements, not peak

A floor you bought. A ceiling someone else set.



A Fabric quota is the maximum total CU deployable across capacities in one subscription in one region, and it varies by subscription type. It sits on the same axis and in the same unit as a reservation but works in the opposite direction: the reservation is a floor committed to, the quota a ceiling someone else set, and the capacities run between them. The quota drawn here, 512 CU, is illustrative; no default is documented.

Microsoft Fabric quotas

You cannot deploy past it. You can ask.

No capacity can take a subscription past its quota, per region.

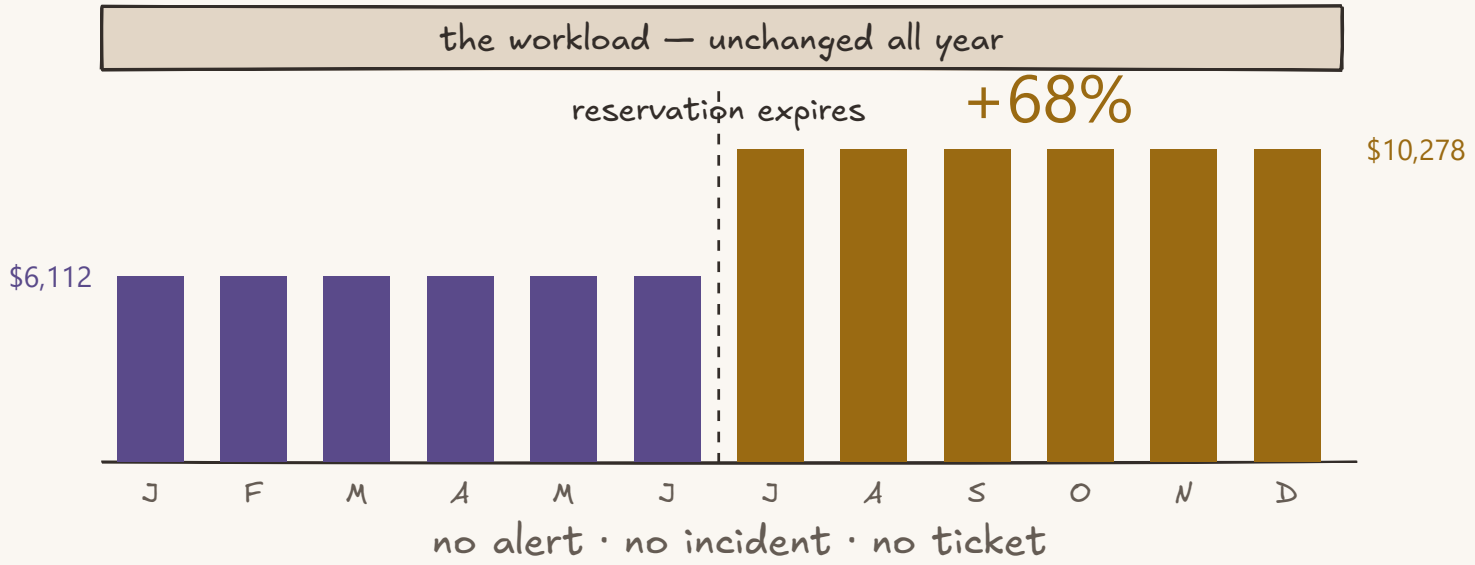
Need more? **Request a raise.**

In constrained regions, a raise is **not guaranteed**.

Check quota **before reserving**: reserve only deployable CU.

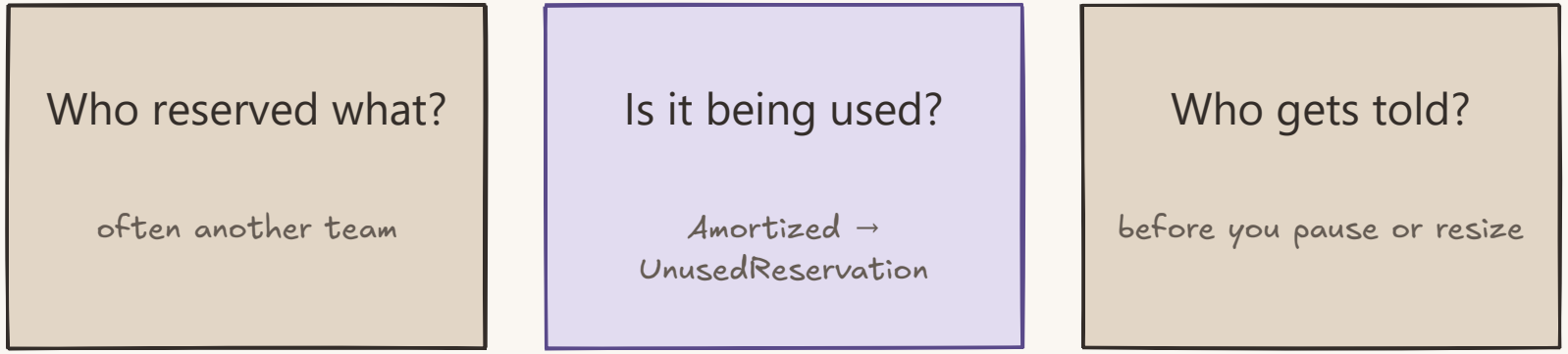
A Fabric quota caps the CU a subscription can deploy in one region. Check the granted quota in the subscriptions and region intended to consume a reservation before purchasing it; do not commit to reserved CU you cannot deploy and use. A quota increase can be requested through Azure Quotas or support, but is not guaranteed, especially in constrained regions. An unapproved request is not deployable headroom. Quota is not billed; zero usually means the Microsoft.Fabric resource provider is not registered.

Your reservation expires. Nothing breaks.



An expiring reservation breaks nothing: the workloads keep running and are re-billed at pay-as-you-go, raising one F64's monthly compute bill from $6,112 to $10,278, about 68%, with no alert. Auto-renew exists but is off by default. A reservation can be returned for a prorated refund issued as Azure credit, subject to a cap of USD 50,000 of cancelled commitment per billing scope in a rolling twelve-month window. There is no resize: increase by exchanging, or by buying an additional reservation in the same region.

Reservations are someone else's job



Reservation management is often owned by a different team from the one that runs capacities, such as FinOps or procurement. Three questions keep the two aligned: what has been reserved and by whom; whether it is being used, visible as the UnusedReservation charge in Amortized cost; and who is told before a capacity is paused, resized or removed. Buying a Fabric capacity reservation requires the Owner or Reservation purchaser role on the billing subscription. On an Enterprise Agreement the Reserved Instances policy must be enabled before anyone other than an enterprise administrator can buy; under a Cloud

Remember the licensing boundary

Power BI < F64	Power BI F64+	Other Fabric items
Pro per viewer	Free + Viewer	No paid per-user licence

SKU guardrails → Design

The F64 boundary is a Power BI consumption rule. At F64 or larger, on Premium capacity and on a trial capacity, a Free licence plus the Viewer role allows Power BI viewing; below F64, and on A SKUs, viewers need Pro or PPU (or an applicable trial). Creating or sharing Power BI content, higher workspace roles and Pro/PPU workspace types retain their paid-licence requirements. Non-Power BI Fabric items can be created and shared using Fabric Free on any F SKU, including F2, with appropriate workspace permissions; no paid per-user licence is required. Separately, SKU size sets workload-specific guardrails

You picked three policies, and meant one

a compute commitment	$6,112 a month reserved, $10,278 not
a licence boundary	where your viewers stop needing Pro
a guardrail step	Direct Lake gets five times the rows

Choosing an F64 sets three unrelated policies at once: a compute commitment of $6,112 a month reserved against $10,278 on pay-as-you-go if it never stops ($73,344 against $123,341 a year), a licensing boundary beyond which Power BI viewers no longer need a Pro licence, and a guardrail step that raises the Direct Lake and model-size ceilings. Most estates choose the size for one of those reasons and inherit the other two.

Microsoft Fabric licenses

04 · Monitoring

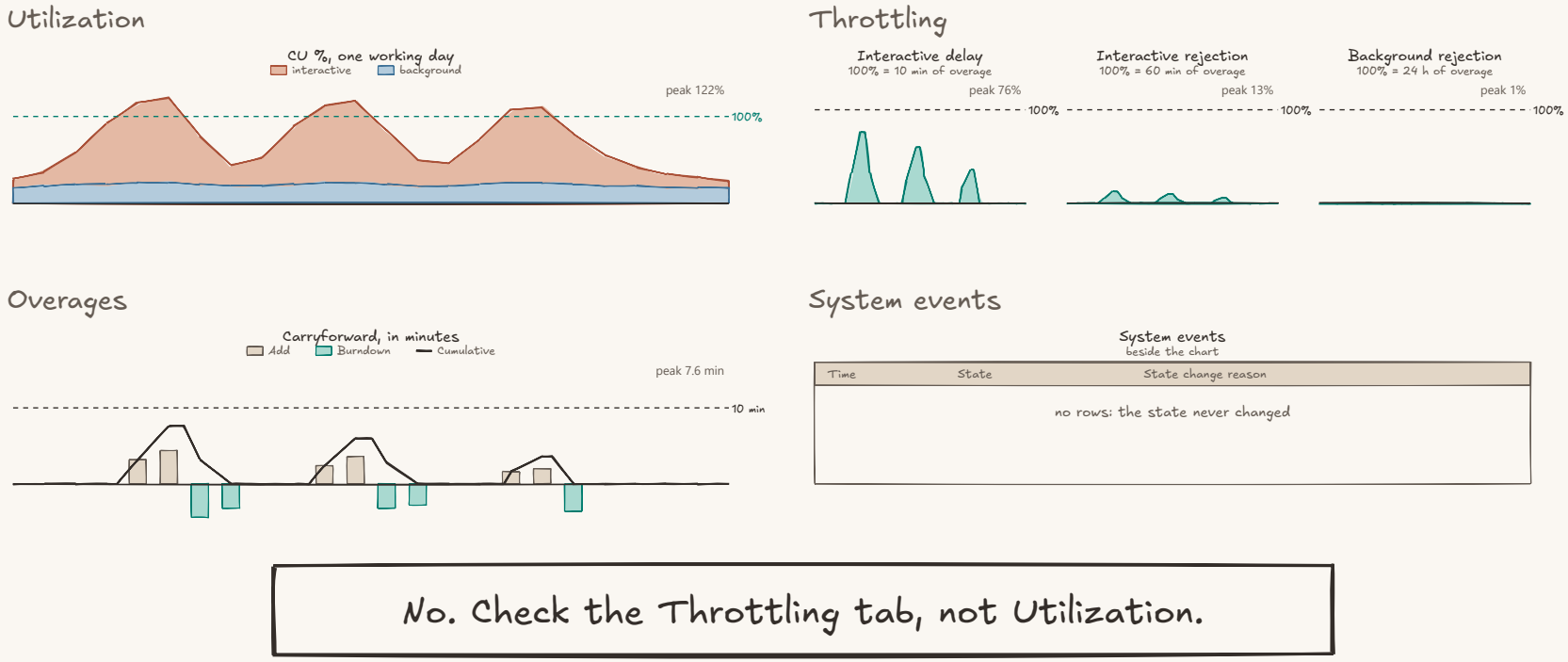
Enough about money.

Now the real work: watching your capacity run.

Monitoring: where to look. After how a capacity is billed comes how to watch one run: which tools show a capacity's consumption and throttling, what each one keeps, and how to tell a capacity that is merely busy from one that is in trouble. This part is about where to look; what the patterns mean follows in the next part.

Use the Microsoft Fabric Capacity Metrics app

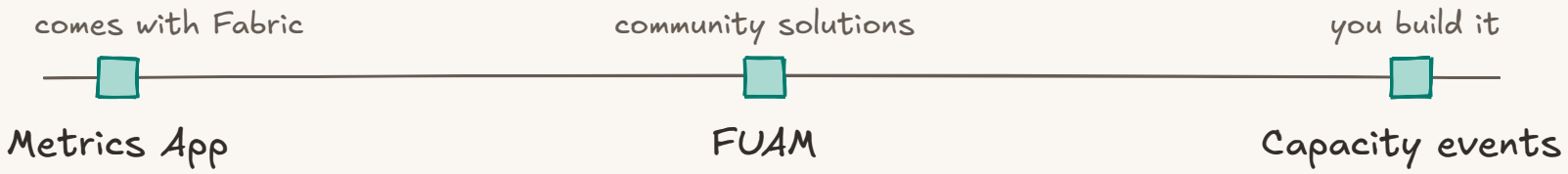
Is this capacity in trouble?



No. Check the Throttling tab, not Utilization.

The Compute page of the Fabric Capacity Metrics app carries a *Capacity utilization and throttling* visual with Utilization, Throttling and Overages tabs, and beside it a System events table listing every change in the capacity's state. Utilization shows CU consumed against the capacity's allocation, and a capacity is allowed to burst above it. Exceeding 100 percent is not itself throttling: the excess is carried forward as capacity overage, and any later period below 100 percent spends the spare capacity paying that overage back down. Throttling begins only once the carried-forward overage outlives the free buffer of ten idle-

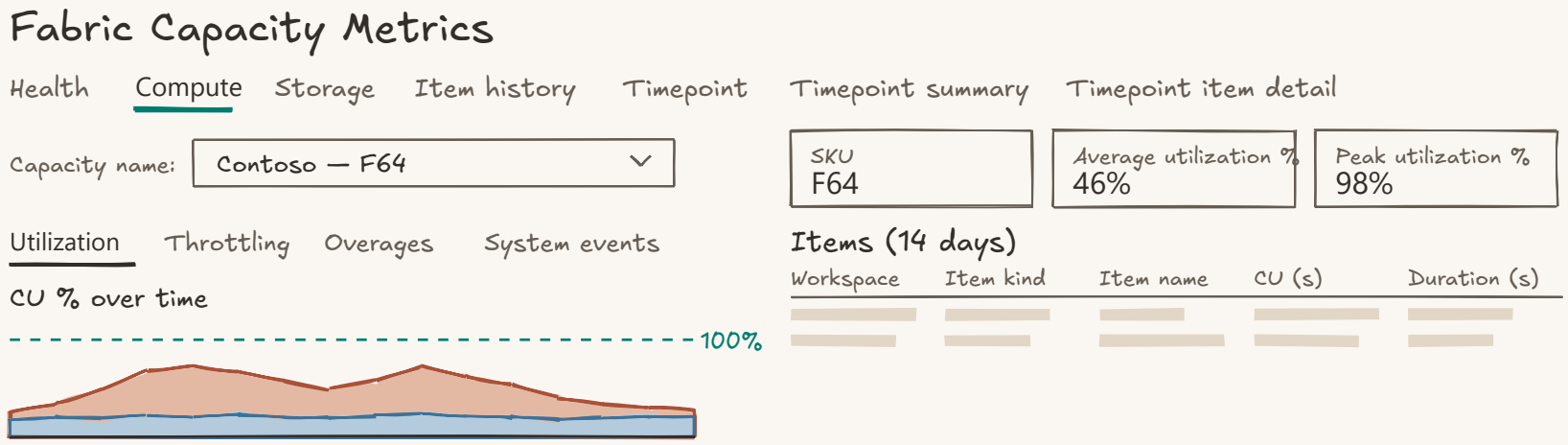
Three places to monitor a capacity



Three monitoring surfaces, ordered by who builds them: the Fabric Capacity Metrics app, which comes with Fabric; community solutions such as FUAM, which someone else builds and maintains and which copy the app's data into a store of your own; and the capacity events Fabric publishes to Real-Time hub, which you route and keep yourself. Pages such as Item history and Timepoint belong to the Metrics app.

Use the Microsoft Fabric Capacity Metrics app

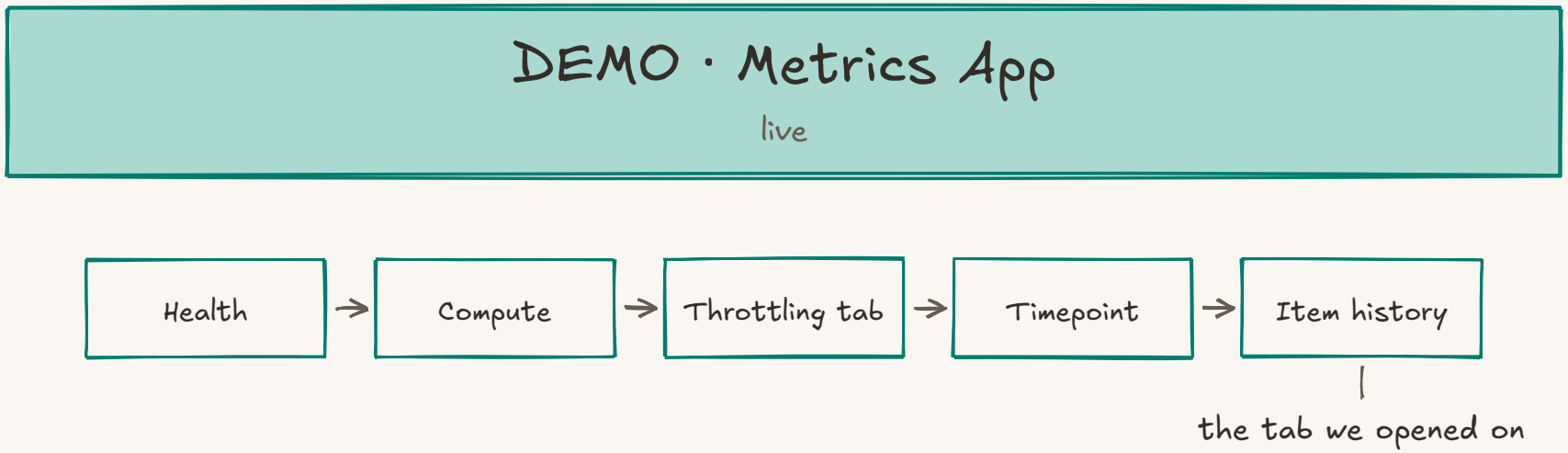
Capacity Metrics app: what's where



The Fabric Capacity Metrics app is organised as pages: Health, Compute, Storage, Item history, Timepoint, Timepoint summary and Timepoint item detail, plus two pages for autoscale compute for Spark. The Compute page covers the last fourteen days, Storage and Item history the last thirty. Data appears with roughly ten to fifteen minutes of latency, and dimensions such as new workspaces refresh only at midnight local time.

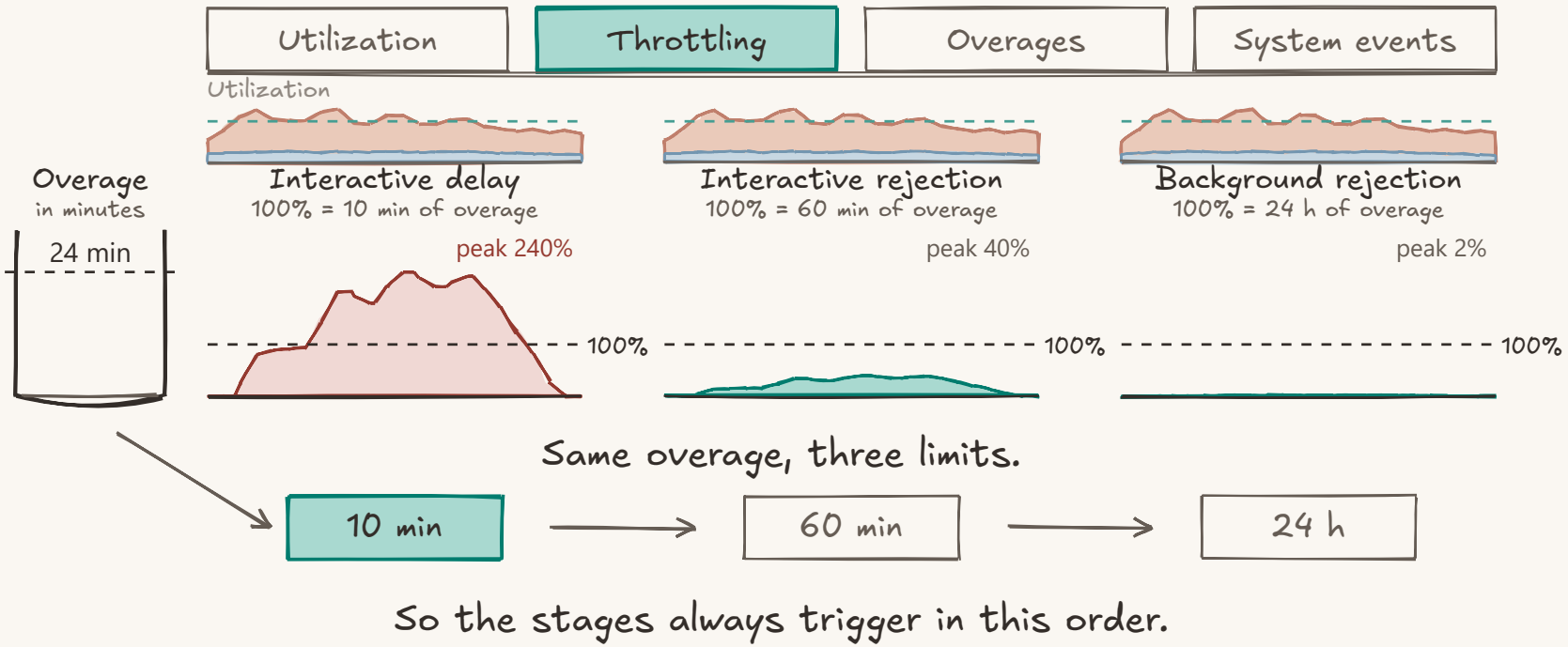
Use the Microsoft Fabric Capacity Metrics app

Demo: Capacity Metrics app



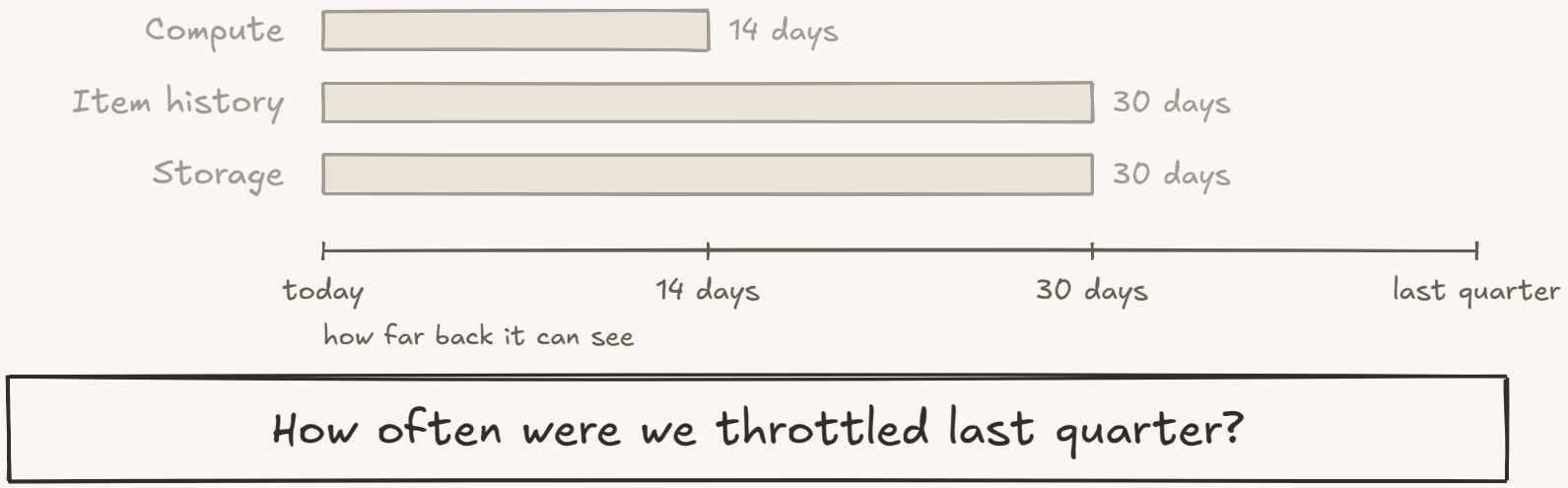
The demo route through the Fabric Capacity Metrics app is live by default, with a drawn fallback branch on the slide: the Health page to orient across capacities, the Compute page for the fourteen-day window with its utilisation, throttling and overage tabs and its System events table, the Throttling tab, which is where throttling is actually diagnosed, a drill into a single timepoint to show the thirty-second grain, and finally the Item history page for a thirty-day view per item — sliced by the user who ran each operation and by billable versus non-billable, with throttling in seconds per operation.

Throttling tab: one graph per stage



The Throttling tab carries one graph per throttling stage: interactive delay, interactive rejection and background rejection. All three plot the same accumulated overage, each normalised to its own allowance — ten minutes, sixty minutes and twenty-four hours respectively — so a value above 100 percent means that stage is in effect. Twenty-four minutes of overage therefore reads as 240, 40 and 2 percent on the three graphs. Because the three share a numerator and differ only in denominator, their percentages are always in descending order, and the stages escalate strictly: a capacity reaches background rejection only after

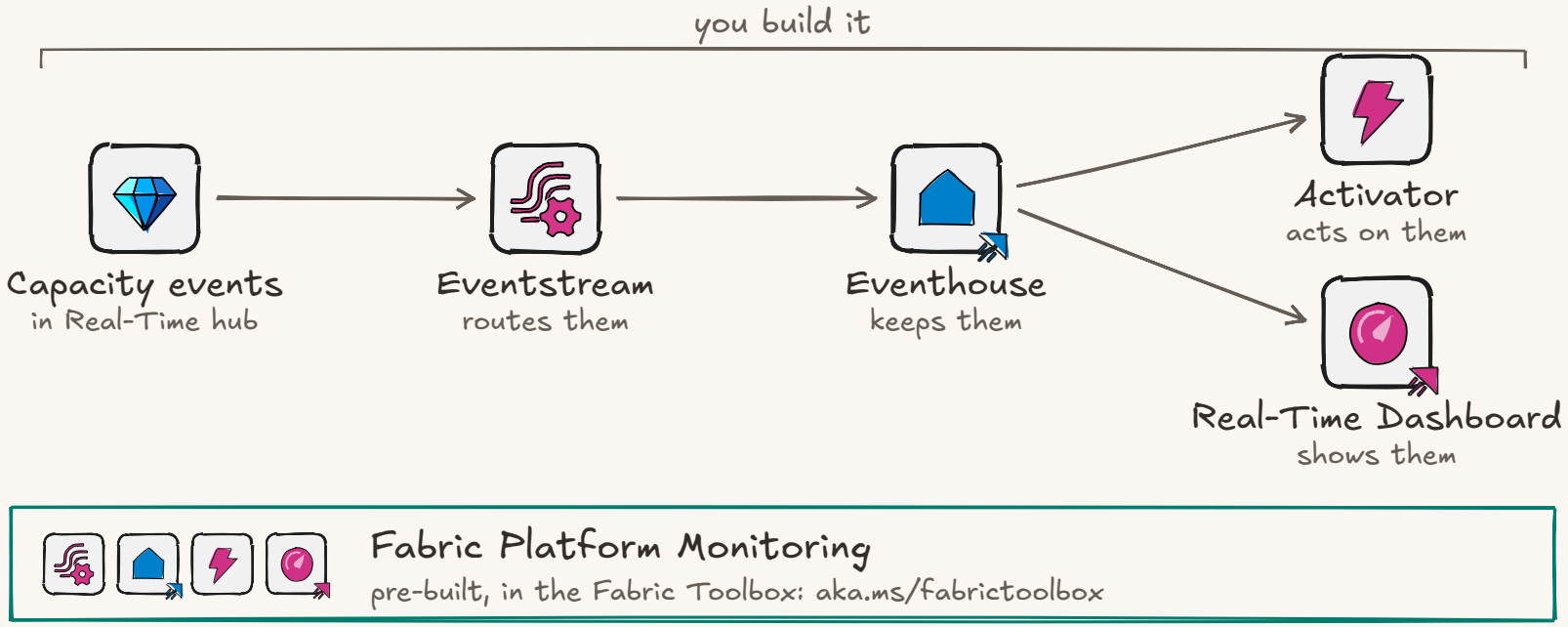
Retention, per Metrics App page



Retention across the pages of the Fabric Capacity Metrics app: the Compute page keeps fourteen days, and the Item history and Storage pages thirty. The app is therefore not an archive: nothing in it answers a retrospective question about a quarter that has already passed — how often a capacity was throttled, or whether it is getting worse. That history has to be kept somewhere else, in a store of your own.

Use the Microsoft Fabric Capacity Metrics app / Item History page in the Fabric Capacity Metrics app (preview)

Capacity events: build your own



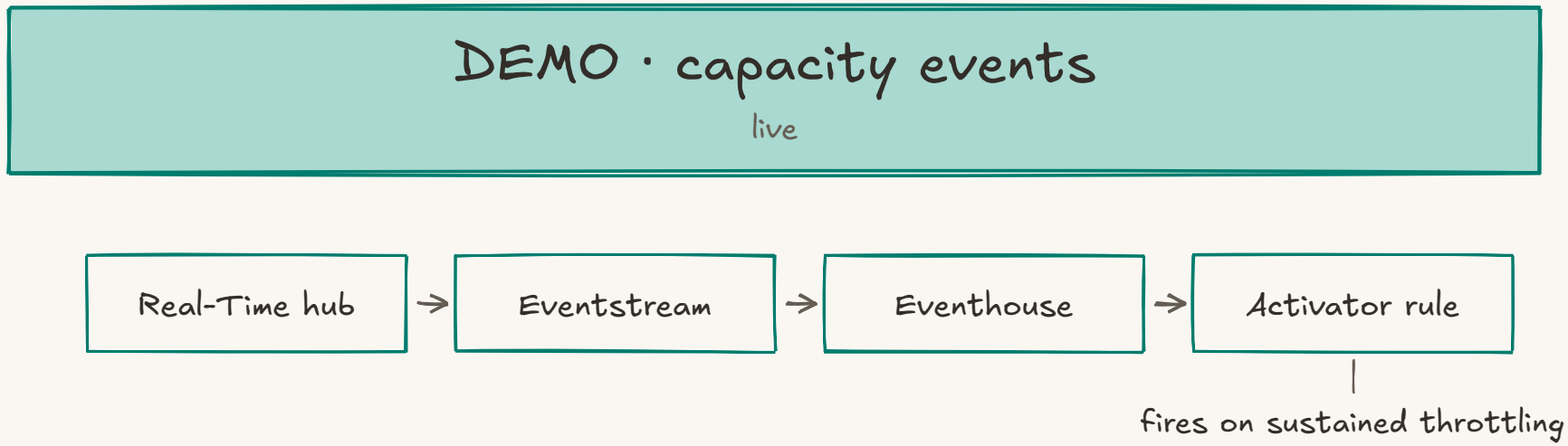
Fabric publishes capacity events to Real-Time hub, and the pieces to keep and act on them are standard Real-Time Intelligence items: an Eventstream routes the events, an Eventhouse stores them with a retention period you set, Activator acts on them, and a Real-Time Dashboard displays them. Because you build and own the pipeline, you can maintain it yourself rather than depending on a third party. Fabric Platform Monitoring, a solution accelerator in Microsoft's fabric-toolbox repository, deploys the same pieces pre-assembled, including a capacity module that reads these events. It is community-supported rather than a

Capacity events: three kinds

State on change	Summary every 30 seconds	Operation every operation
overloaded, paused — and why?	how close to each stage?	who ran it — throttled?
`stateChangeReason`	`interactiveDelayThresholdPercentage`	`throttlingDelayMs`

Fabric capacity events come in three kinds. State events are sent only when a capacity changes state — overloaded, paused, resumed, deleted — and carry the reason for the change. Summary events arrive every thirty seconds per active capacity and carry, among other fields, the percentage of each throttling threshold in use and the overage added, burned down and outstanding. Fabric capacity operation events are not aggregated: one row per operation that consumes capacity, with its CU milliseconds, throttling delay, interactive or background classification, the identity that ran it and whether it succeeded

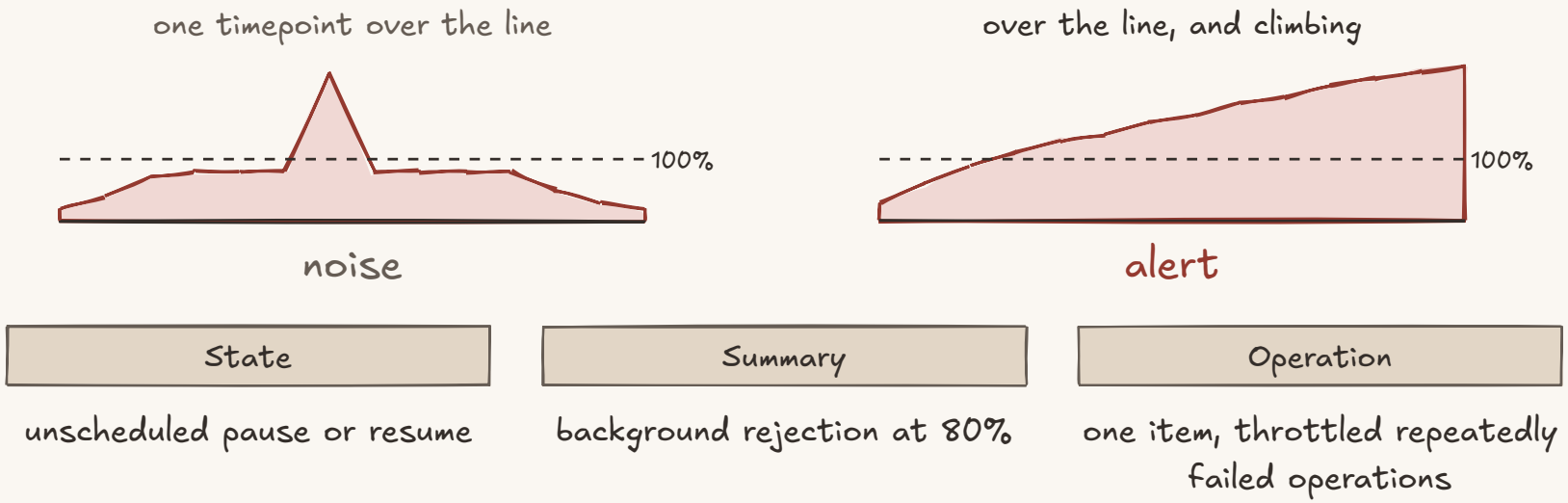
Demo: capacity events, end to end



The end-to-end build, shown live: find the Fabric capacity events in Real-Time hub, route them through an Eventstream into an Eventhouse for storage with a retention period you control, query what has arrived, and attach an Activator rule that fires on sustained throttling. The event detail page in Real-Time hub offers both starting points directly: create an eventstream, and set an alert.

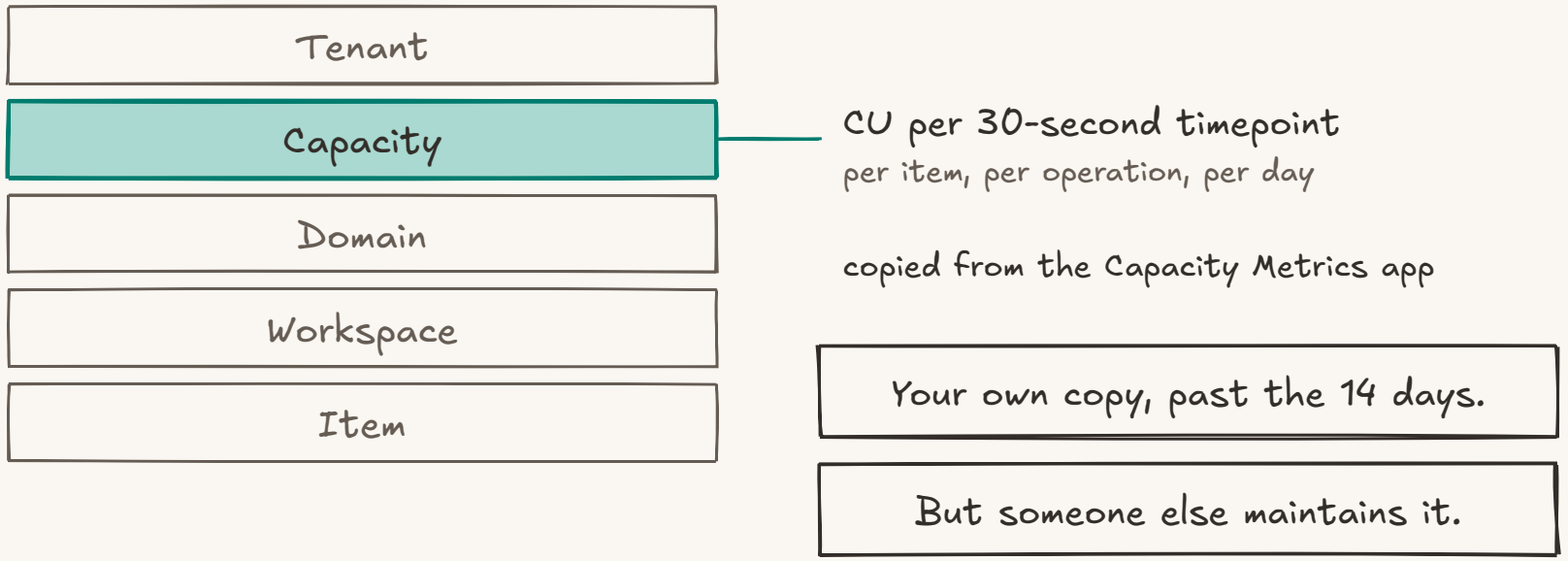
Explore Fabric capacity overview events in Fabric Real-Time hub / Explore Fabric capacity operation events in Fabric Real-Time hub

Which capacity alerts to set



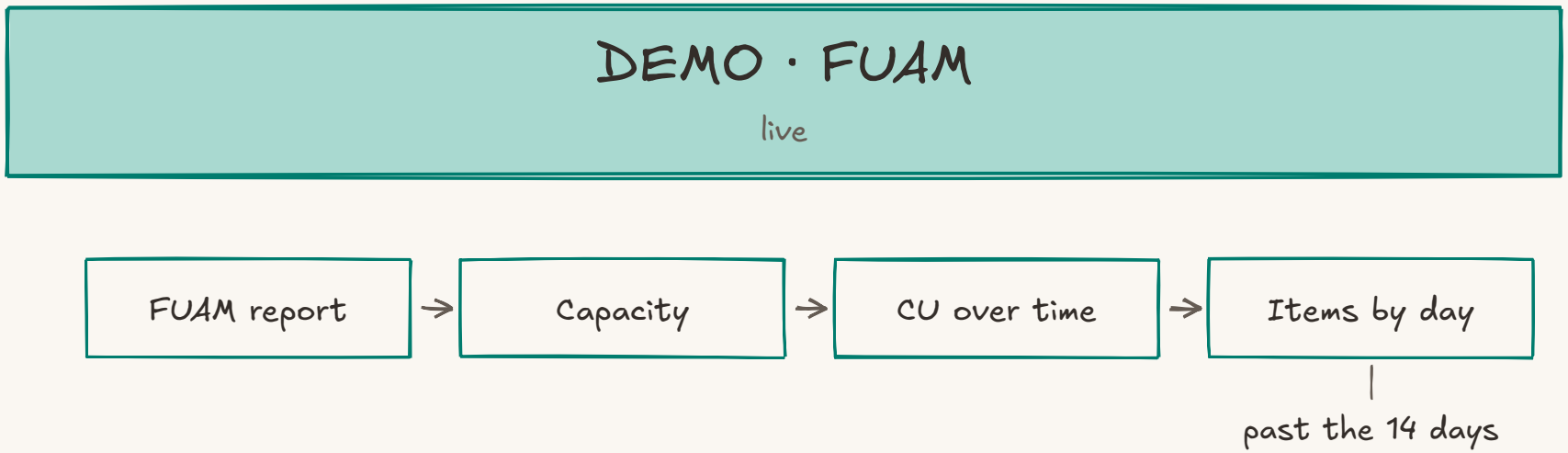
Alerting on capacity events is worth doing narrowly. A single timepoint above the interactive delay threshold is normal and is what smoothing absorbs; alerting on it trains people to ignore the channel. Sustained and rising overage indicates the capacity is not catching up, and because the throttling stages escalate in order it is the early warning for the rejection stages. Beyond that, one alert per kind of event earns its place: on State events, an unscheduled pause or resume; on Summary events, background rejection above eighty percent of its threshold; on Operation events, grouped by item, repeated throttling delay or

FUAM: Fabric Unified Admin Monitoring



FUAM (Fabric Unified Admin Monitoring) is a community-maintained solution from the fabric-toolbox repository. It reports at five levels — tenant, capacity, domain, workspace and item — with drill-through between them. At the capacity level it stores CU per thirty-second timepoint and CU per item, per operation and per day, extracted from the Fabric Capacity Metrics app's semantic model. Because it merges each extraction into its own store, it keeps that history well past the app's fourteen days. Like every community solution, it is maintained by others on a best-effort basis rather than supported by Microsoft, and it

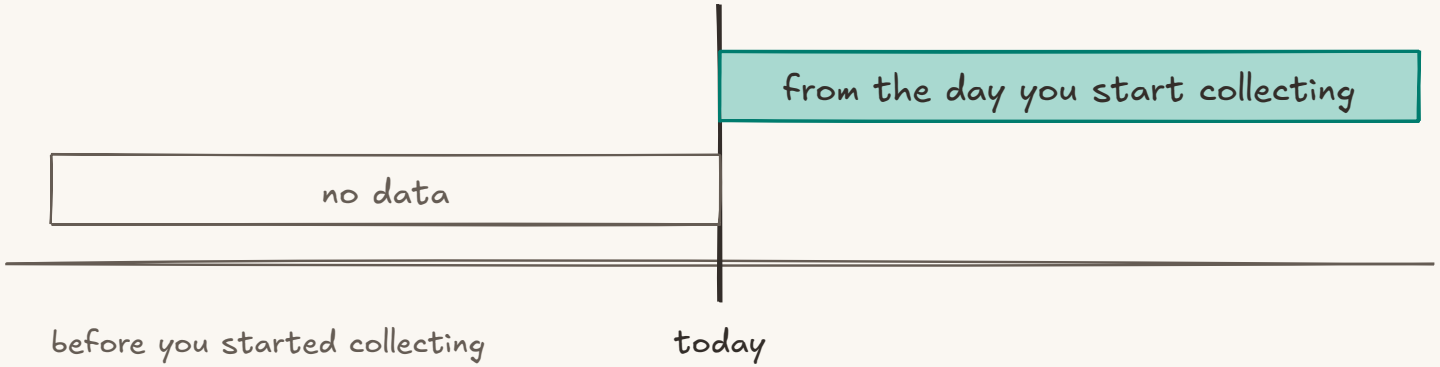
Demo: FUAM



A short live demonstration of FUAM at the capacity level: its report, capacity-level CU over a date range longer than the Metrics app's fourteen days, and CU per item per day at the same grain the app provides.

FUAM — Fabric Unified Admin Monitoring (fabric-toolbox)

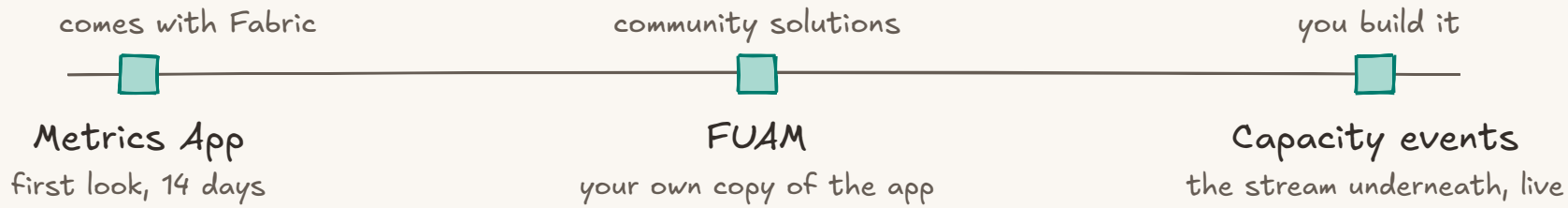
Capacity events and FUAM: no backfill



Turn it on today.

Neither way of keeping capacity history is backfilled. Fabric capacity events begin when you subscribe; FUAM's history begins when it is deployed — its first load reaches back only as far as the Metrics app still holds, at most fourteen days. Starting early therefore has value independent of whether anything is built on the data yet. Two further caveats apply to the events. Percentages derived from them will not match the Metrics app unless the smoothing and bursting accounting is reproduced exactly, though per-operation attribution is reliable. And some workspace and item names arrive as placeholders, flagged by

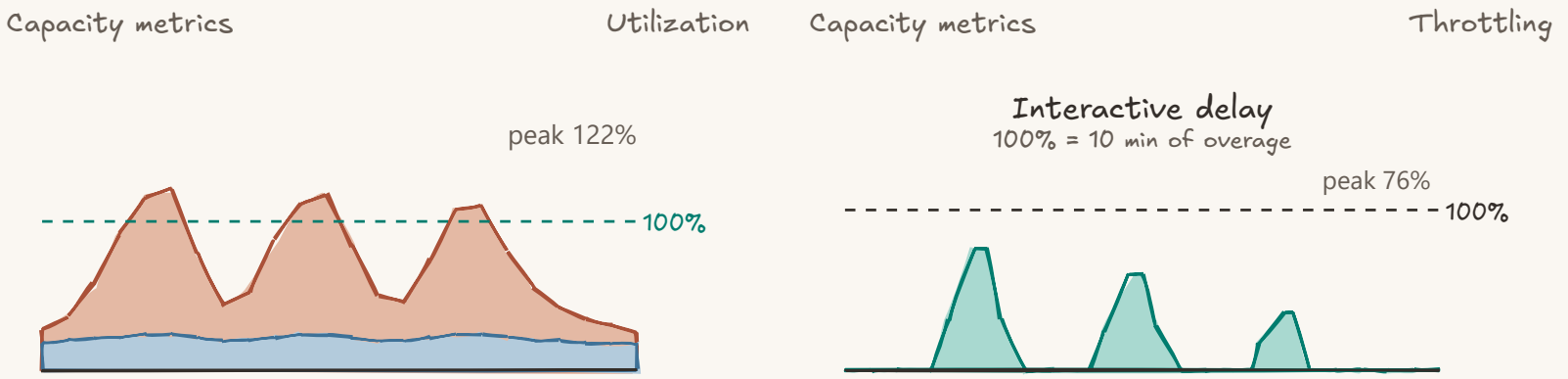
Recap: when to use which



The three monitoring surfaces and when each is the right one to reach for: the Fabric Capacity Metrics app, with pages such as Item history and Timepoint, for a first look across fourteen days; community solutions such as FUAM for a copy of the app's data kept past those fourteen days, maintained by someone else; and the capacity events in Real-Time hub for the stream underneath the app, to know as it happens and to keep the history yourself.

Use the Microsoft Fabric Capacity Metrics app

Everything so far was where to click.



Next / 05 Understand what's happening

Chapter four covered where to look: the same day can look like trouble on the Utilization tab and be healthy on the Throttling tab, and only the tools that keep history can say what happened last quarter. Chapter five covers what the patterns on those surfaces mean, beginning with an exercise in profiling a workspace from exactly the tools described here.

Use the Microsoft Fabric Capacity Metrics app

05 · Understand

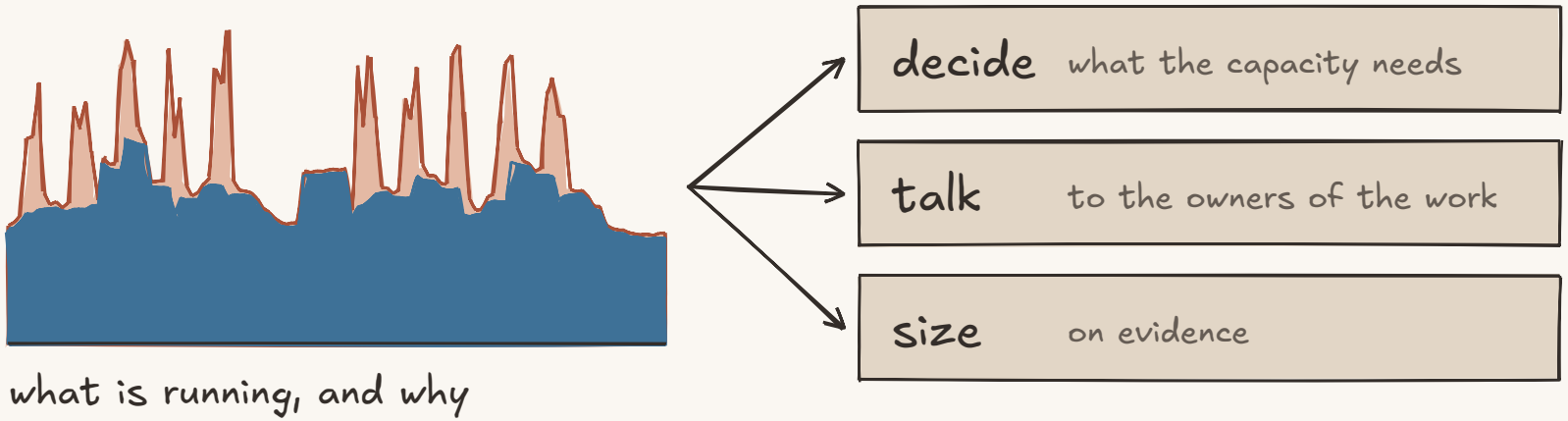
Now: what is it all doing?

Reading what runs on a capacity, before changing anything.

This chapter reads consumption patterns: what runs on a capacity, how each workload looks on a utilisation chart, and how patterns combine, before any setting is changed.

Why understand before we act?

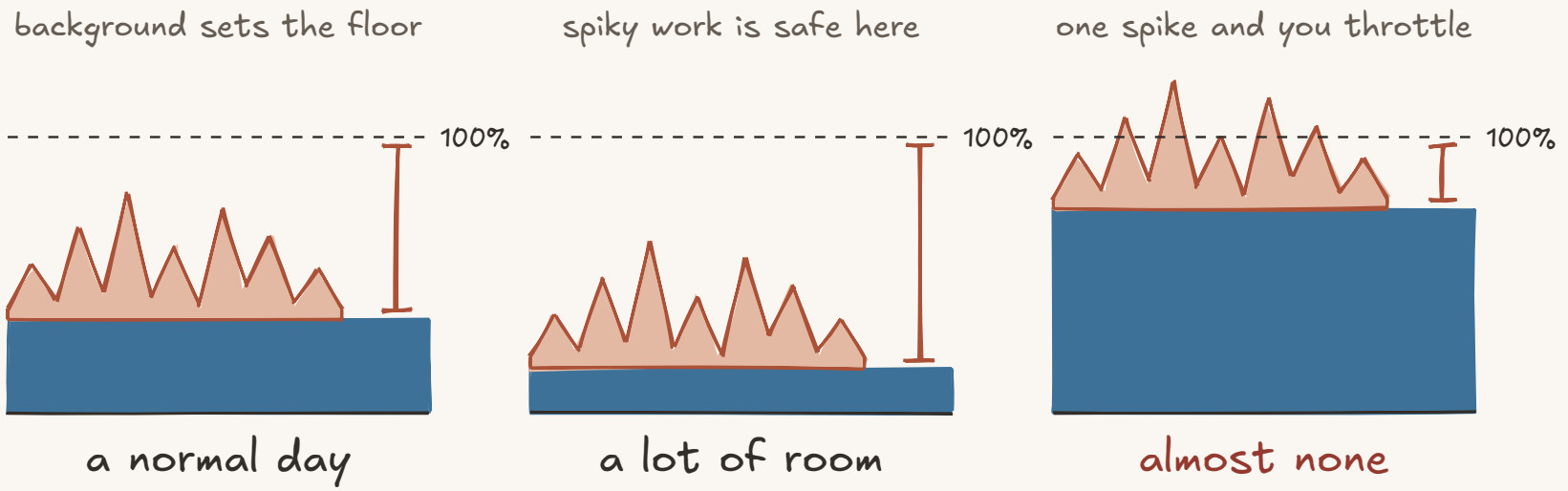
read it first · then decide, talk, size



Understanding what consumes a capacity precedes any change to it. Knowing which workloads run where, and what they are for, is the basis for three things: deciding what the capacity needs; discussing optimisations with the owners of the work from evidence rather than from aggregate utilisation; and grounding sizing decisions in observed behaviour.

The operational levers — scaling, pausing, resizing — are comparatively simple to apply. Their value depends entirely on that

How much room is left for interactive?

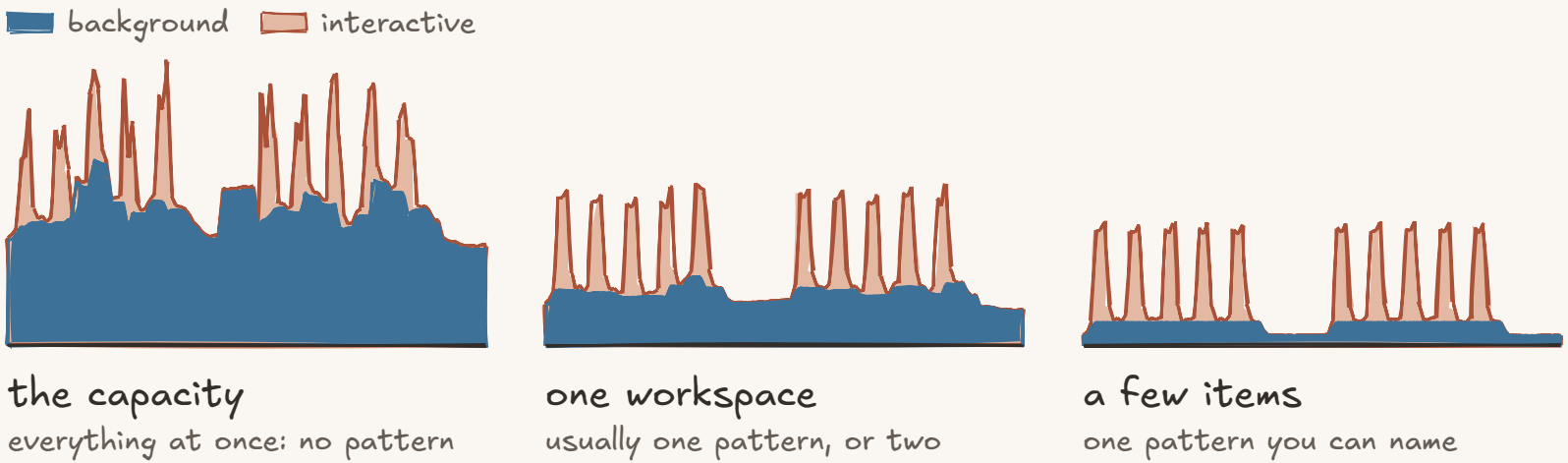


Same spikes. The floor decides.

Background operations establish a baseline of consumption; interactive operations must fit in the headroom above it. Interactive demand is erratic rather than smooth, so it needs headroom to spike into, and how much it needs depends on when it arrives. Three capacities can show similar averages while having very different amounts of room for the same interactive spikes.

Zoom in and a few patterns appear.

read the workspace · drop to the items when it is a mix



Fabric capacity operation events: every operation, with its workspace and item.

Consumption on a capacity falls into a small number of recognisable patterns, but only at the right level of detail. At capacity level every workload is summed and no pattern can be distinguished.

The workspace is the preferred level of analysis: it is the practical unit of control, since it can be moved between capacities, constrained, divided or charged back without changes to its content, and it usually carries one pattern or a simple combination

Let's play a game.

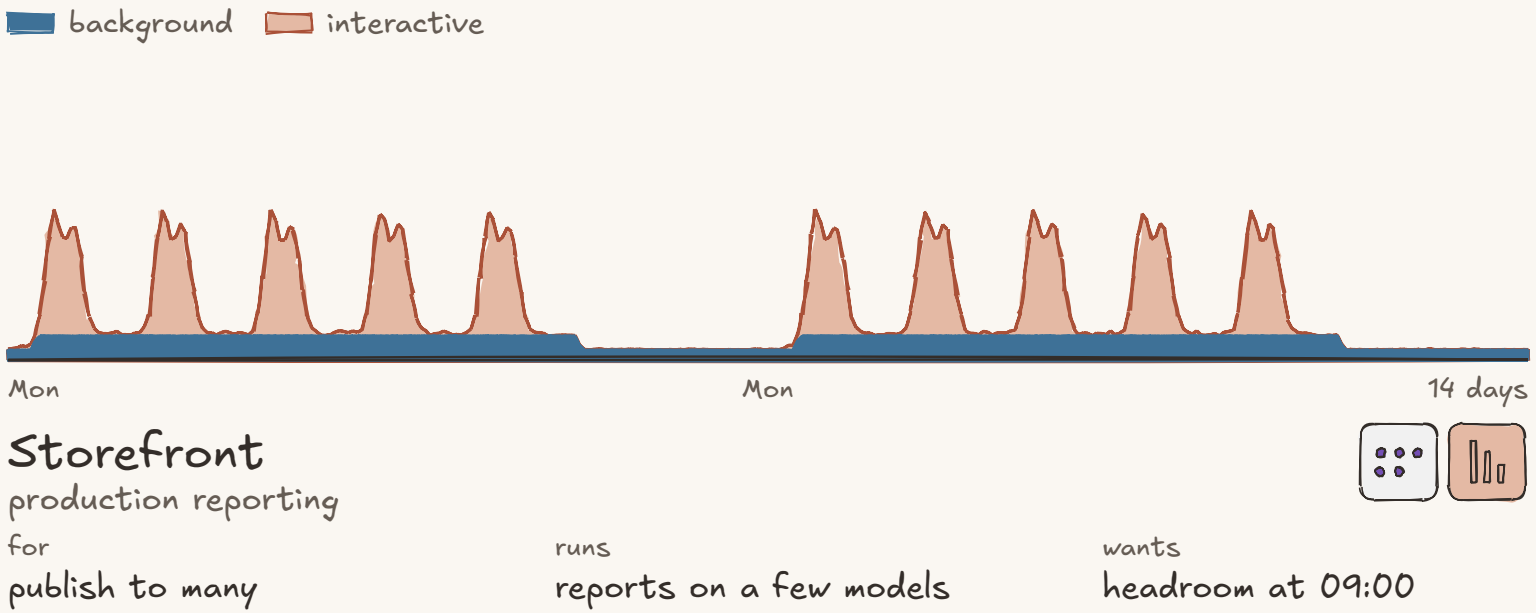
chart first · you name it · then the answer



An exercise: eight utilisation charts, one per workload pattern, each shown before it is named so the room reads the shape first.

Pattern 1: which workload is this?

Weekday office hours with a lunch dip, dead weekends, a thin refresh floor

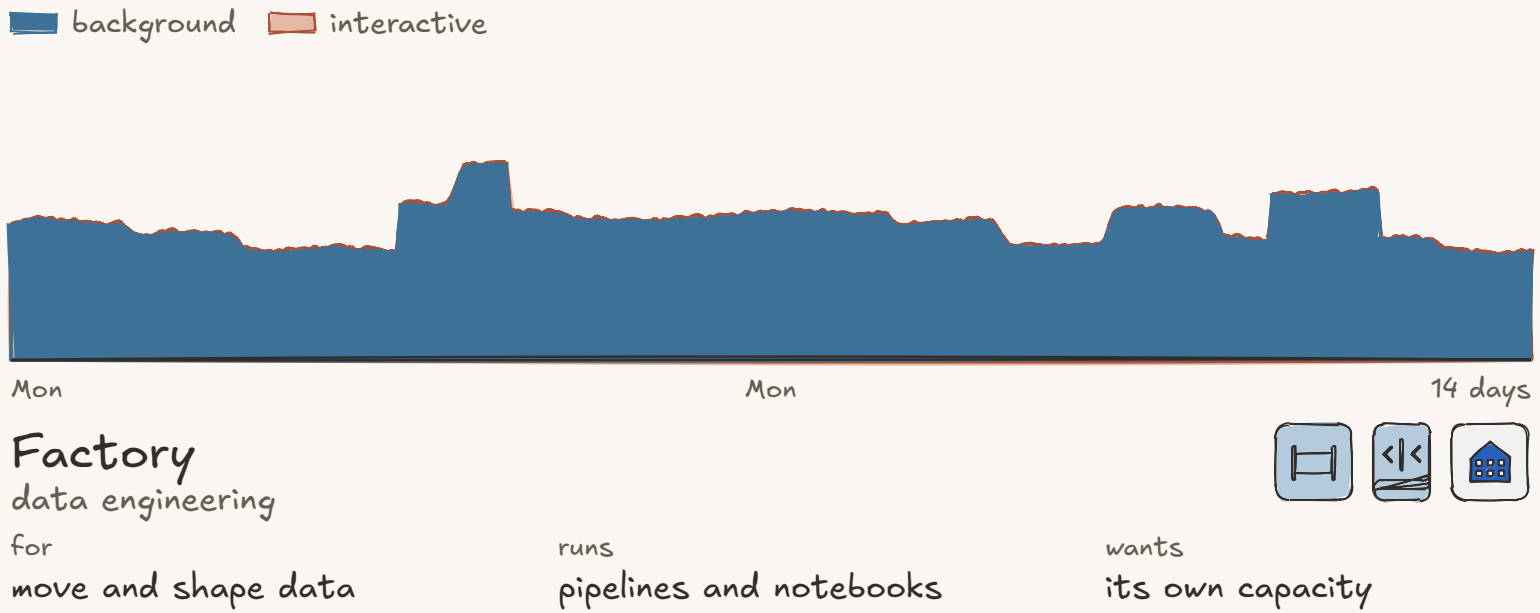


Storefront — production reporting. A small number of authors publish content that a large audience consumes during business hours: weekday interactive peaks in the morning and afternoon, a midday dip, and near-zero weekend activity.

Its background component is a scheduled refresh before the working day. Because background consumption is smoothed over 24 hours, that refresh appears as a thin, near-constant floor that falls away at the weekend, not as a spike at refresh time. Its

Pattern 2: which workload is this?

Round the clock, every day: a wandering plateau, almost no interactive

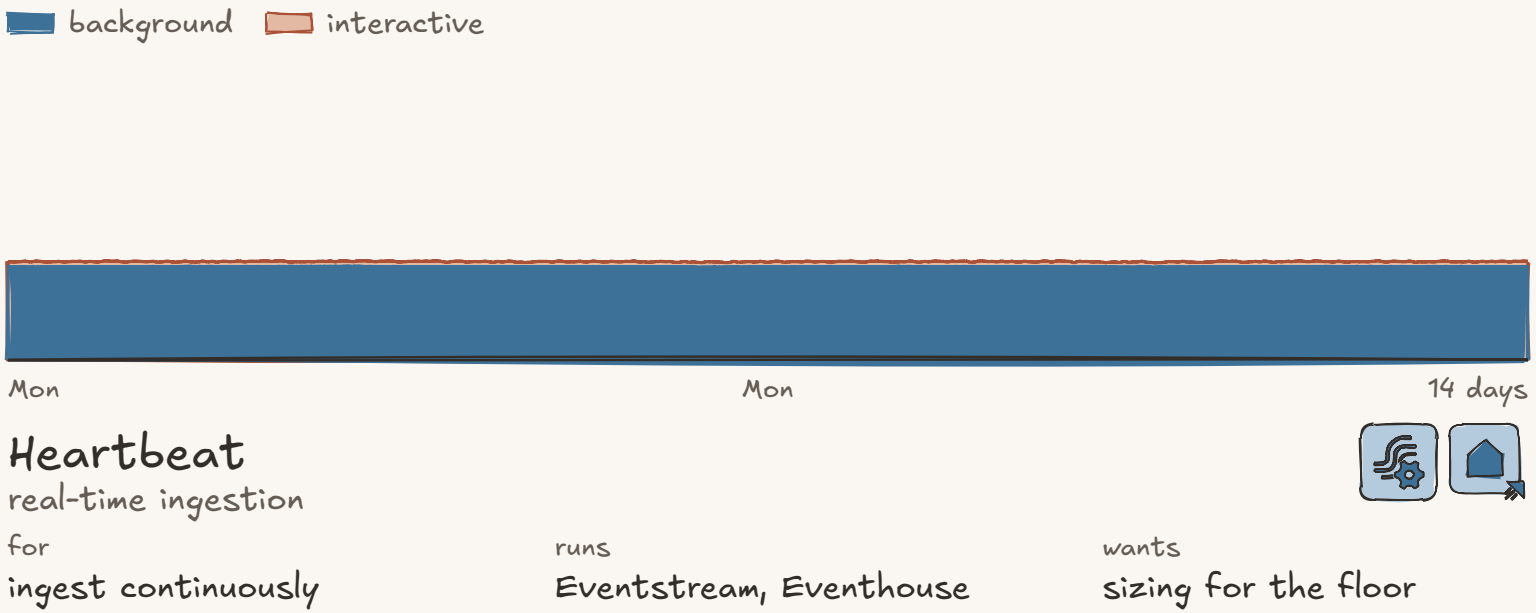


Factory — data engineering. Pipelines, notebooks and dataflows move and shape data on a schedule, with effectively no interactive consumption because the workspace has no audience.

Background operations are smoothed across a 24-hour window, so discrete jobs appear as a continuous plateau rather than as spikes. The plateau varies with the size of each day's batch and steps up for a day after an unusually large run, but it follows

Pattern 3: which workload is this?

Dead flat, day and night, weekdays and weekends alike

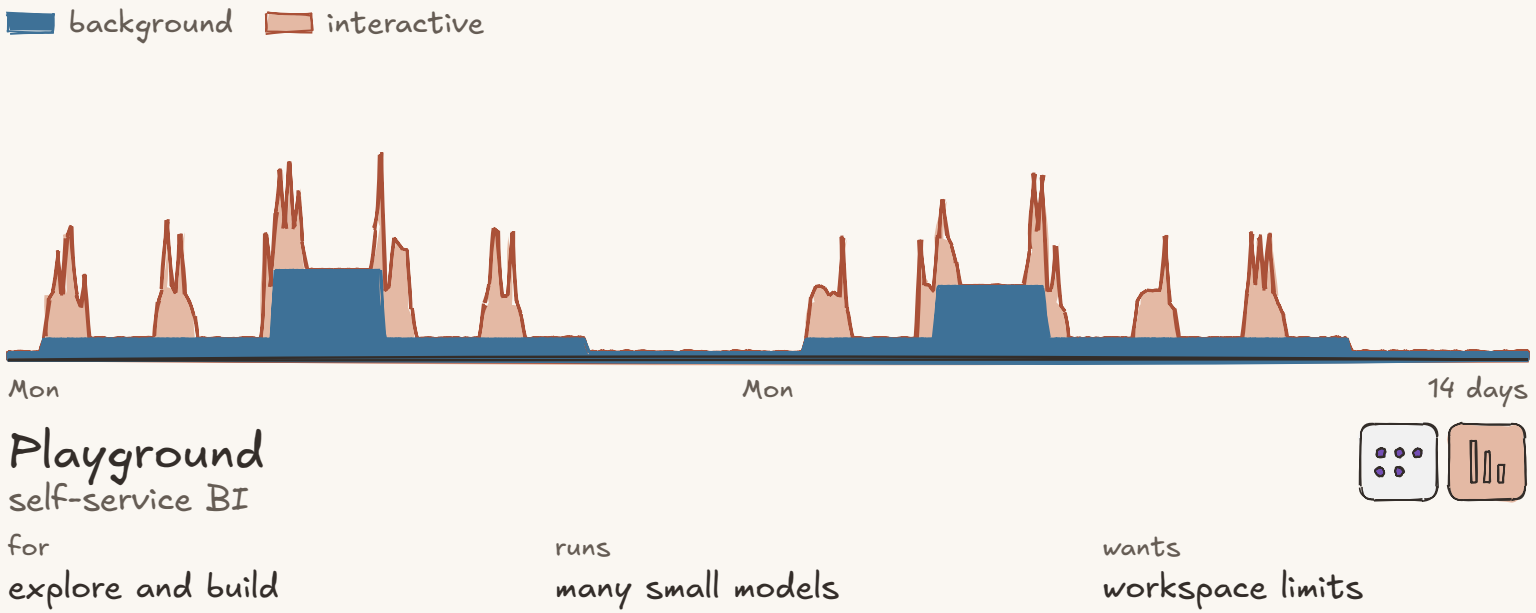


Heartbeat — real-time intelligence. Continuous ingestion through Eventstream, Eventhouse and Activator produces a consumption profile that does not vary by hour or by day, because it is driven by event volume rather than human activity.

The absence of a quiet period means the capacity must be sized for the sustained floor rather than an average, and pausing offers no relief.

Pattern 4: which workload is this?

The same working day every weekday, jagged with small spikes, over a low floor

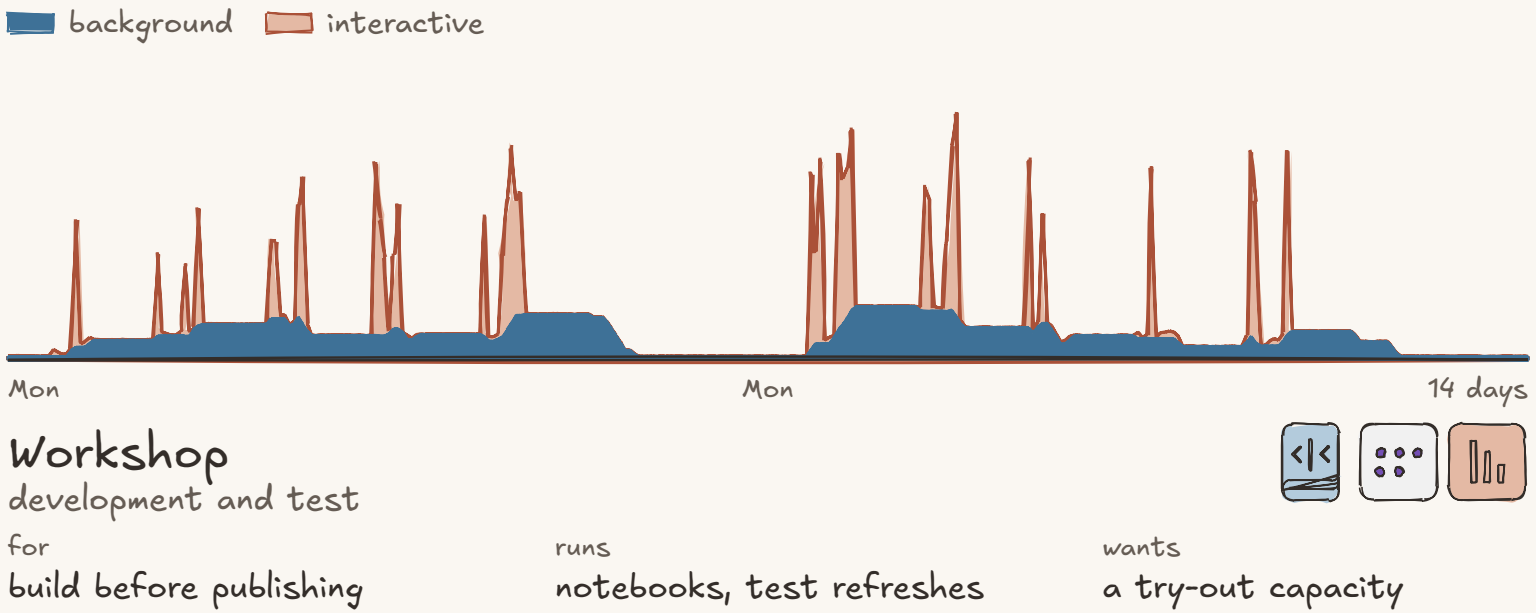


Playground — self-service. A large number of small semantic models and reports owned by many different people produces a working-day profile that repeats every weekday, roughened by frequent small interactive spikes rather than forming one smooth business-hours peak.

Its background floor is low but carries occasional day-long humps: a single heavy operation, such as a large dataflow or model

Pattern 5: which workload is this?

Working hours only, tall and erratic interactive bursts, no two days alike

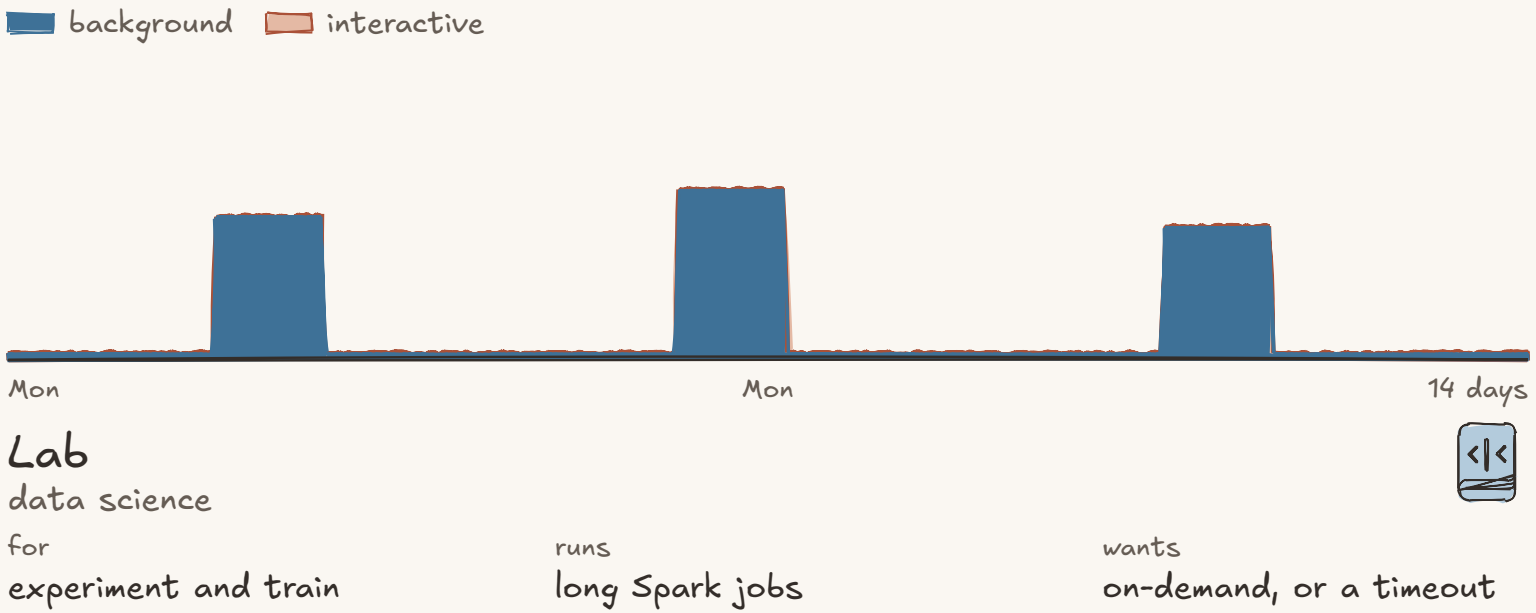


Workshop — development and test. Irregular notebook execution, test refreshes and debugging during working hours, with no weekly periodicity and the most volatile interactive consumption of any pattern.

Its distinguishing feature is the temporal offset between the two kinds of work: a job runs, stepping up the smoothed background floor, and the developer inspects the result roughly an hour later, producing an interactive burst. That lag indicates

Pattern 6: which workload is this?

Nearly empty, then a tall background block that holds for exactly a day

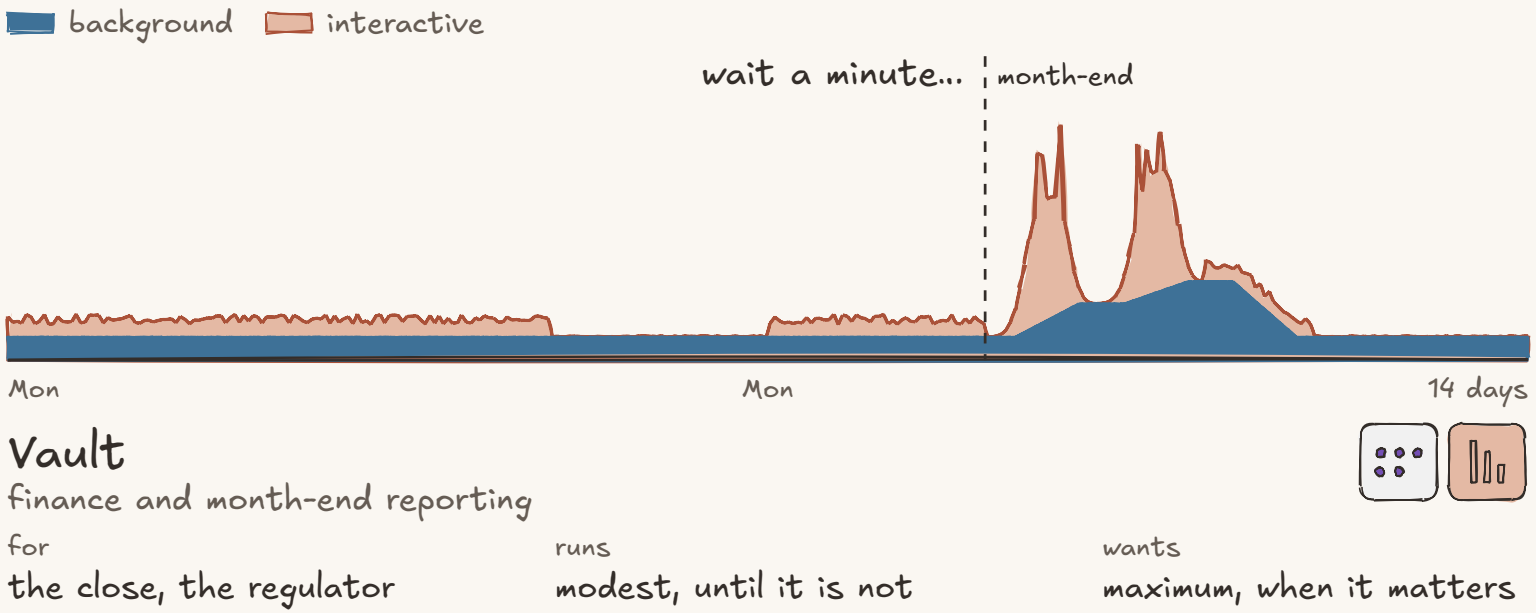


Lab — data science and machine learning. A small number of long-running Spark jobs at irregular intervals, each consuming substantially more CU than anything else in the field guide.

Because the work is background, each run is smoothed across a 24-hour window and therefore occupies capacity long after it completes. The consequence is that a Lab's impact is generally felt by co-tenant workspaces rather than by itself.

Pattern 7: which workload is this?

Modest all month, then interactive spikes and a rising floor at the close

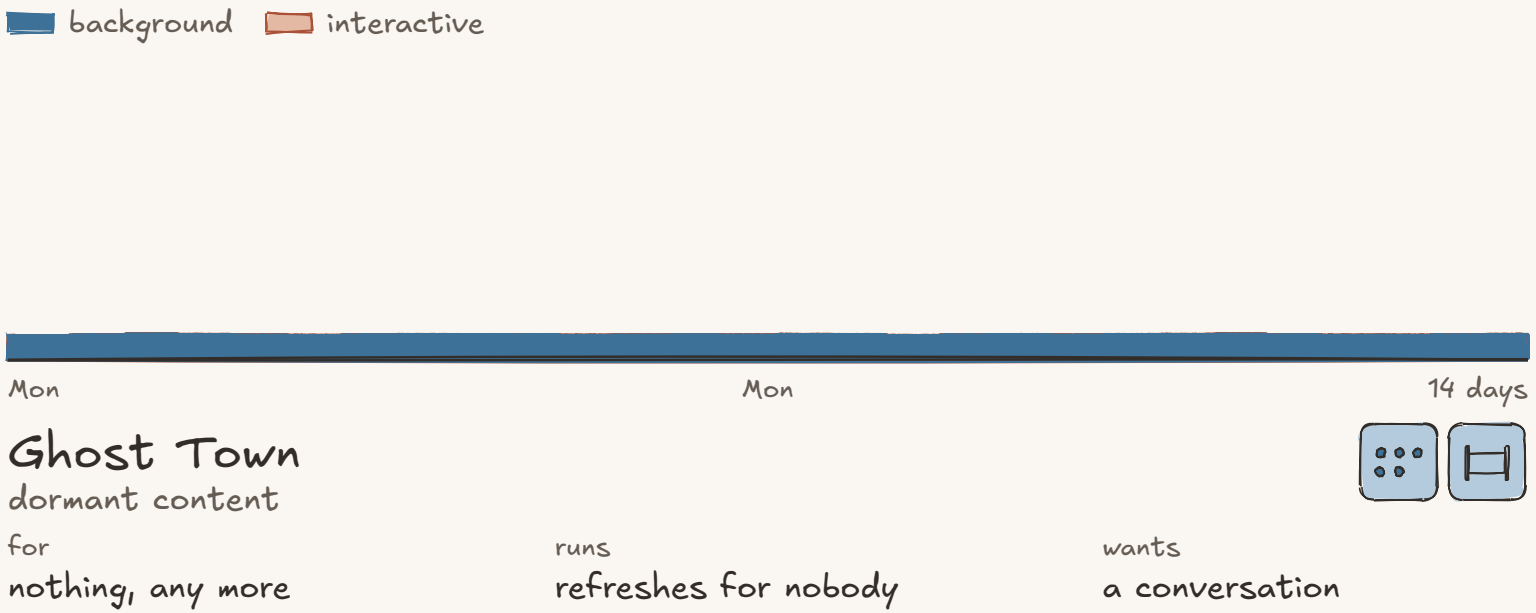


Vault — finance and month-end reporting: close packs, executive reporting and regulatory workloads, where the calendar rather than the working day drives demand.

Consumption is modest for most of the month. On calendar boundaries — month, quarter and year end — interactive use spikes and the smoothed background floor rises as refreshes are rerun; the timing, not the shape, identifies the pattern. What it

Pattern 8: which workload is this?

A flat background floor, and not one interactive query in fourteen days

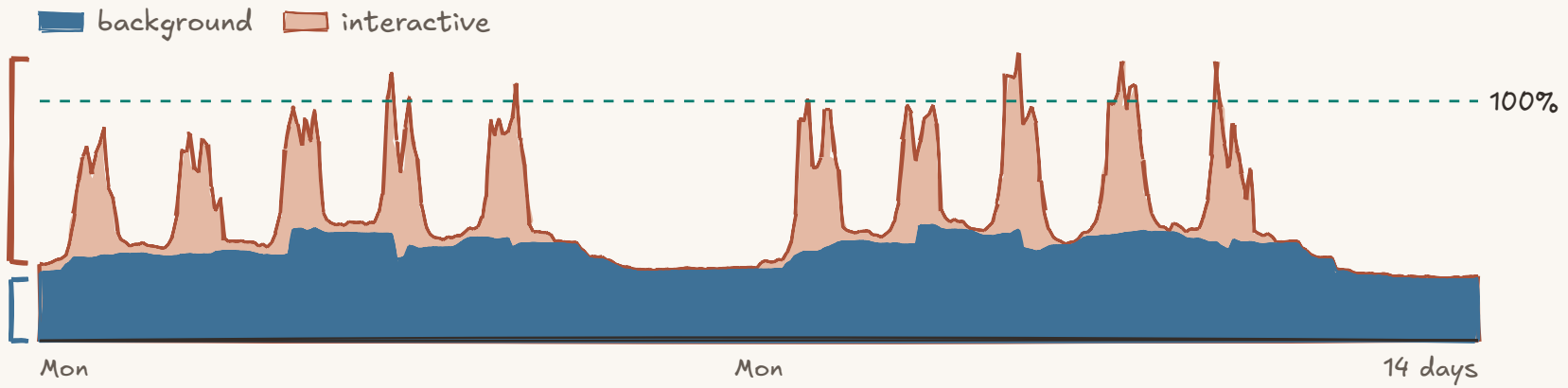


Ghost Town — dormant content. Scheduled background operations continue to consume CU while no interactive queries occur against the workspace.

The tell is the absence of Query operations, not a user count of zero: scheduled refreshes execute under the owner's identity. The reading identifies a candidate for review — dormant content may still be required for retention or infrequent reporting —

Two questions sort all eight patterns.

is it the right size · do we need guardrails



Is this the right size?

Are we properly sized?

people: Storefront, Playground, Workshop, Vault

Do we need guardrails?

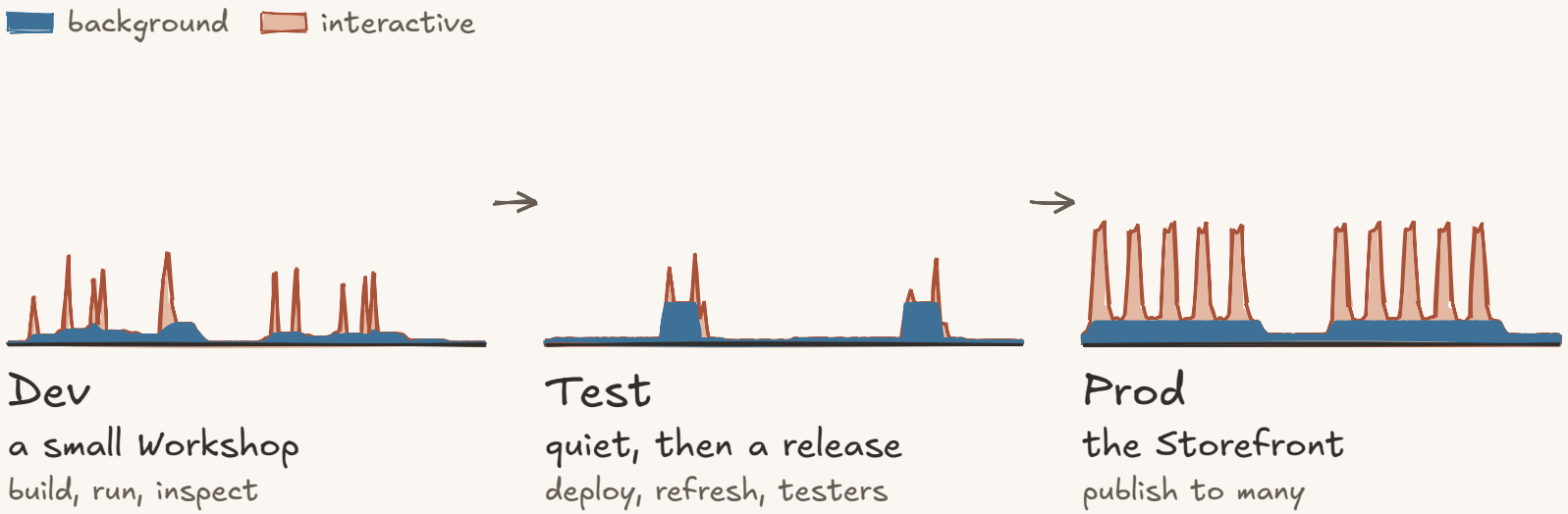
a floor that keeps growing up

machines: Heartbeat, Factory, Lab, Ghost Town

The eight workspace patterns divide into two groups, each prompting one question about the capacity. Interactive, people-driven patterns — Storefront, Playground, Workshop and the Vault at the close — are the ones whose spikes reach or cross the capacity limit; their question is whether the capacity is correctly sized. Background, machine-driven patterns — Heartbeat, Factory, Lab and Ghost Town — set a floor that grows as work accumulates; their question is whether guardrails are needed.

Dev, Test, Prod: three patterns.

Dev → Test → Prod: the same reports, three shapes

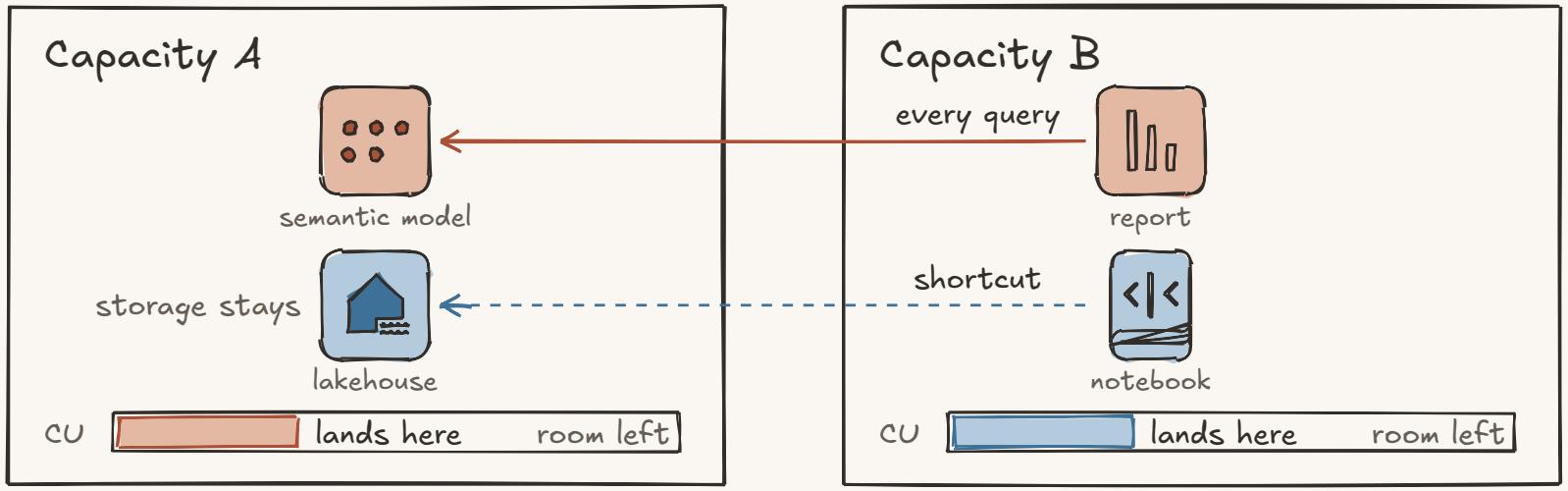


A single solution deployed through Dev, Test and Production workspaces — the layout deployment pipelines encourage — produces a different consumption pattern at each stage, even though the content is the same.

Dev resembles a small Workshop: irregular development runs followed by inspection, with no weekly periodicity. **Test** is close to idle except around releases, when the deployment triggers a full refresh (a day-long step once smoothed) followed by

Who pays for the compute?

the room left for Power BI and interactive work is what the CU leaves



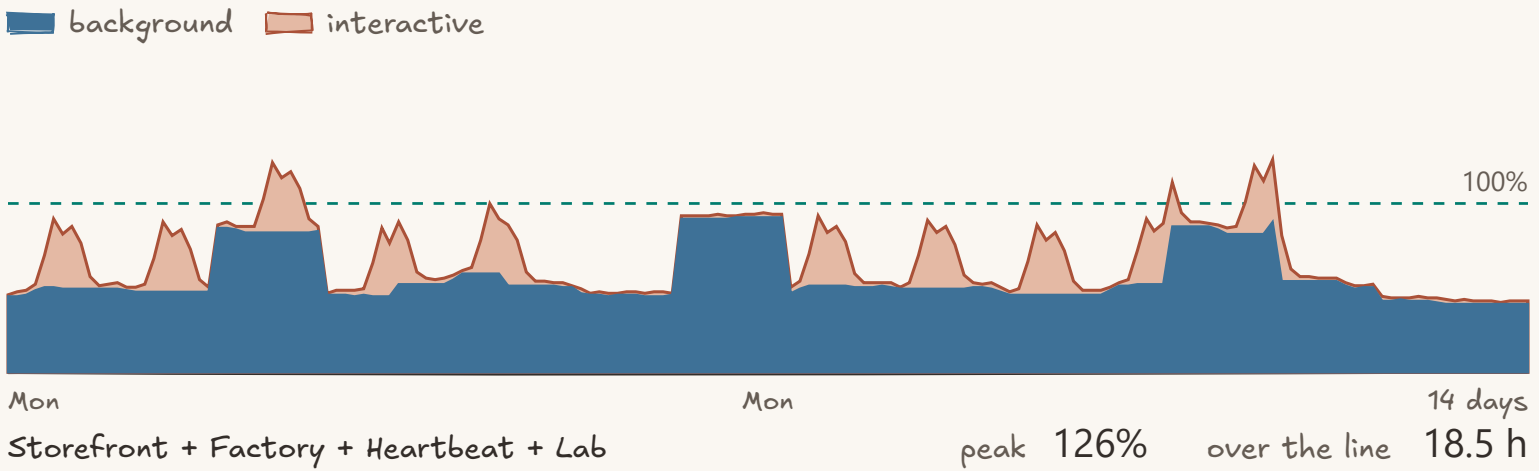
Compute lands where the work runs.

Compute is charged to the capacity on which the work executes, which is not necessarily the capacity of the item a user interacts with.

Reports and semantic models. A report connected to a semantic model in another workspace issues its queries to that model; the query operations are the semantic model's, and their CU is consumed on the capacity hosting the model's workspace.

Which mix breaks the capacity?

every capacity chart is a sum · the smallest part can tip it over

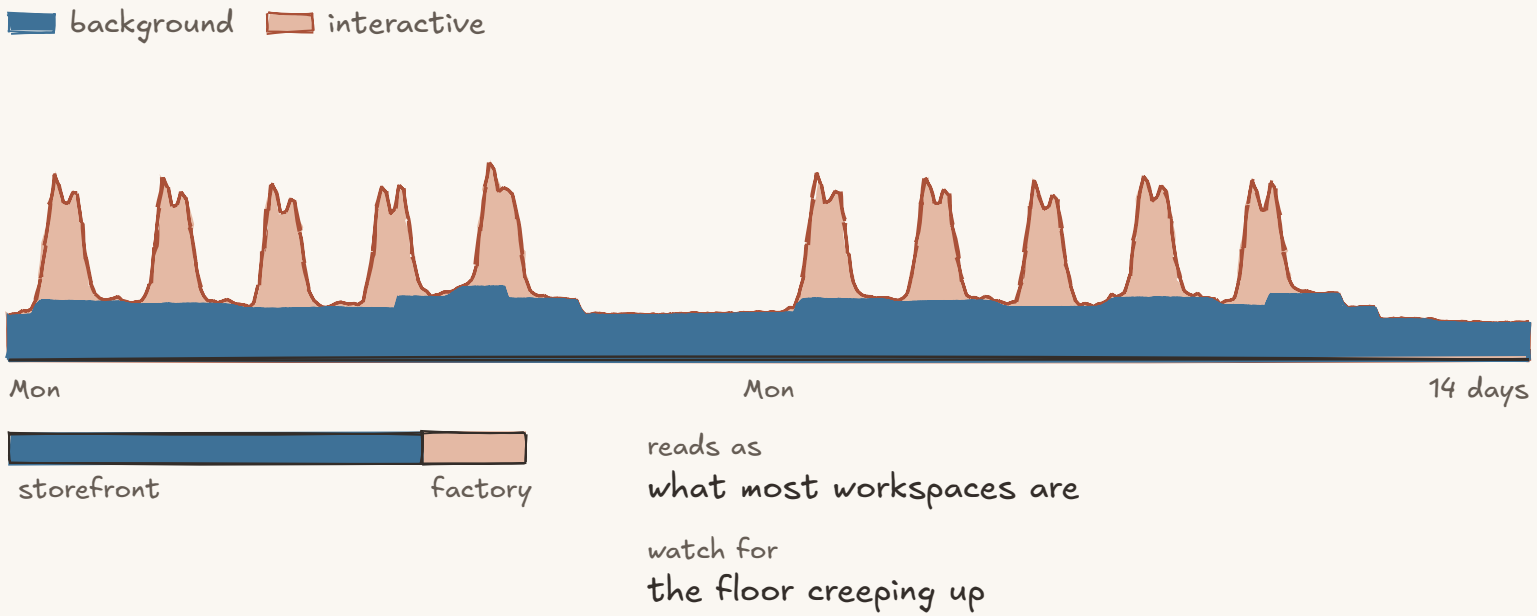


Every capacity chart is the sum of the workspaces placed on it. The interactive chart combines six illustrative workspaces, one per pattern, and any subset can be switched on to see the combined utilisation, its peak and the time spent above the allocation.

A Storefront, a Factory and a Heartbeat together peak at 97% of the allocation. Adding a Lab — the smallest contributor by

One workspace, two patterns.

Classic Power BI: Storefront + a little Factory

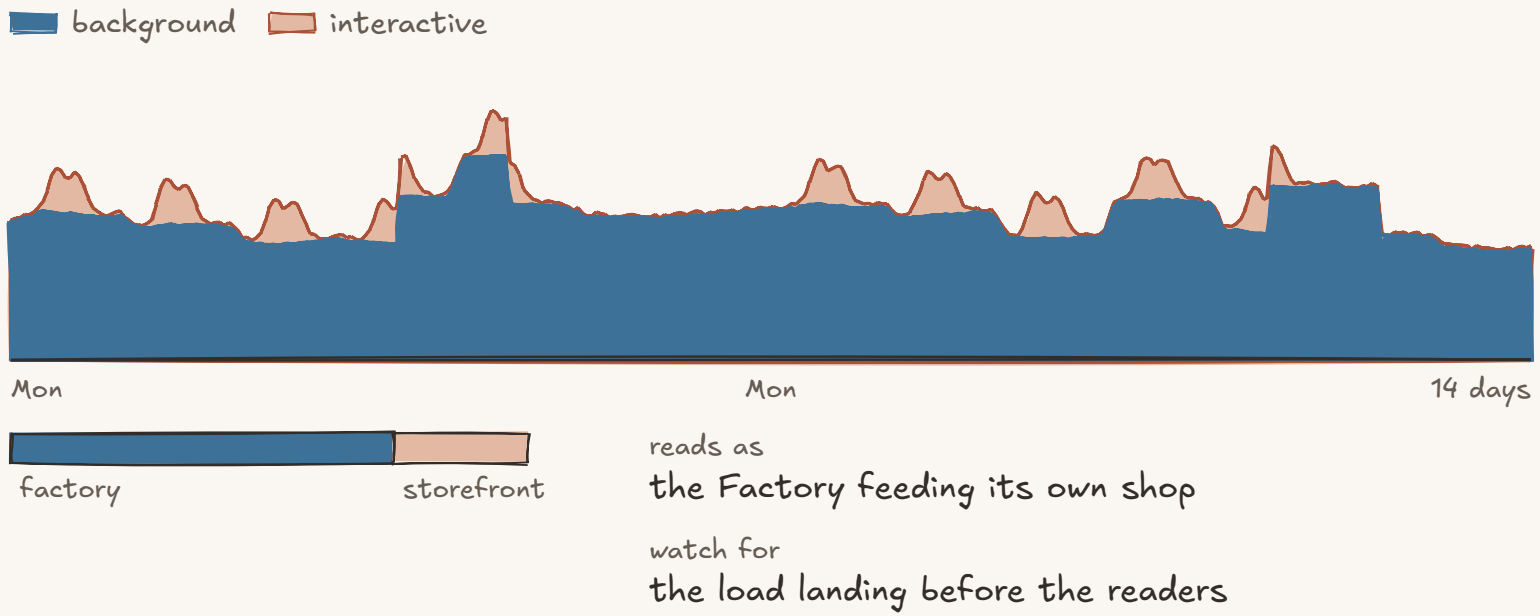


Pure patterns are a teaching device. Real workspaces are combinations, and the useful skill is estimating proportions rather than assigning a single label.

The most common combination — a Storefront with a light Factory beneath it, typically a semantic model with a dataflow feeding it — is shown with a mix bar expressing the share each pattern contributes.

Pipelines feeding the reports.

Medallion: Factory + Storefront



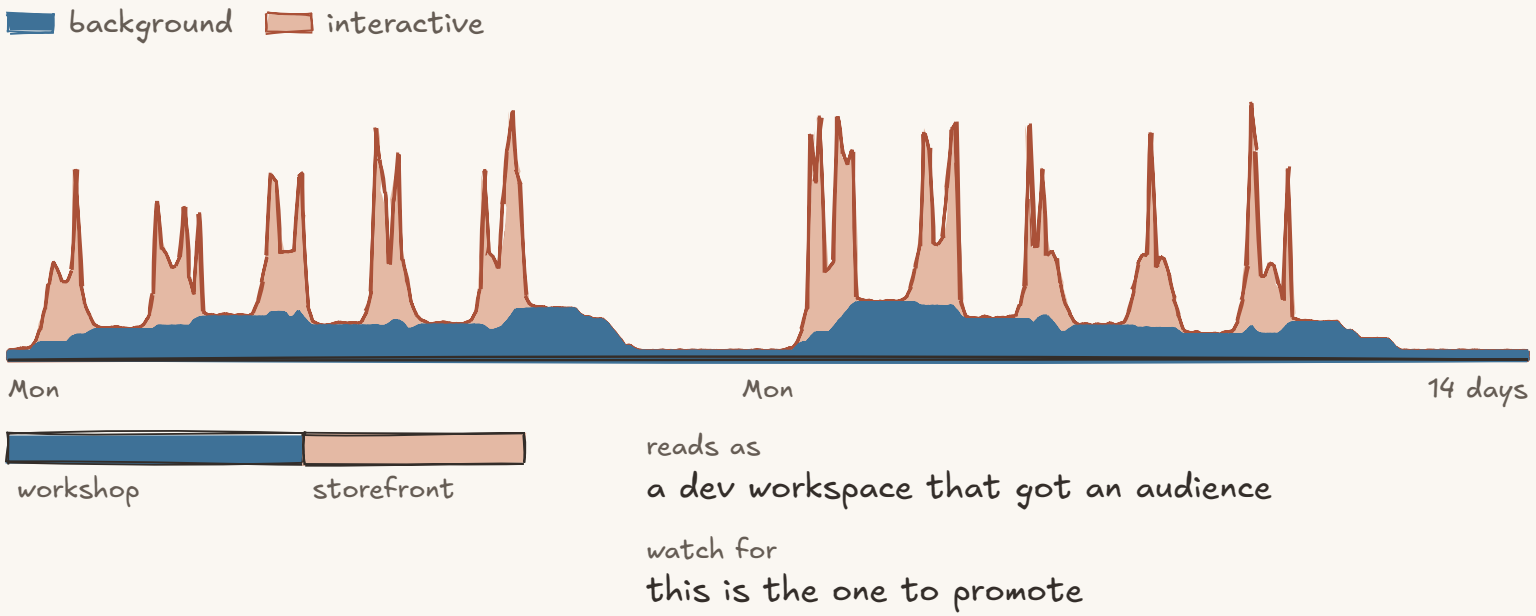
Medallion — factory + storefront. Reads as the Factory feeding its own shop.

The mix bar expresses the proportion each archetype contributes to the combined profile. Identifying the components matters because the archetype responsible for a capacity problem is frequently not the one contributing the most consumption.

Particular attention: the load landing before the readers.

Dev work that found an audience.

Shadow Storefront: Workshop + Storefront



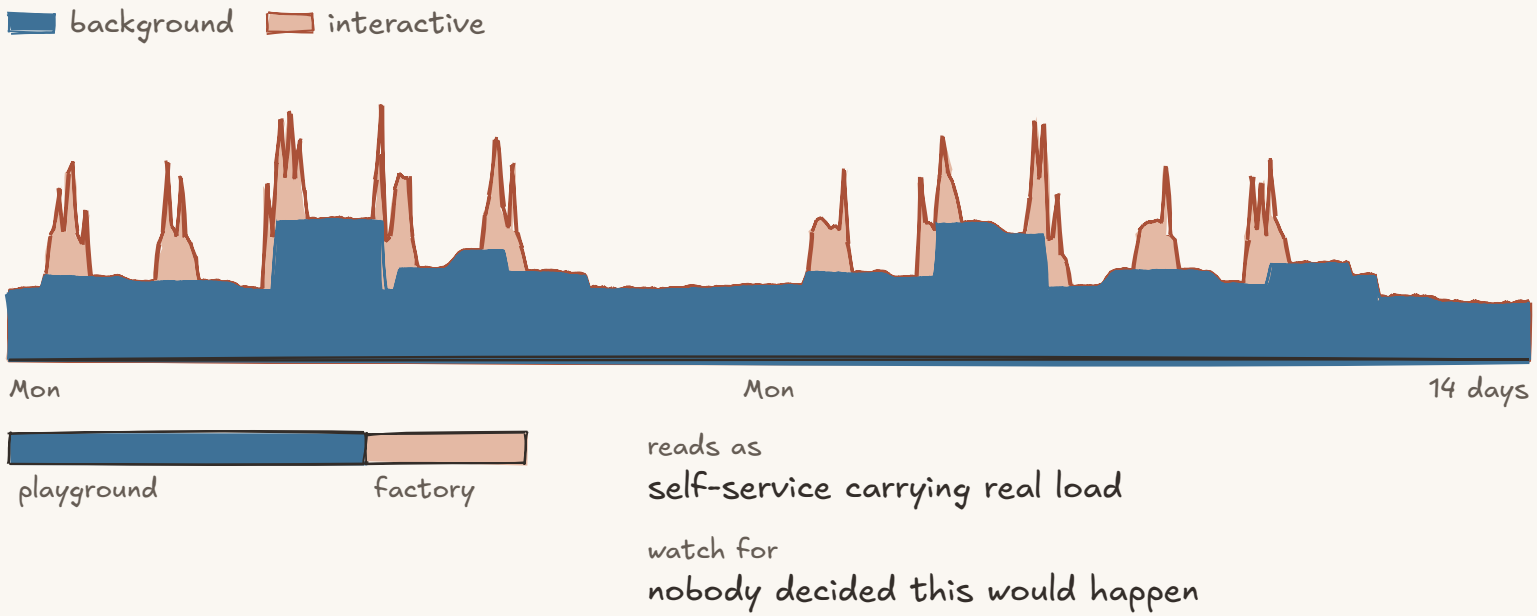
Shadow Storefront — workshop + storefront. Reads as a dev workspace that got an audience.

The mix bar expresses the proportion each archetype contributes to the combined profile. Identifying the components matters because the archetype responsible for a capacity problem is frequently not the one contributing the most consumption.

Particular attention: this is the one to promote.

Self-service carrying real load.

Playground that grew up: Playground + Factory



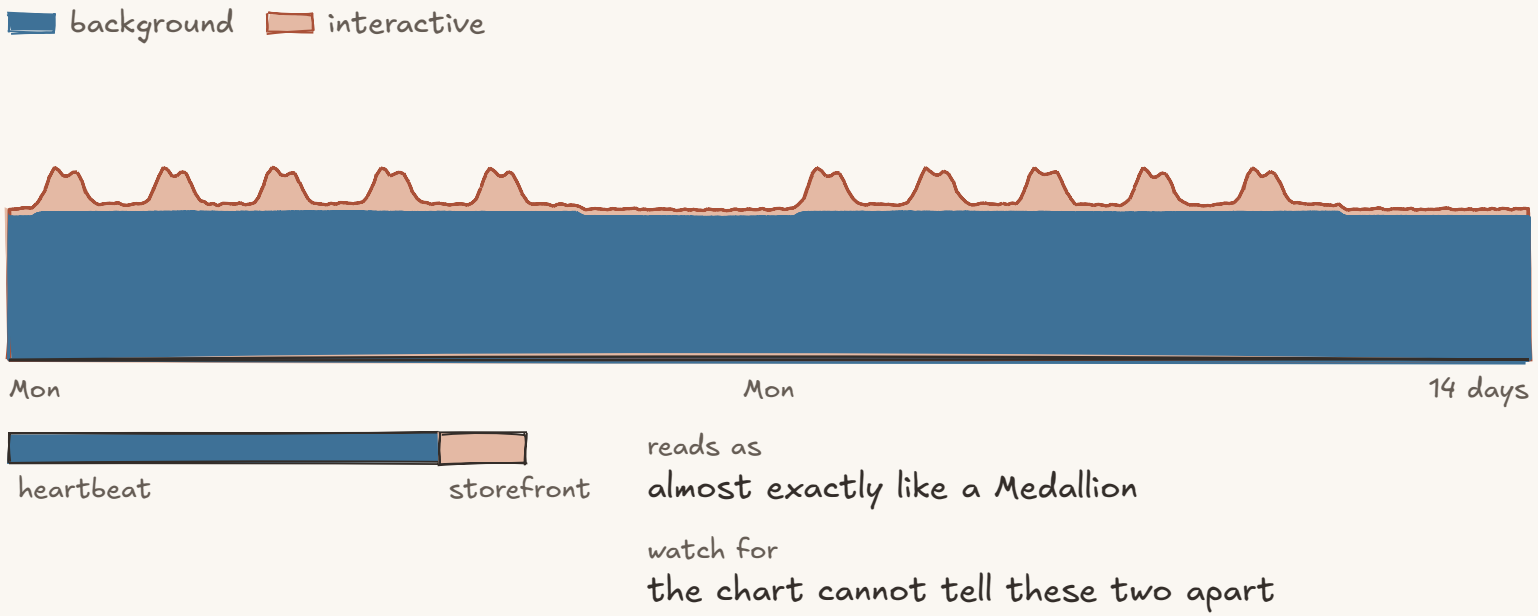
Playground that grew up — playground + factory. Reads as self-service carrying real load.

The mix bar expresses the proportion each archetype contributes to the combined profile. Identifying the components matters because the archetype responsible for a capacity problem is frequently not the one contributing the most consumption.

Particular attention: nobody decided this would happen.

Two recipes, one picture.

Live dashboard · the shape asks, the items answer

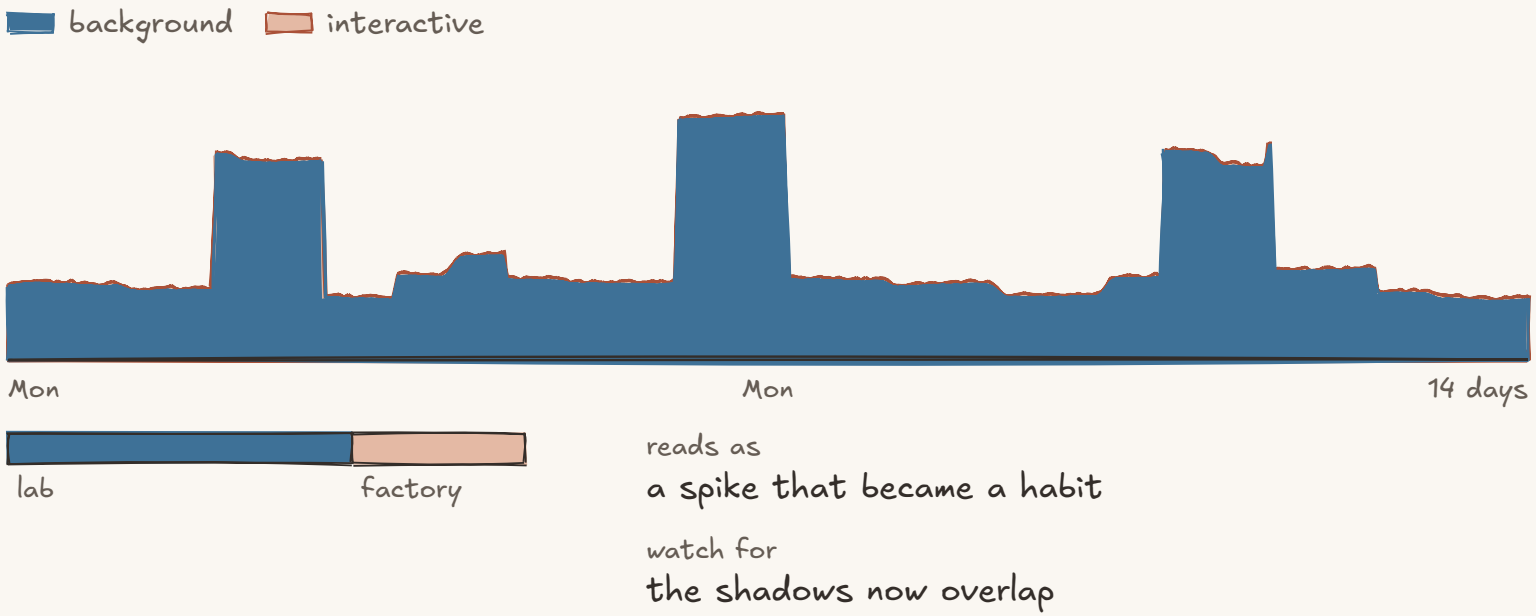


Live dashboard — a Heartbeat providing continuous ingestion with a Storefront reading layer above it.

This combination is very nearly indistinguishable from a **Medallion** on a utilisation chart: both present a flat background floor with a business-hours interactive hump. The distinction cannot be recovered from shape alone and must come from the item types present in the workspace.

A Lab job put on a schedule.

Lab that got scheduled: Lab + Factory



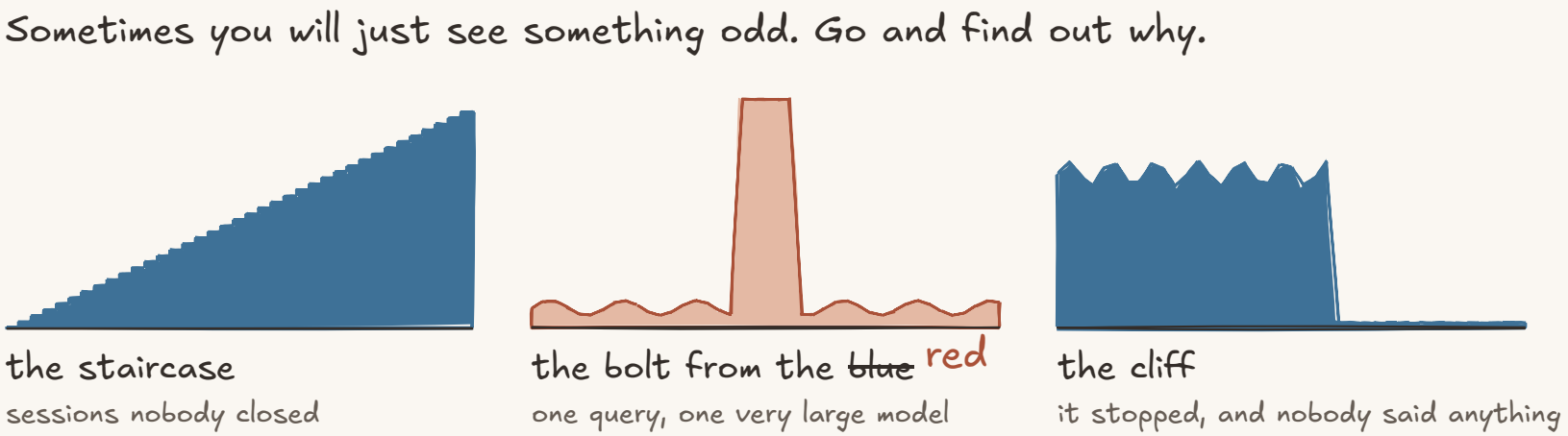
Lab that got scheduled — lab + factory. Reads as a spike that became a habit.

The mix bar expresses the proportion each archetype contributes to the combined profile. Identifying the components matters because the archetype responsible for a capacity problem is frequently not the one contributing the most consumption.

Particular attention: the shadows now overlap.

Some shapes fit no pattern.

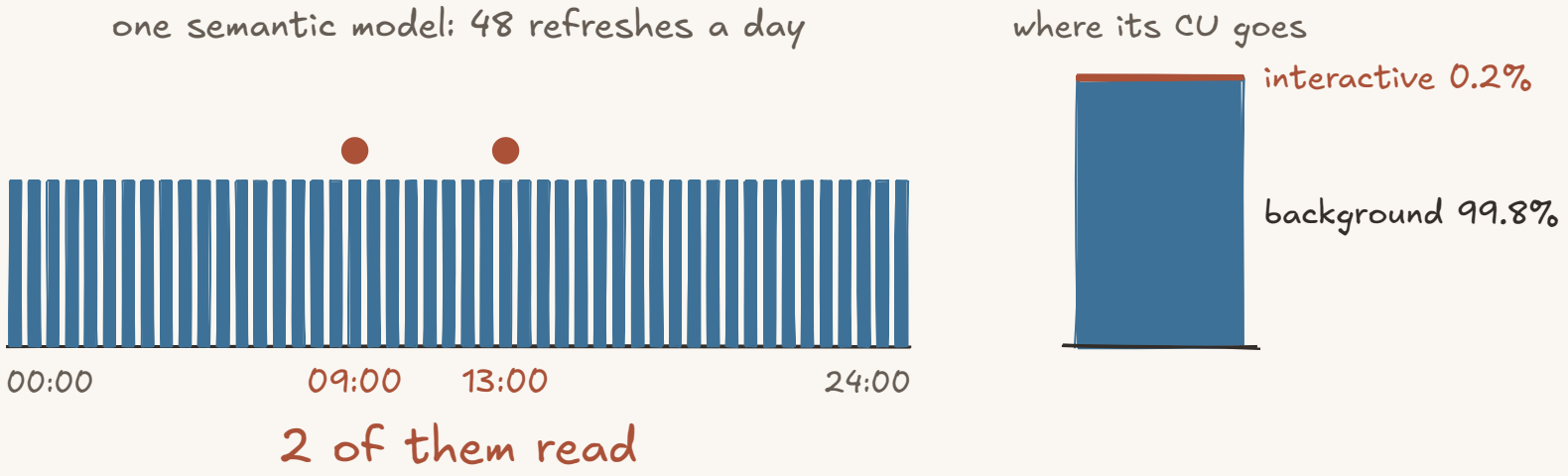
go and find out why



The patterns describe common behaviour, not all behaviour. Three anomalies worth recognising:

The staircase — evenly spaced, monotonically rising background consumption, indicating sessions that were opened and never closed, each logging further usage at a fixed interval. **The bolt from the blue** — an isolated interactive spike against an otherwise flat profile, typically a single expensive query against a large model. **The cliff** — sustained activity that stops abruptly

Refreshed 48 times, read twice.



Not wrong. But is it worth it?

Utilisation can be read by purpose as well as by shape. Interactive consumption is largely people querying data to reach an insight; background consumption is the preparation of that data. A semantic model whose consumption is almost entirely background is spending the capacity on refreshes rather than on use.

The illustrative model is refreshed every 30 minutes — 48 times a day, the scheduled maximum on a Fabric capacity — and

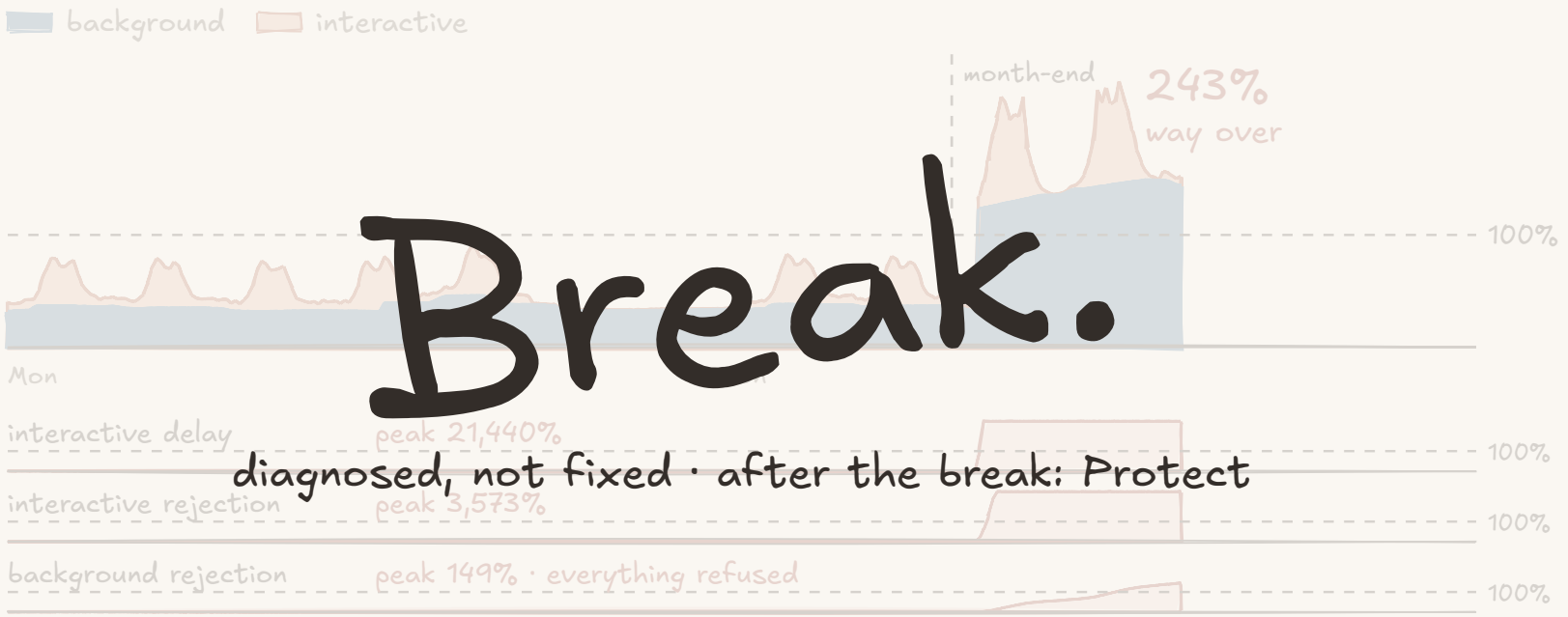
Thank you, partners.



Don't forget to visit the partners.

Thank you to the partners of dataMinds Connect 2026: Alistar, adesso, bmatix, Cartière, Cegeka, Cloubis, datashift, delaware, devart, Digital Excellence, element61, IRIS, JUVO, LACO, lytix, PlainInsight, Redgate, Scalefree, Sparkle, Tabular Editor, TriFinance and U2U. Please visit them during the break.

We leave it broken.



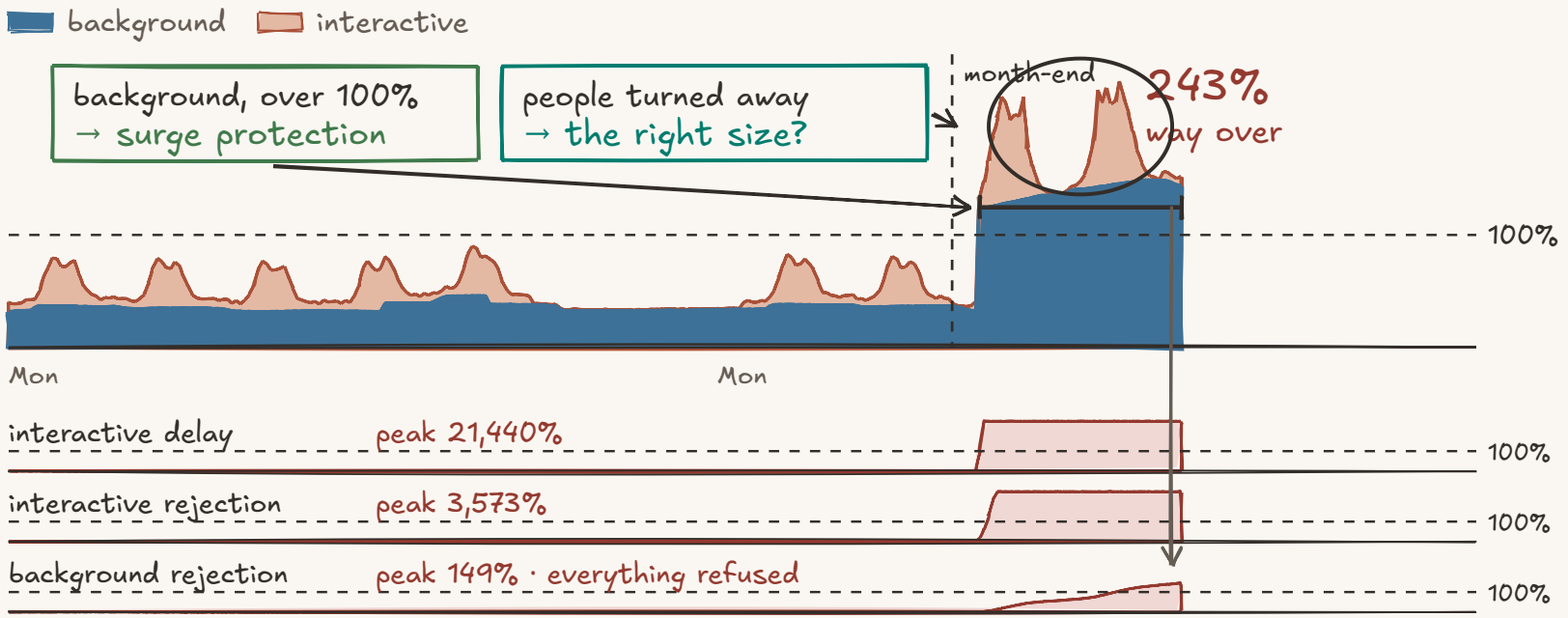
Break.

diagnosed, not fixed · after the break: Protect

The first half ends on an illustrative capacity holding three of the chapter's workspace patterns: a Storefront, a Factory and a Vault. Every ordinary day fits under the capacity's limit. At month-end the Vault's close coincides with the Factory's month-end batch — background work, smoothed over 24 hours — and utilisation rises to 243% of the capacity, above 100% for 48 hours.

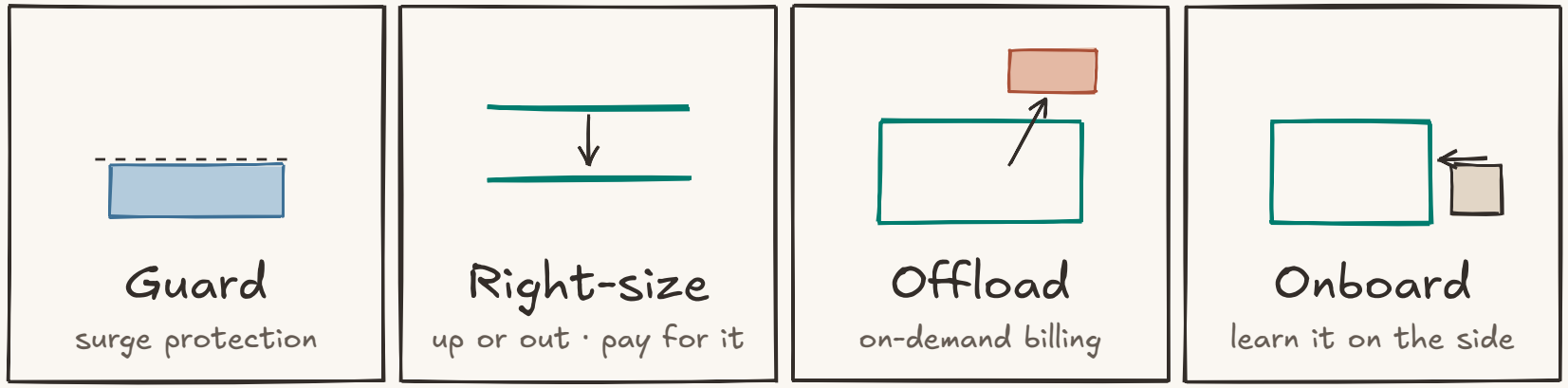
Throttling follows from the accumulated overage. Microsoft Fabric delays new interactive requests once more than 10 minutes

We left it broken.



The capacity left at the break holds a Storefront, a Factory and a Vault. Ordinary days fit under 100%; at month-end the Vault's close takes it to 243% and interactive work is rejected. Two things are happening: at month-end the background work alone is over the capacity — the Factory's batch under the Vault's close — taking the room interactive users need, which surge protection guards against; and the interactive work on top is turned away, which raises the question whether the box is the right size for its busiest days.

06 · Optimize and protect

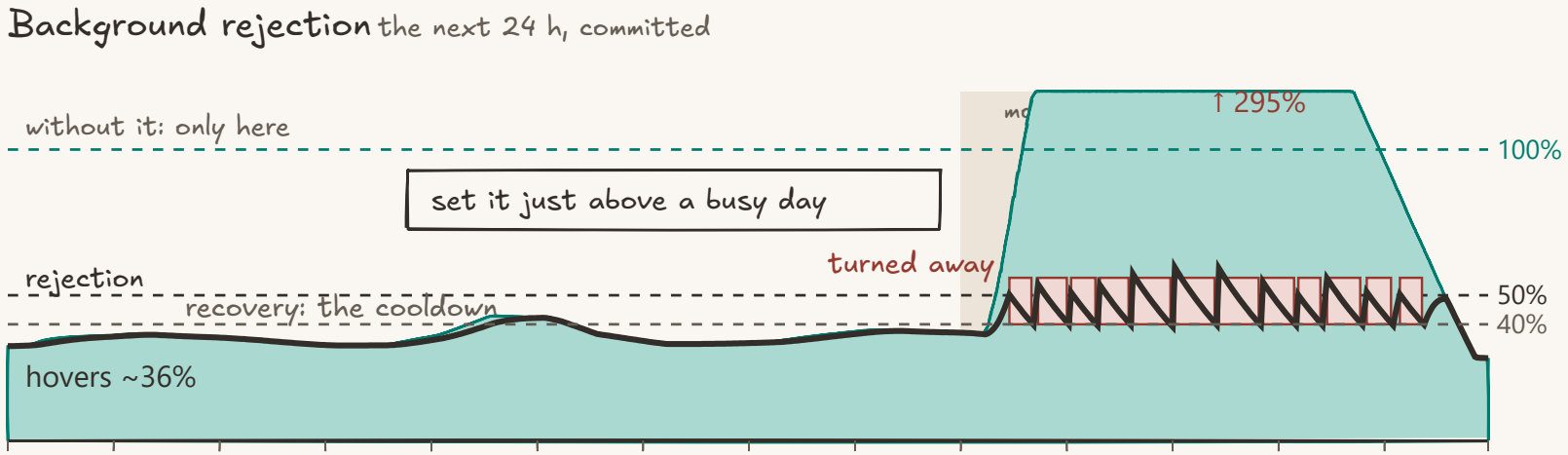


lighter work — the one lever you don't own

Four levers protect and right-size a capacity: guard it with surge protection, size it and choose how to pay for the size, offload work that can run elsewhere, and onboard new work on a separate capacity until its shape is known. Making the work itself lighter is cheaper still, and belongs to the people who build the content.

Optimize and protect: agreed storyline

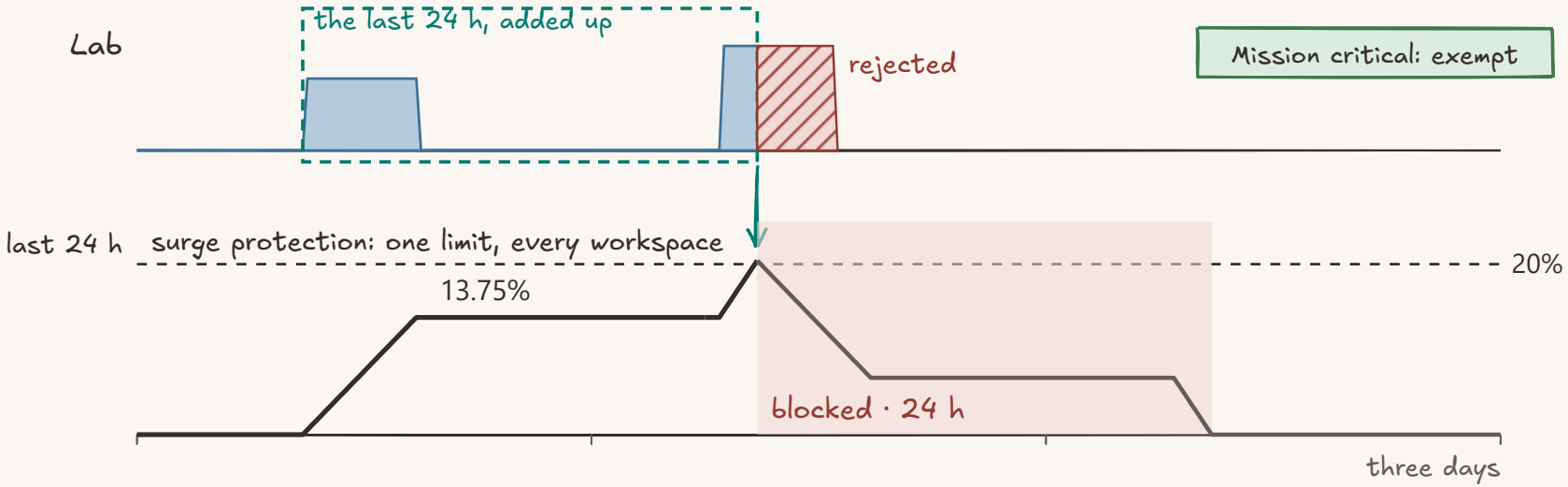
Surge protection: cool the background.



A protection for Power BI.

Capacity-level surge protection compares two thresholds with the capacity's 24-hour background percentage — the share of the next 24 hours already committed, shown on the Capacity Metrics app's Background rejection chart — not with raw background usage. Without it, new background work is rejected only at 100%. With it, reaching the rejection threshold turns new background work away; running jobs continue. Once the percentage falls below the recovery threshold, background work is accepted again. A starting point: rejection a little above a busy ordinary day, recovery near where the chart usually sits. The

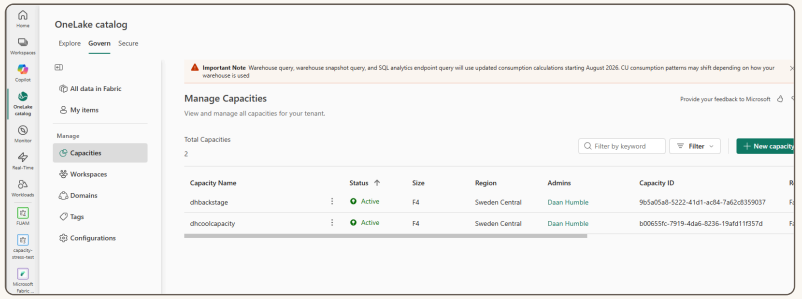
Surge protection, per workspace.



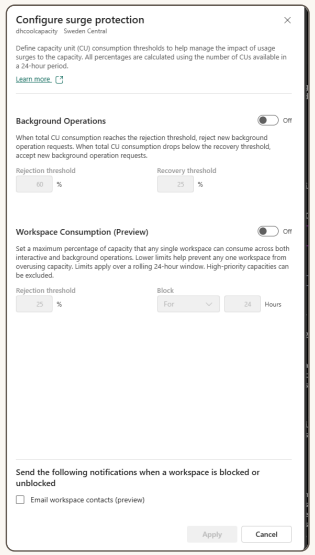
Blocked at the limit.

Workspace-level surge protection (preview) keeps a running total per workspace: its compute over the last 24 hours, as a percentage of the capacity's daily CU, checked every few minutes. The total rises while the workspace runs, holds while it is idle, and falls only as old usage leaves the 24-hour window. One limit applies to every workspace on the capacity. A workspace that reaches it is blocked — all new interactive and background operations are rejected — for a set number of hours or until an admin unblocks it. Mission critical workspaces are exempt from the limit, but not from capacity-level throttling. The 20% limit

Surge protection, live.



1 Govern › Capacities



2 Configure surge protection



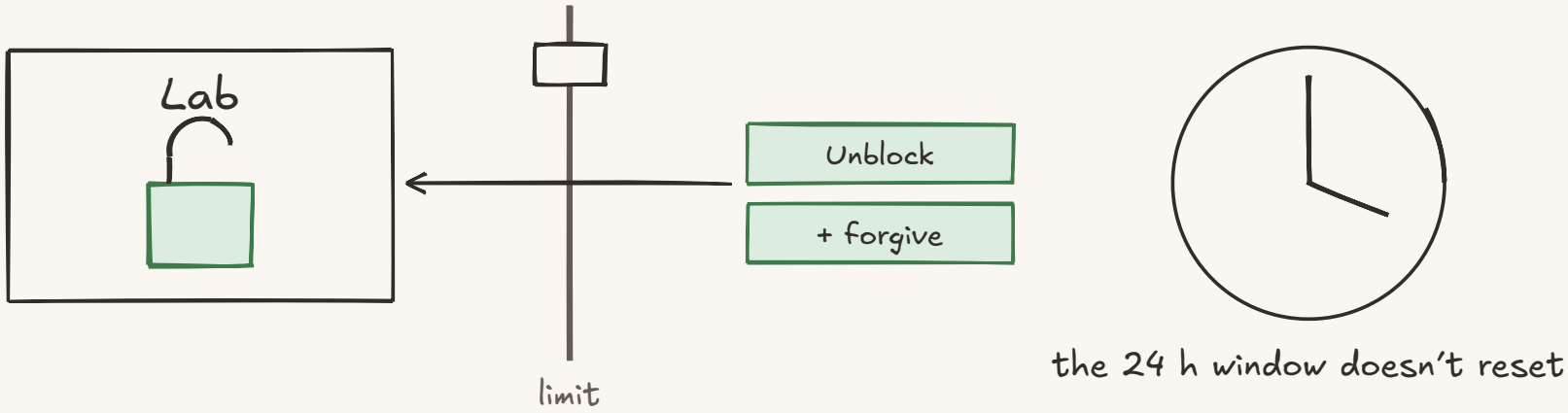
3 Workspace availability

Live on the demo tenant · fallback captured 2 Oct 2026

Both levels of surge protection are configured per capacity under OneLake catalog › Govern › Capacities, in the Configure surge protection panel: background rejection and recovery thresholds, and one workspace consumption limit with a block duration. Each workspace's availability — Available, Mission critical or Blocked — is set from the capacity's workspace list. For scale: an F2 has 48 CU-hours a day, so a 5% limit is 2.4 CU-hours.

Manage surge protection for Fabric capacities

The one to remember.

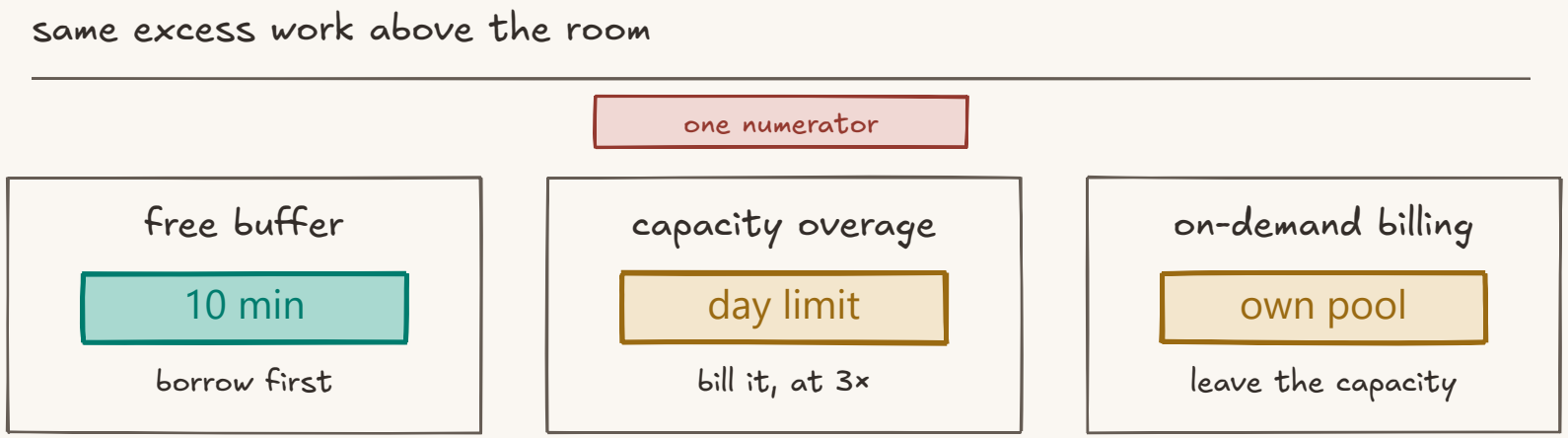


Raising the limit doesn't unblock.

Raising or deleting a workspace limit does not unblock a workspace that is already blocked. A block ends when its configured duration expires, when the workspace's consumption falls back within the limit, or when an admin unblocks it; an admin can also forgive the consumption already recorded. The rolling 24-hour window does not reset after a block, so without forgiveness a workspace can be blocked again at once.

Manage surge protection for Fabric capacities

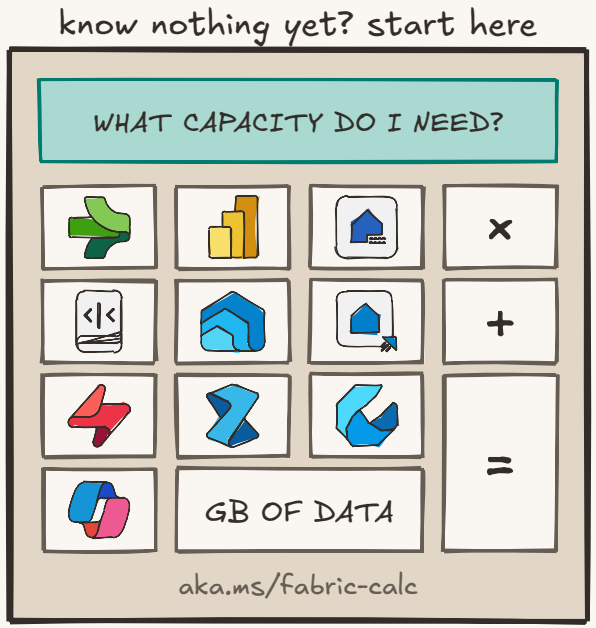
Three names, three jobs.

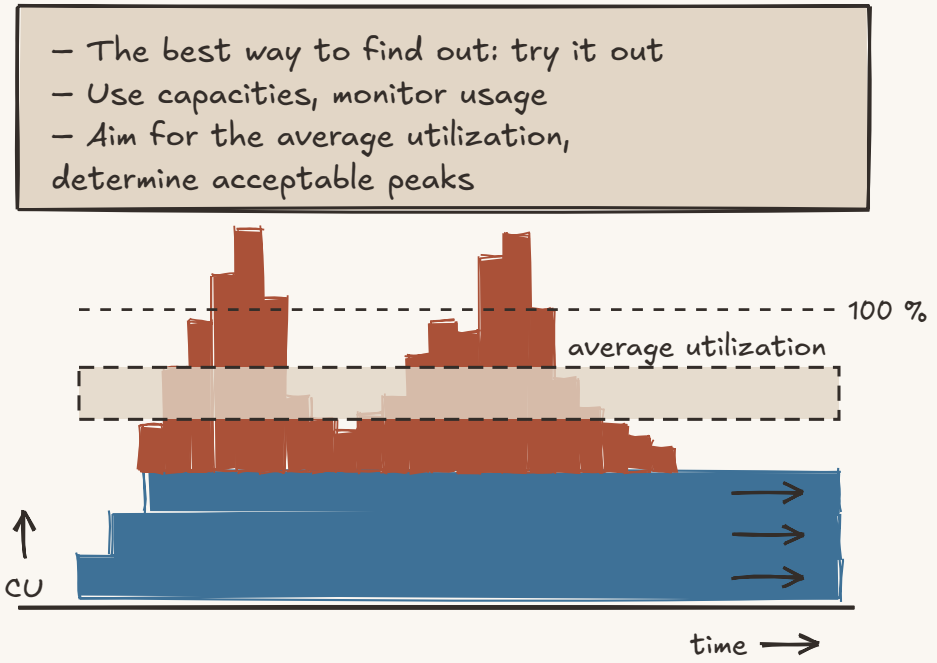


Three similarly named features do different jobs. The free buffer — Microsoft's documentation calls it overage protection — lets a capacity borrow ten minutes of future CU before throttling starts. Capacity overage bills usage above the SKU at three times the pay-as-you-go rate instead of throttling it, up to a limit. On-demand billing moves work to a separate pool billed per use. F SKUs have no autoscale.

Fabric throttling and the free buffer / Capacity overage in Microsoft Fabric / On-demand billing for Spark / Capacity settings — no autoscale on F SKUs

What size SKU do you need?

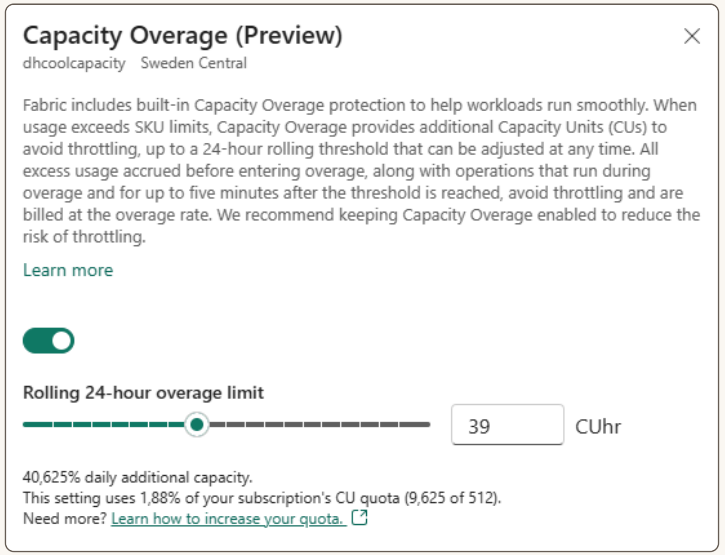




The best predictor of the capacity a workload needs is its past usage: try it on a capacity, monitor the usage, aim for the average utilization and decide which peaks are acceptable. With no history yet, the Fabric Calculator (aka.ms/fabric-calc) gives an initial estimate from the workloads and the volume of data. Treat that estimate as a starting point to be checked against measured usage.

Scale your Fabric capacity / Microsoft Fabric licenses and capacity SKUs

How high do you set it?



a rolling 24 h limit

in CU-hours, not a cap: running work finishes

it spends your quota

a 24th of the limit, as CU

so how high?

Microsoft: under ⅓ of your day

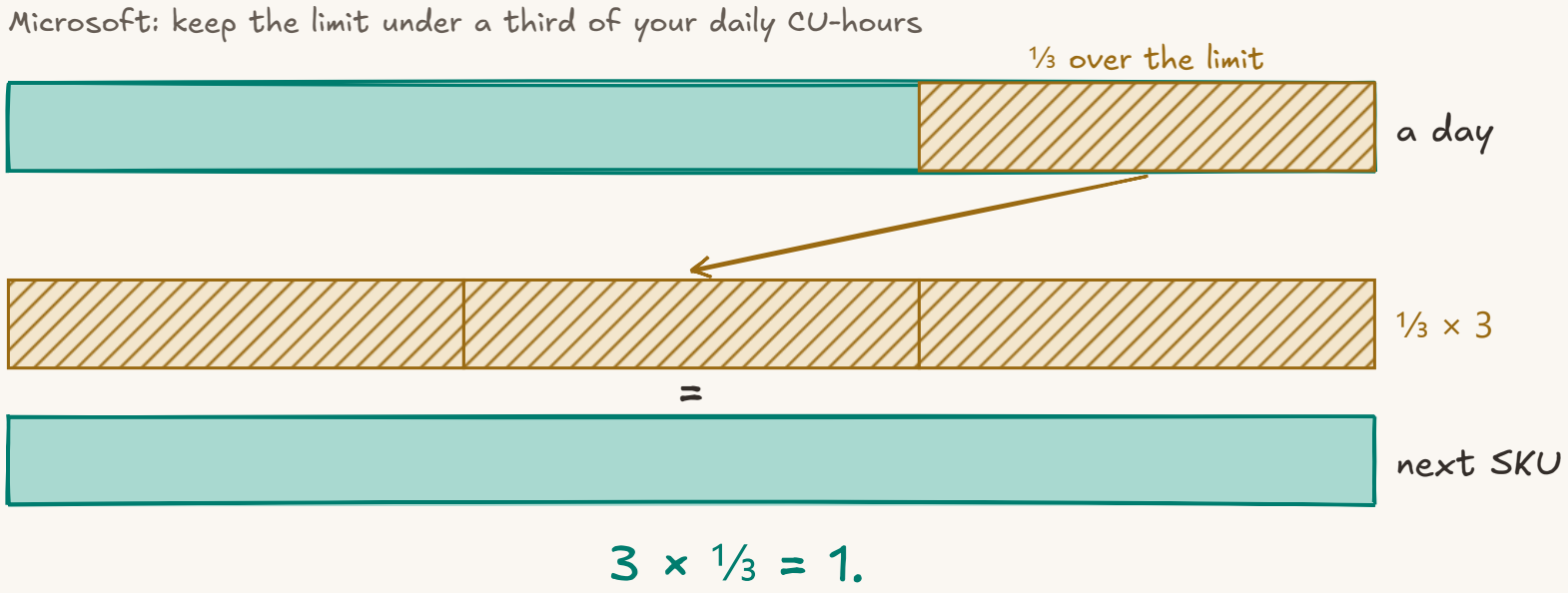
Captured 2 Oct 2026 · an F4 on the demo tenant

Capacity overage is set as a rolling 24-hour limit in CU-hours. It is a spending threshold, not a hard cap: work already running continues, so charges can exceed it. The limit uses Fabric quota — a twenty-fourth of the limit, in CU. On the F4 shown, a 39 CU-hour limit is about 40% of the day's 96 CU-hours. Microsoft advises keeping it under a third of the capacity's daily CU-hours.

Capacity overage in Microsoft Fabric / Fabric quotas

Why a third?

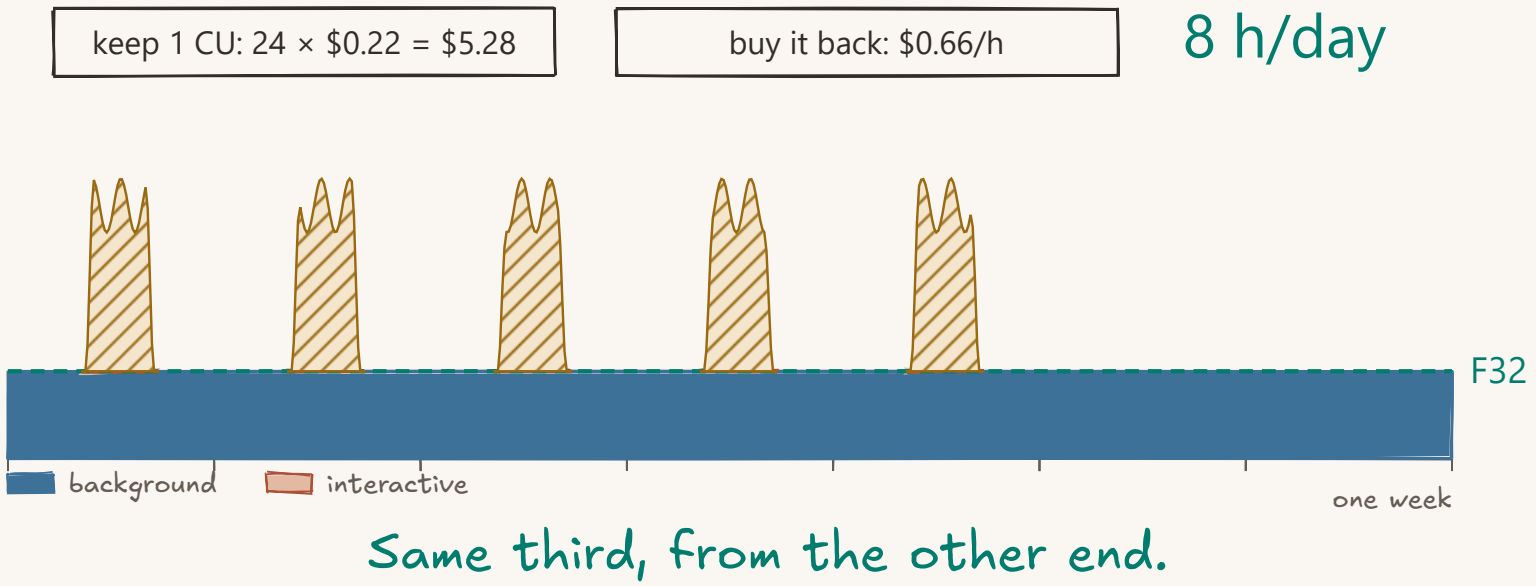
3 × ⅓ = 1



Microsoft advises keeping the overage limit below one third of the capacity's daily CU-hours, the point at which costs match scaling up. The reason is arithmetic: overage costs three times the pay-as-you-go rate, and a third of a day at three times the price is one full day — the next SKU up.

Capacity overage in Microsoft Fabric / Optimize and protect: the arithmetic behind every number

Size for the floor. Buy the spike.

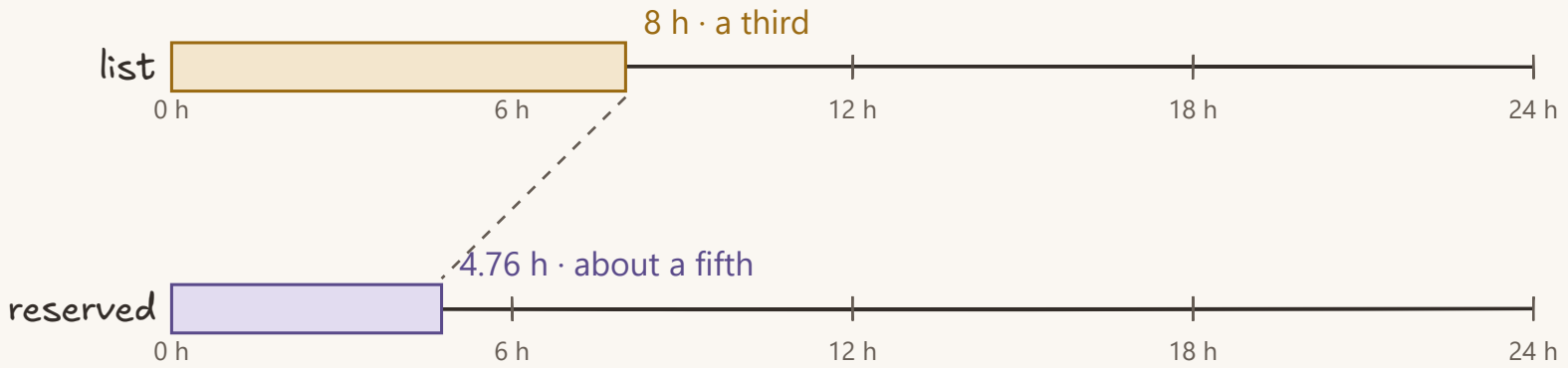


Same third, from the other end.

Turned round: keeping one CU costs 24 h × $0.22 = $5.28 a day at pay-as-you-go; buying it back as overage costs $0.66 an hour. Below 8 hours of use a day, the smaller SKU plus overage is cheaper — the same third of the day.

Capacity overage in Microsoft Fabric / Optimize and protect: the arithmetic behind every number

The better your baseline deal...



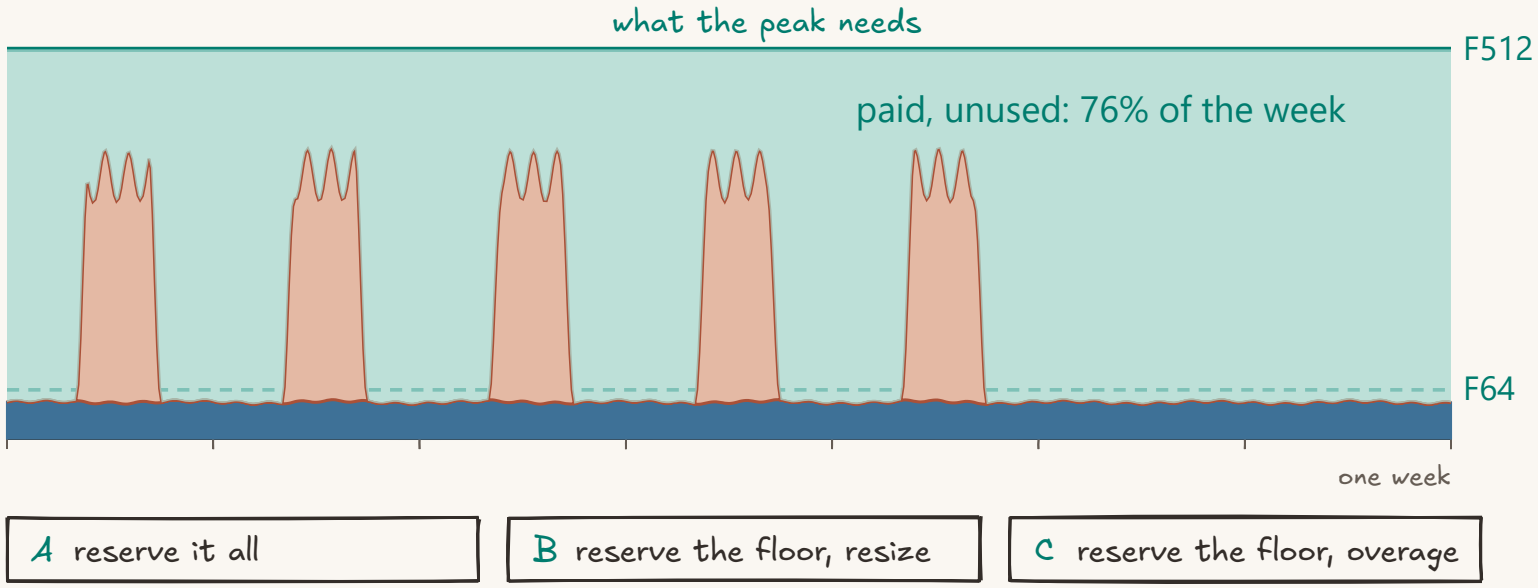
negotiate well: $0.1308/CU-h reserved ≈ ⅕ of the $0.66 overage · 5.05×

One third at list. A reservation at about ⅕ of the overage price — if you negotiated well.

A good reservation lowers the price of keeping a CU to $0.1308 an hour — about one fifth of the overage price, 5.05 times cheaper at the list reservation rate; a weaker deal gives a weaker ratio. The break-even falls from 8 hours a day to 4.76: about a fifth of the day. The better the baseline price, the worse the peak trade.

Optimize and protect: the arithmetic behind every number / Capacity overage in Microsoft Fabric

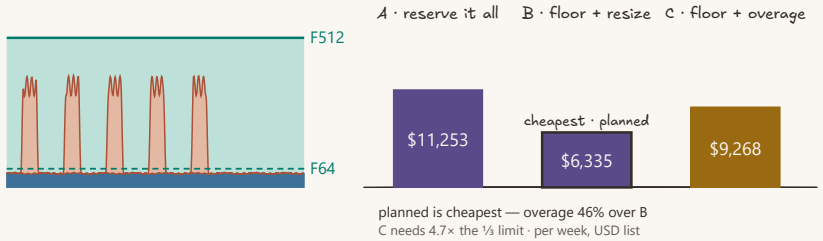
A floor, and an island.

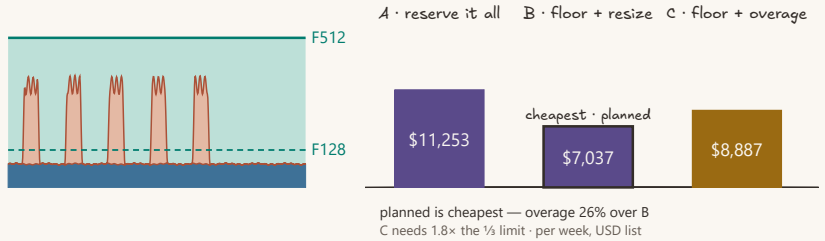


An illustrative week: a steady background floor around 48 CU, below F64, and a weekday island of interactive work peaking near 383 CU, which needs an F512. Sized for the peak, 76% of the capacity's week is paid for and unused. Three ways to pay for it: reserve the whole F512; reserve the floor's F64 and resize for the island; or reserve the F64 and buy everything above it as capacity overage.

Optimize and protect: the arithmetic behind every number

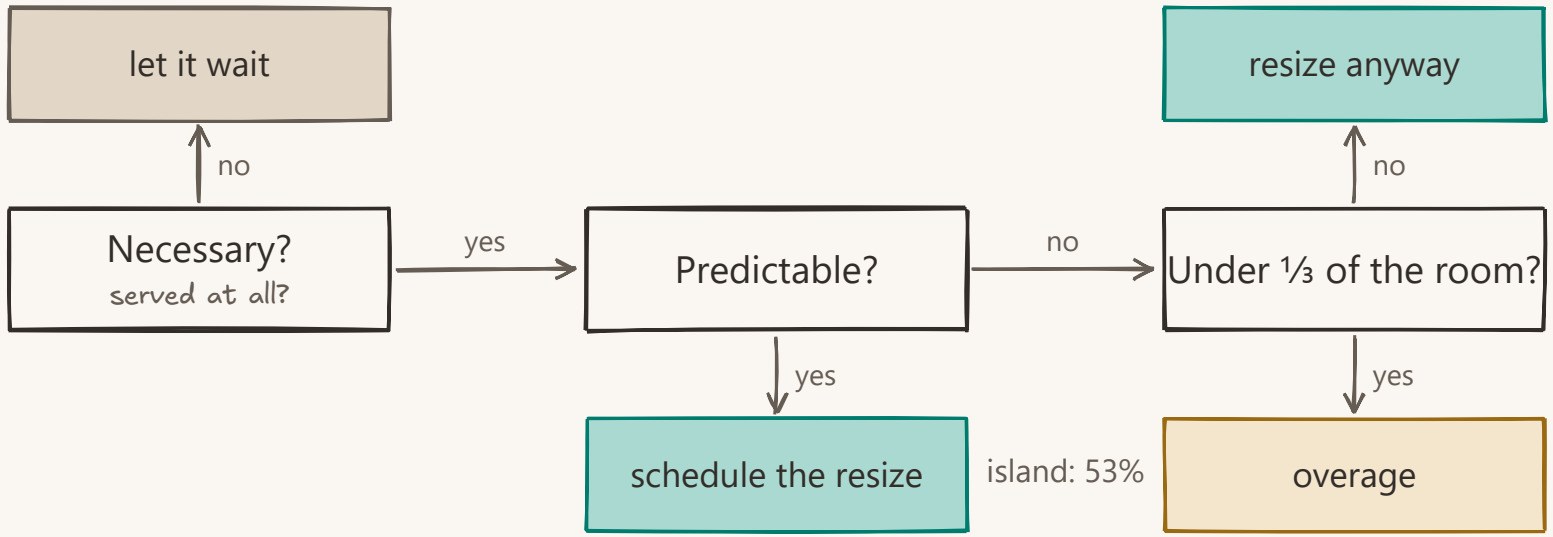
Three ways to pay for the island.





Three ways to pay for the island week, at list prices: A reserves the F512 the peak needs ($11,253 a week); B reserves the floor's F64 and resizes to F512 for the island ($6,335); C reserves the F64 and buys the island as capacity overage ($9,268), which needs a daily limit 4.7 times Microsoft's one-third guidance. A floor above F64 doubles the floor's SKU and its reservation; The resize wins in every state of the desk, if the island is predictable; a shorter or lower island only narrows the gap to C — at a two-hour island overage comes close but is not cheaper. Overage is the price of not having to predict

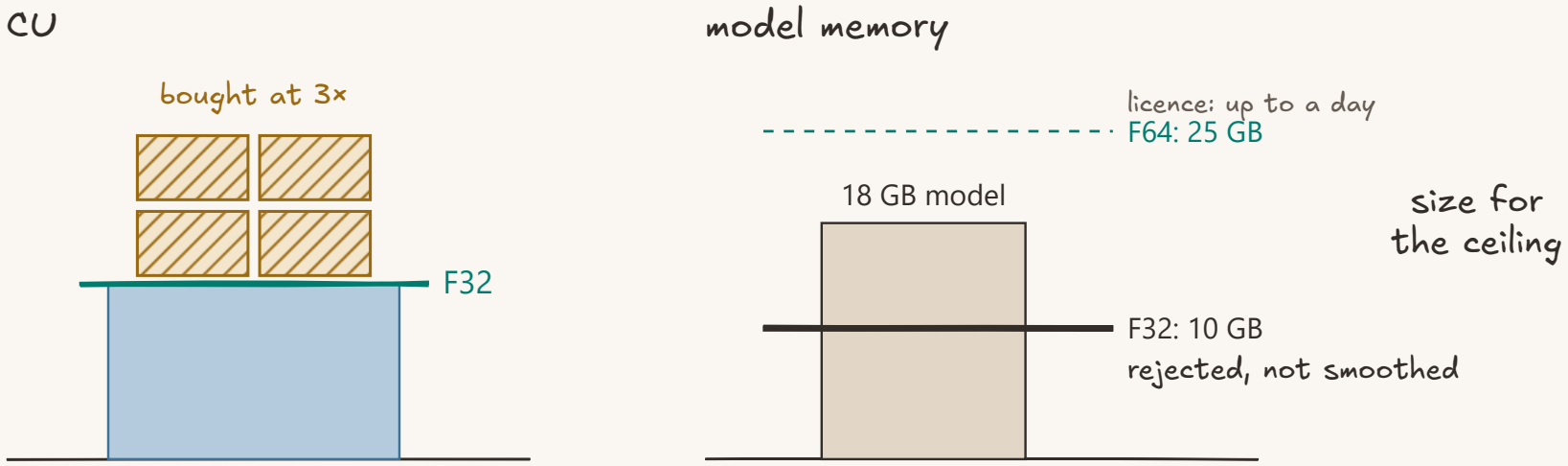
Three questions, in order.



Everything here is one third.

A decision tree of three questions. Is the work necessary — does it have to be served at all? If not, let it wait or make it lighter. If it is: is the peak predictable? Yes — schedule the resize and reserve the floor. No — would overage use under a third of the room a resize adds? Yes — buy overage. No — resize anyway. On the island, overage would use 53% of the room a resize adds, so the resize wins.

What overage can't buy.

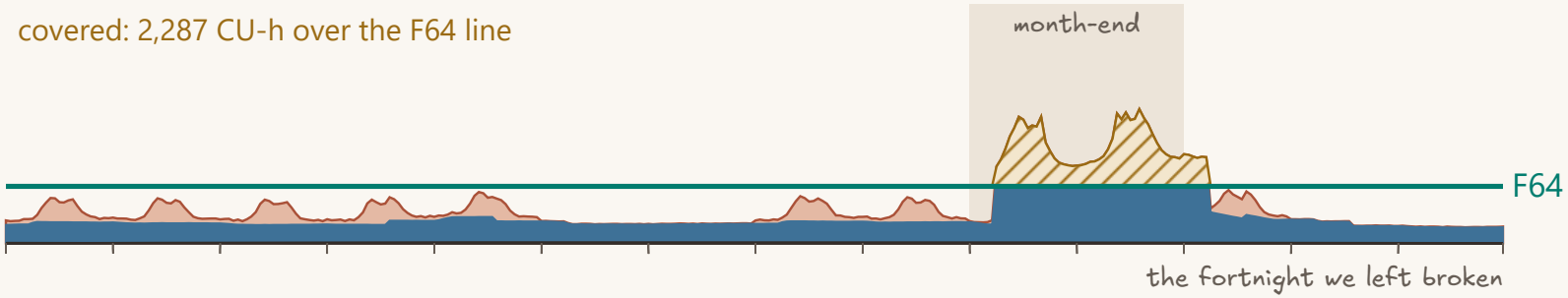


Overage buys CU-hours. Not ceilings.

Overage buys CU-hours, not the SKU's ceilings: memory, burst and node limits scale with the SKU, and past them work is rejected rather than smoothed. A job can fail on a healthy-looking capacity for that reason. Free viewers also require F64 or larger regardless of the cost arithmetic. Crossing F64 can take up to a day for licences to update.

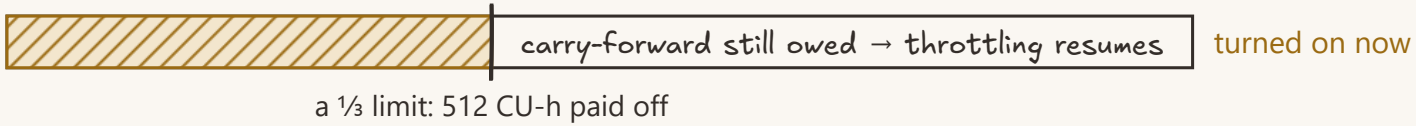
Capacity overage in Microsoft Fabric / Warehouse burstable capacity / Microsoft Fabric licenses and capacity SKUs / Scale your Fabric capacity

Switch it on mid-incident?



2,287 CU-h × $0.66 = $1,509

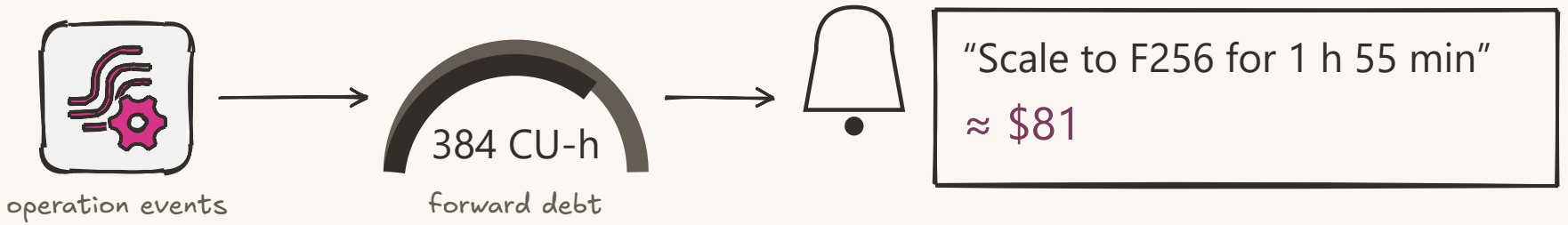
the whole fortnight's overage, USD list · its busiest 24 h: 1,252 CU-h, 82% of the day



It pays what the limit allows — not everything owed.

On the F64 we left broken, capacity overage would have covered the 2,287 CU-hours above the line, at three times pay-as-you-go: about $1,509 at list price for the fortnight. The busiest 24 hours hold 1,252 CU-hours, 82% of the day, well past the one-third guidance. Enabling overage during throttling bills the cumulative carry-forward at the point throttling would start, but only up to the rolling 24-hour limit; beyond it throttling resumes until usage leaves the window. Capacity overage is on by default for new capacities.

An alert with a price.



$$S' = L + D \div T$$

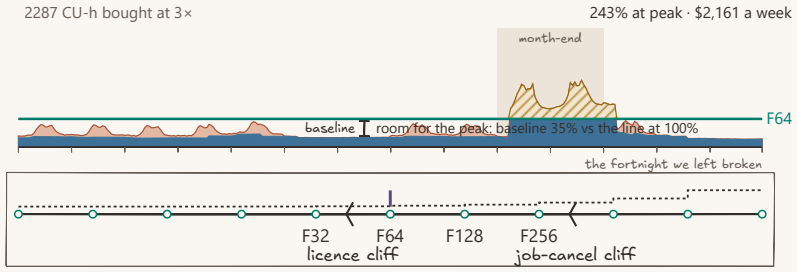
56 + 384 ÷ 2 = 248 CU → F256

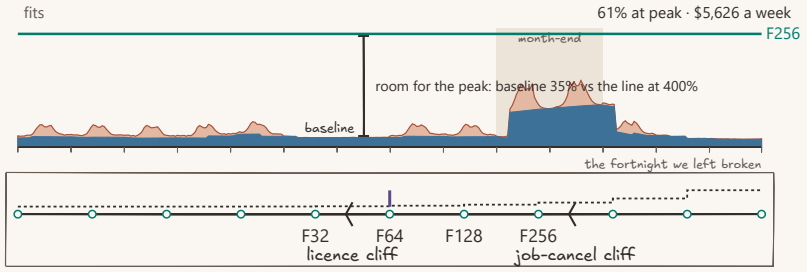
describe | compare | **react** | predict | act

won't match the Metrics App · can't see ceilings

Fabric capacity operation events expose the forward capacity overage live: the sum of CU in operations whose smoothing window ends in the future. Inverting the recovery formula gives the size needed to be healthy within a target time, $S' = L + D \div T$: within two hours here, 248 CU, so F256. An alert can therefore carry its own recommendation and price. The figures will not match the Capacity Metrics app exactly, and guardrails are not visible in the events. It is a method still to be built, not a product feature.

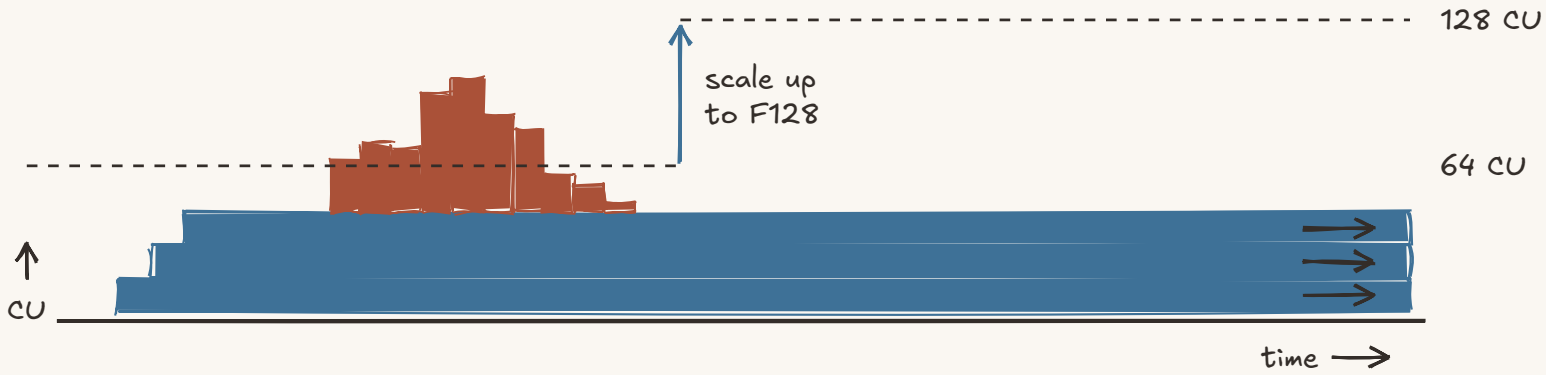
Resize the capacity we left broken.





The broken capacity's fortnight on three SKUs. On F64, the month-end goes 48 hours over the line and is throttled. With capacity overage on, those 2287 CU-hours are bought at three times pay-as-you-go: about $2,161 a week reserved plus overage. An F128 still goes over for 14.5 hours; an F256, on which everything fits, costs $5,626 a week reserved. F SKUs have no autoscale; they resize. Crossing F64 can take up to a day for licences, moving between F256 and F512 can cancel running jobs, and scaling below the reserved size does not lower the bill. Figures are illustrative, at list prices.

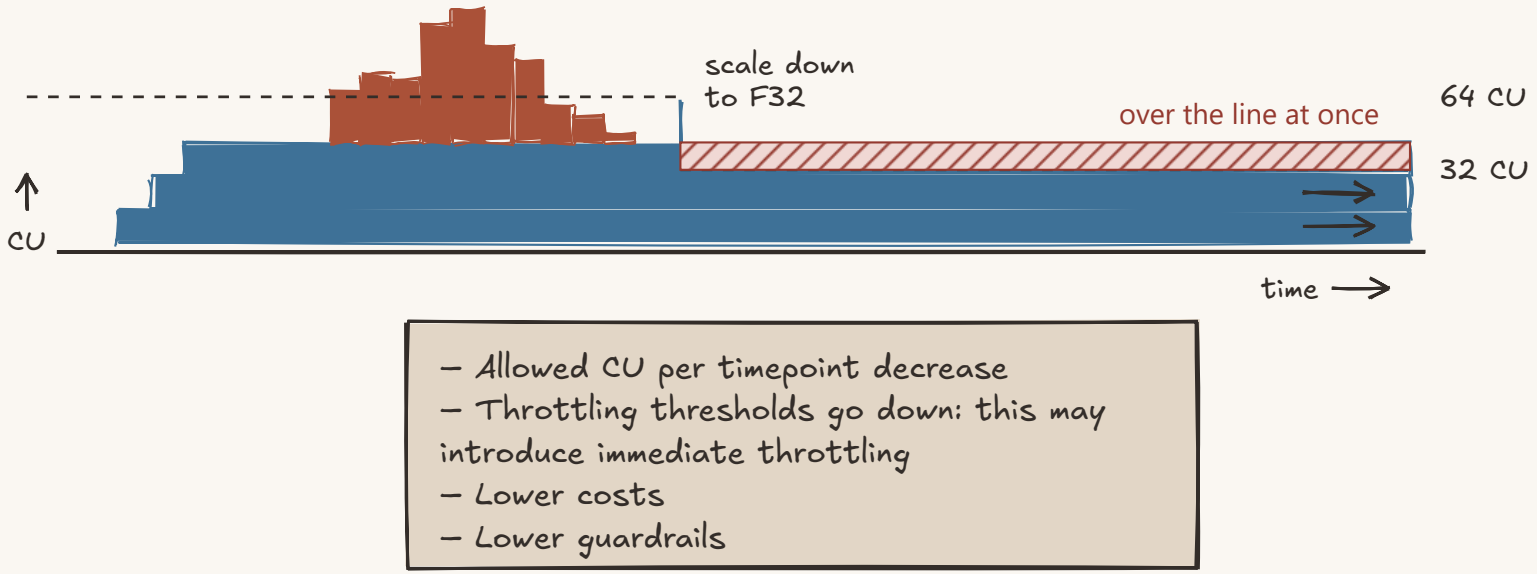
Scale up the capacity.



- Allowed CU per timepoint increase
- Throttling thresholds go up
- "Easy" remediation
- Higher costs
- Bad actors can scale with the capacity

Scaling a capacity up raises the allowed CU at every timepoint, and the throttling thresholds, which are measured against the SKU, rise with it. Carried-forward work is burned down faster because each timepoint has more idle capacity, so remediation is straightforward. The costs are higher, and a workload that misbehaves also gets more room. Moving between F256 and F512 can cancel running jobs. The same work reads as a lower percentage of a larger capacity, so average-utilization targets and acceptable peaks need re-checking after a resize.

Scale down the capacity.

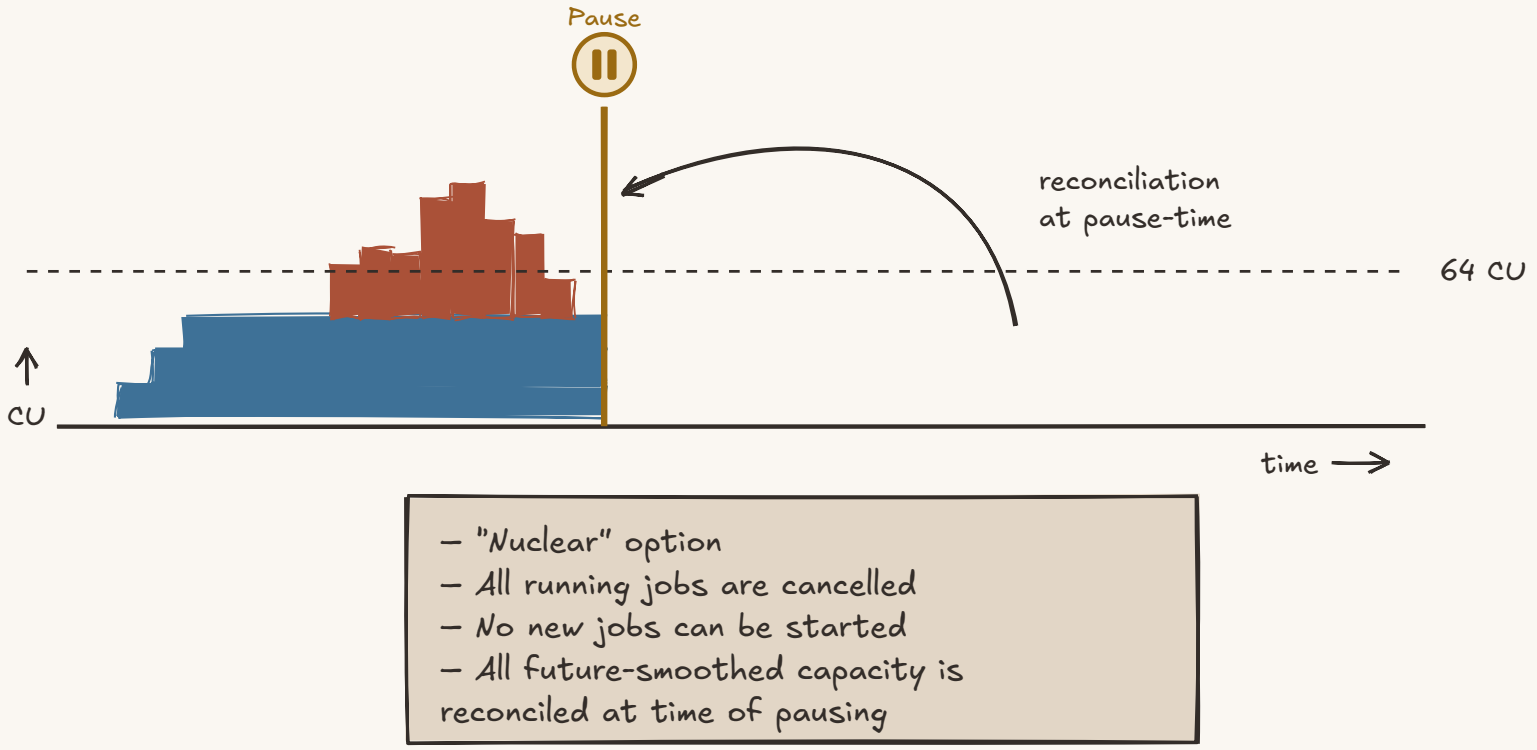


- Allowed CU per timepoint decrease
- Throttling thresholds go down: this may introduce immediate throttling
- Lower costs
- Lower guardrails

Scaling a capacity down lowers the allowed CU at every timepoint and the throttling thresholds with them. If the baseline background load already sits above the smaller size, throttling can start immediately. Costs fall, and so do the guardrails: the per-capacity ceilings. The same work is a larger share of a smaller capacity, so average-utilization targets are reached sooner. Check the baseline against the new size first; scaling below a reservation does not reduce the bill.

Scale your Fabric capacity / Capacity overage in Microsoft Fabric / Fabric throttling and the free buffer

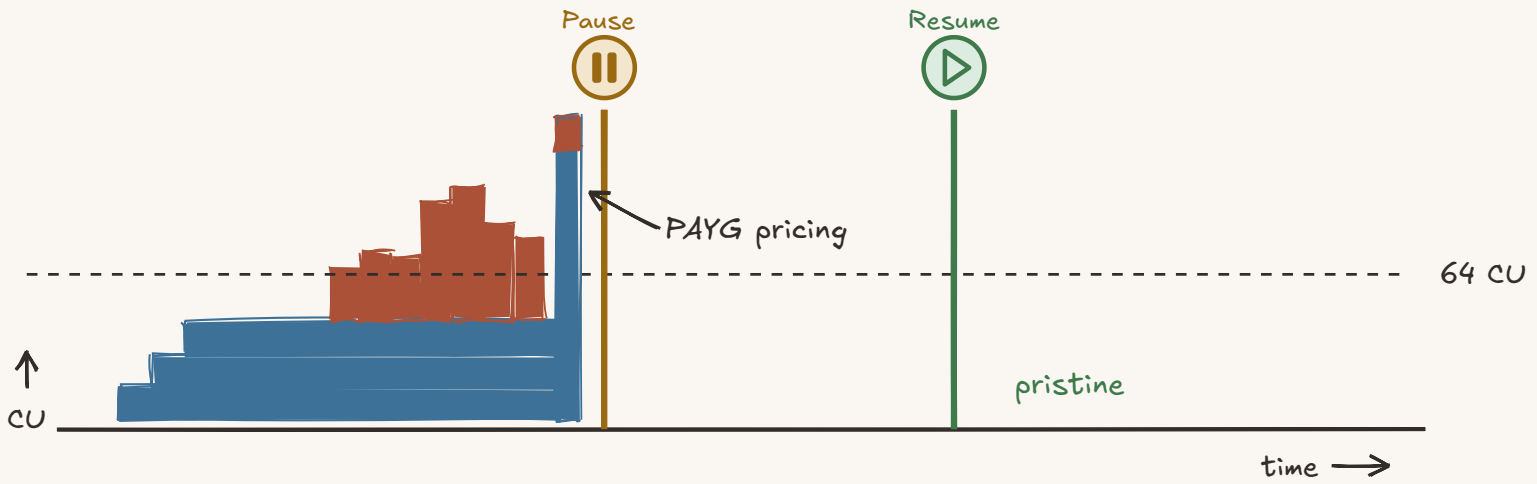
Pause the capacity.



Pausing a capacity is the nuclear option: all running jobs are cancelled and no new jobs can start. Remaining overages and all smoothed, future-dated operations are summed and reconciled at the moment of pausing, then added to the Azure bill. Content on the capacity is unavailable while it is paused. The next chapter works the cost of a pause in detail.

Pause and resume a capacity / Fabric throttling and the free buffer

Resume the capacity.

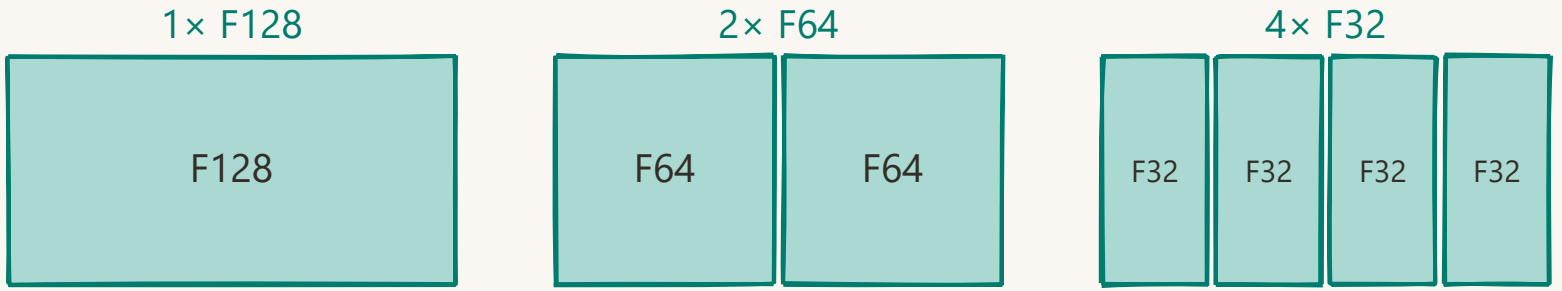


- Reconciled CU show up on the Azure bill at the pay-as-you-go price
- At resume, utilization is pristine
- Consider the costs

When a paused capacity is resumed, the CU that were reconciled at the pause appear on the Azure bill at the pay-as-you-go price, and the capacity's utilization starts clean, with no carried-forward work. Weigh that cost against the alternatives before pausing. The Capacity Metrics app shows a spike around a pause.

Pause and resume a capacity

Up — or out?



Same price. The ceilings don't add up.

Scaling up means one bigger capacity; scaling out means more, smaller ones. 128 CU costs the same either way — $28.16 an hour at list price for one F128, two F64s or four F32s — because Fabric compute is priced linearly. The per-capacity ceilings do not add up: the largest semantic model is 50 GB on an F128, 25 GB on an F64 and 10 GB on an F32, and free viewers need F64 or larger.

Microsoft Fabric licenses and capacity SKUs / Scale your Fabric capacity / Design: scale up or out, the scorecard and its arithmetic

Which gives you room to move?

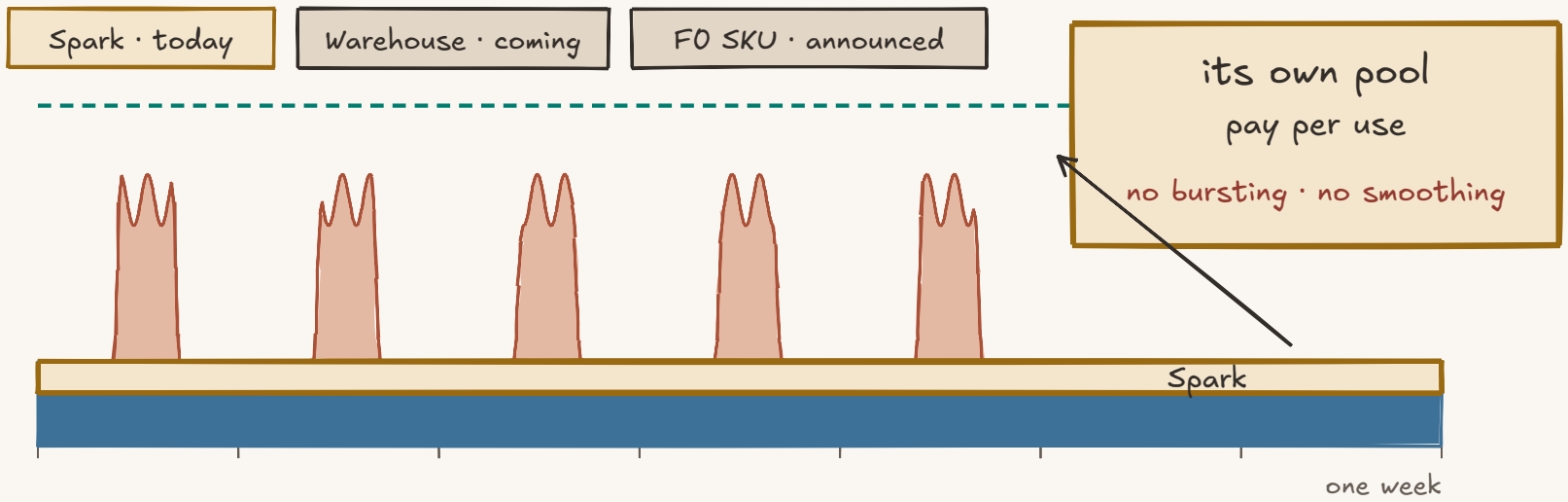
		1× F128	4× F32
	~~Price~~	~~the same~~	
limits	Licensing	●	
	Guardrails	●	●
	Direct Lake	●	
behaviour	Blast radius		●
	Elastic surface	●	
running it	Admin effort	●	
	Chargeback		●
	Reversibility	●	

Up gives room. Out gives walls.

At the same price, the choice between one larger capacity and several smaller ones is about room to move. One larger capacity gives more licensing headroom, higher per-item guardrails and Direct Lake limits, a larger elastic surface for bursts, less admin effort and an easier way back. Several smaller ones give walls: a smaller blast radius when one workload misbehaves, and a simpler chargeback per owner.

Design: scale up or out, the scorecard and its arithmetic / Microsoft Fabric licenses and capacity SKUs

Work can leave the room.

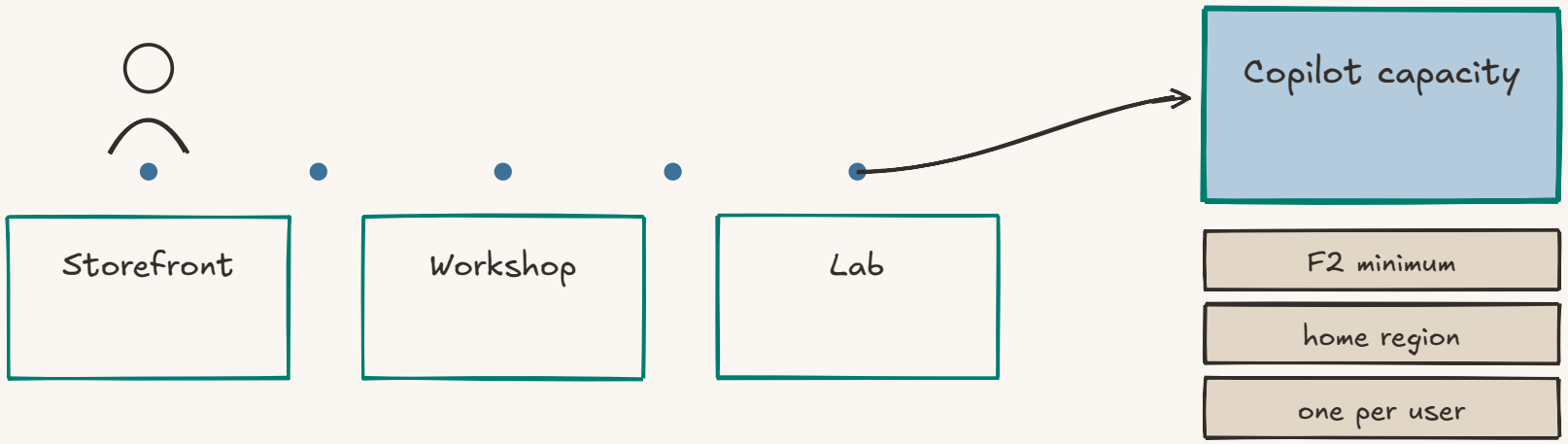


Pay per use when the work leaves the room.

On-demand billing runs work outside the capacity in a separate pool, billed per use, with none of the capacity's ceilings — and no bursting or smoothing. Spark has it today (on-demand billing for Spark, formerly Autoscale Billing for Spark). On-demand billing for Data Warehouse, and per-billing-category on-demand billing — which applies only to the zero-provisioned F0 SKU — were announced in Microsoft's Fabric September 2026 feature summary as coming soon; prices were not yet published.

On-demand billing for Spark

Copilot follows the person.

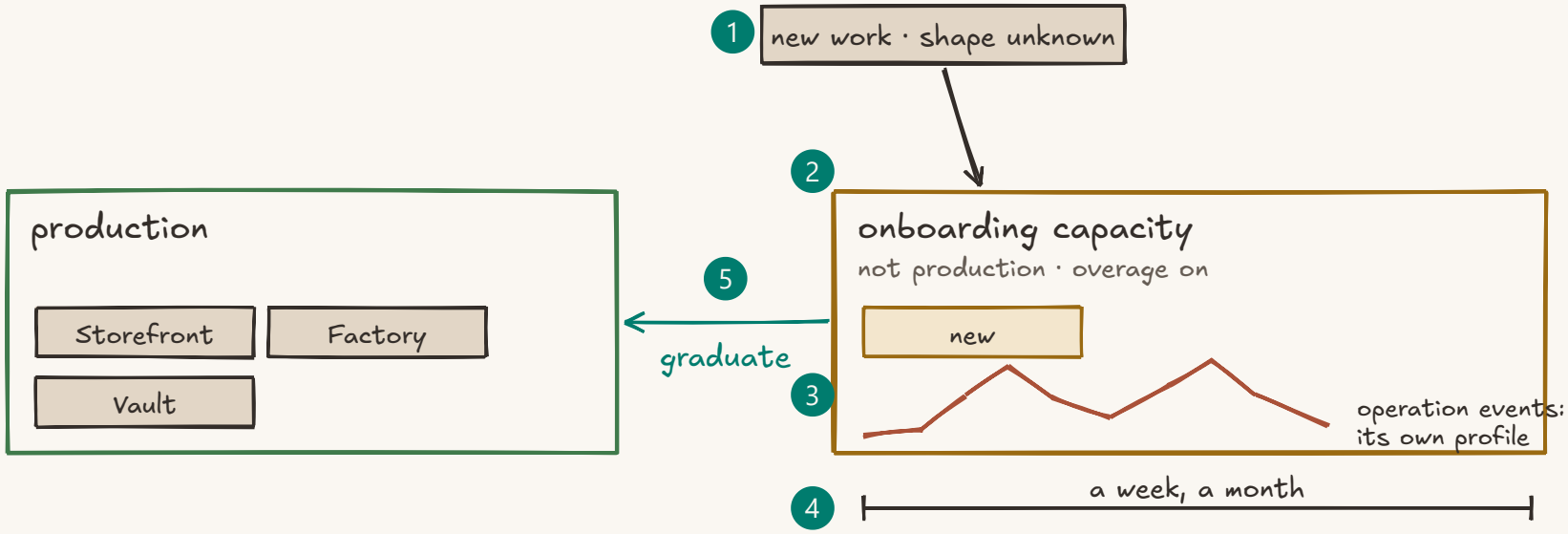


The person asks. The compute happens in the capacity — move the semantic model with the work.

A Fabric Copilot capacity collects a user's Copilot usage wherever they work, instead of billing it to each workspace's capacity. It needs an F2 or larger in the home region, and each user is assigned to one. Copilot usage is background work, smoothed over 24 hours and billed by token.

Fabric Copilot capacity

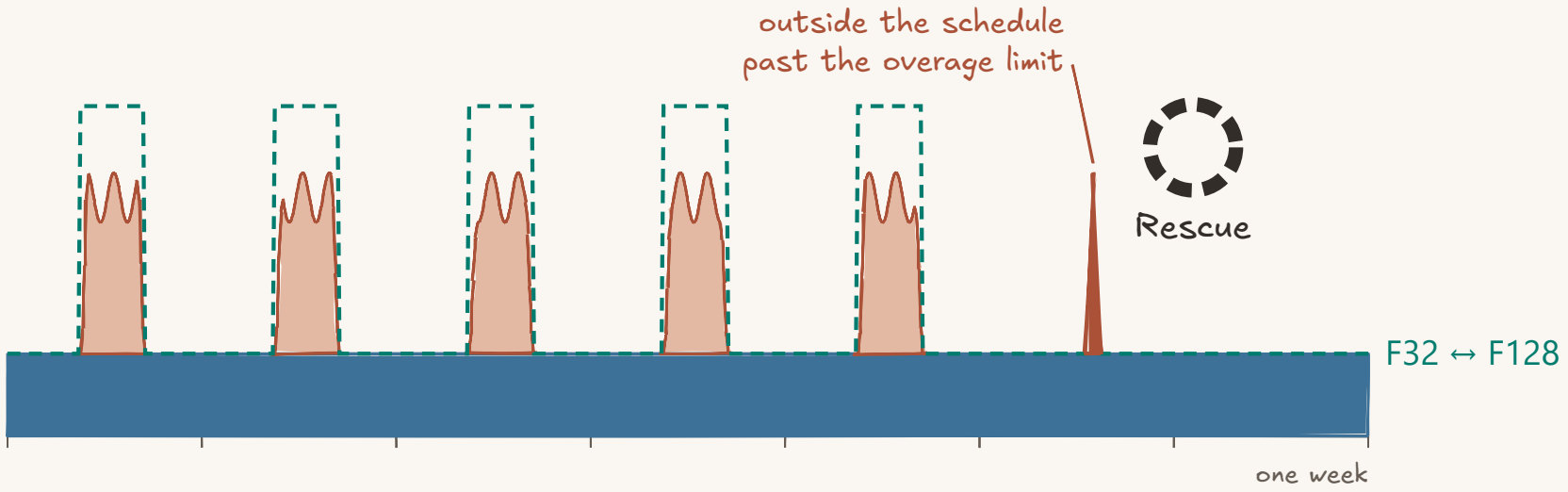
New work has no history.



Learn its shape first.

New workloads have no usage history to size from. Put them first where they cannot interfere with known load: an onboarding capacity, which can be shared by several new workloads and need not be production — capacity overage can be enabled there. Capacity operation events attribute every operation to its workspace and item, so each newcomer gets its own profile even on a shared capacity. Run for an agreed window — a week or a month, depending on the work — then move it to production against criteria agreed in advance. Moving a workspace between capacities can interrupt active refreshes and queries, so

The Tuesday you didn't plan.



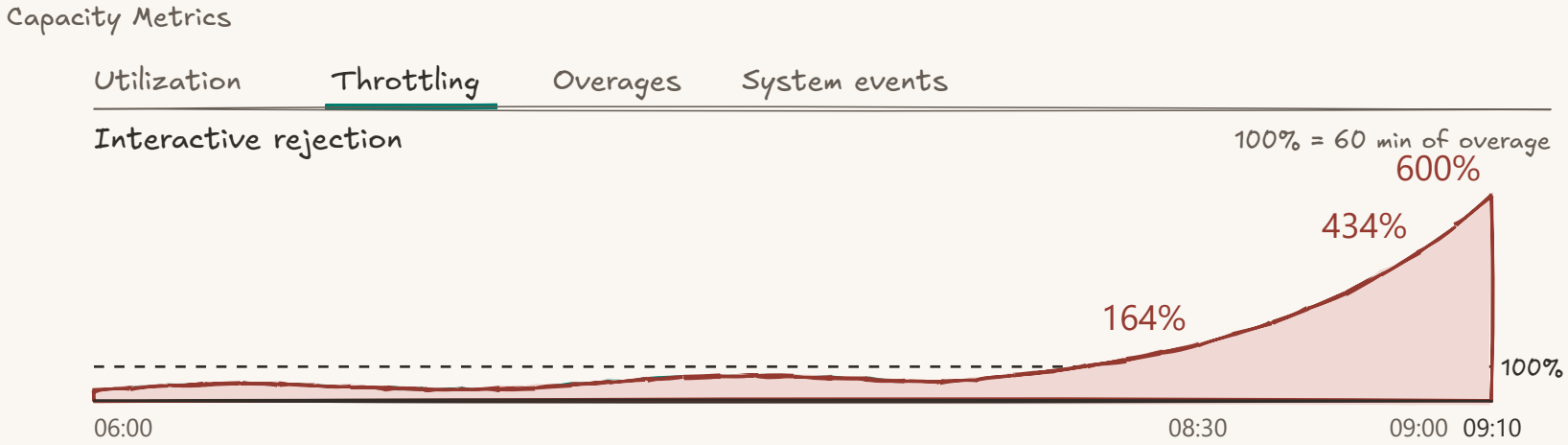
Guarding, right-sizing, offloading and onboarding protect a capacity against the load you can predict, and make an unplanned spike rarer — not impossible. A spike outside the scheduled resize, or past the overage limit, still breaks through. The next chapter covers what to do then.

Optimize and protect: agreed storyline

09:10 on the meter **$0.00/h**

Tuesday, ten past nine.

09:10 · on the meter · nothing extra yet



CFO "Close report won't load."

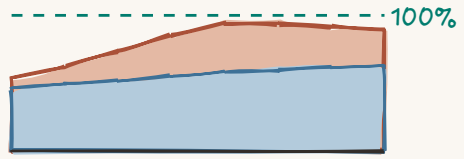
The chapter starts where detection ends. The Capacity Metrics app's Throttling tab shows a production F64 climbing on the Interactive rejection graph, where 100% stands for sixty minutes of overage: 164% at 08:30, 434% at 09:00, 600% at 09:10. Fabric is rejecting interactive requests, so the month-end close report does not load. Until somebody acts, nothing beyond the capacity's own price is being paid. The question now is which move restores service, and what each move costs and breaks.

Capacity Metrics app: the Throttling and Overages tabs / Capacity Metrics app calculations: what a throttling percentage means / Fabric throttling / Rescue: storyline

09:12 on the meter **$0.00/h**

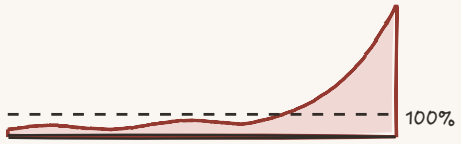
Three questions before any button.

CU or ceiling?



CU.
ceiling: not today

How deep?



600% on Interactive rejection

Still running?



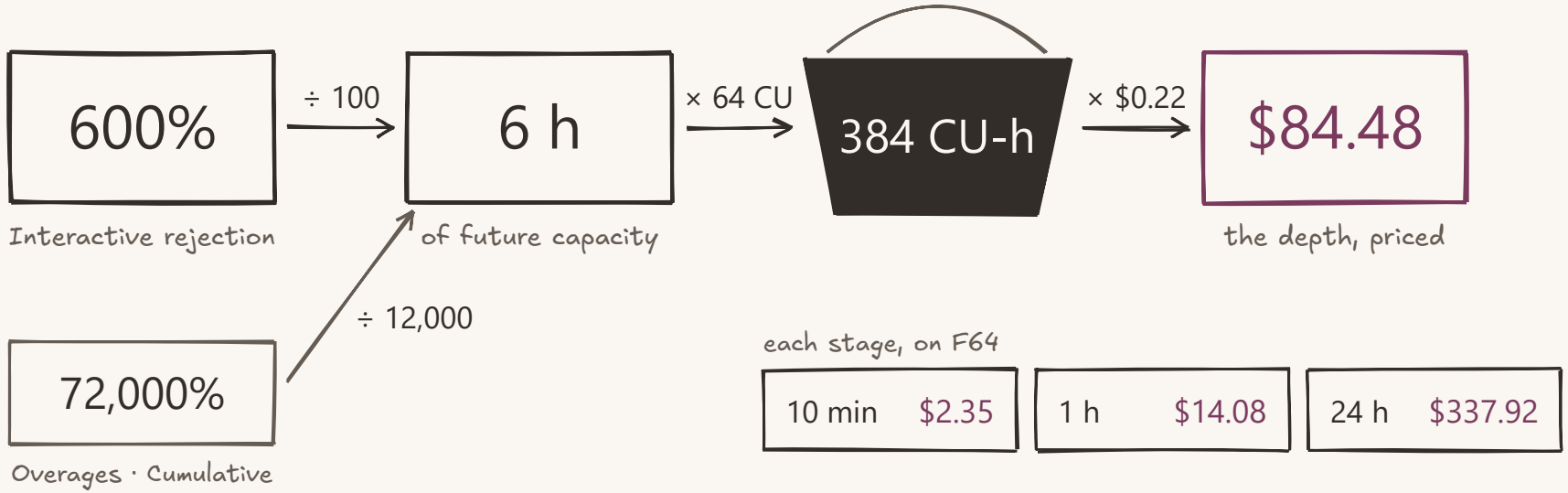
Yes: the Lab, in flight

Before choosing a move, answer three questions. Is it CU at all, or a per-SKU ceiling such as memory or Spark concurrency that no amount of CU fixes? Users see the difference: CU overload reaches them as “Your organization's Fabric compute capacity has exceeded its limits” or CapacityLimitExceeded. How deep is the capacity overage? The Throttling tab shows it as a percentage of each stage's window; here Interactive rejection reads 600%. And is it still running: operations already running are not throttled, so the job that caused the problem keeps running and keeps adding to it.

09:12 on the meter **$0.00/h**

The debt, in dollars.

Interactive rejection ÷ 100 = hours of debt.

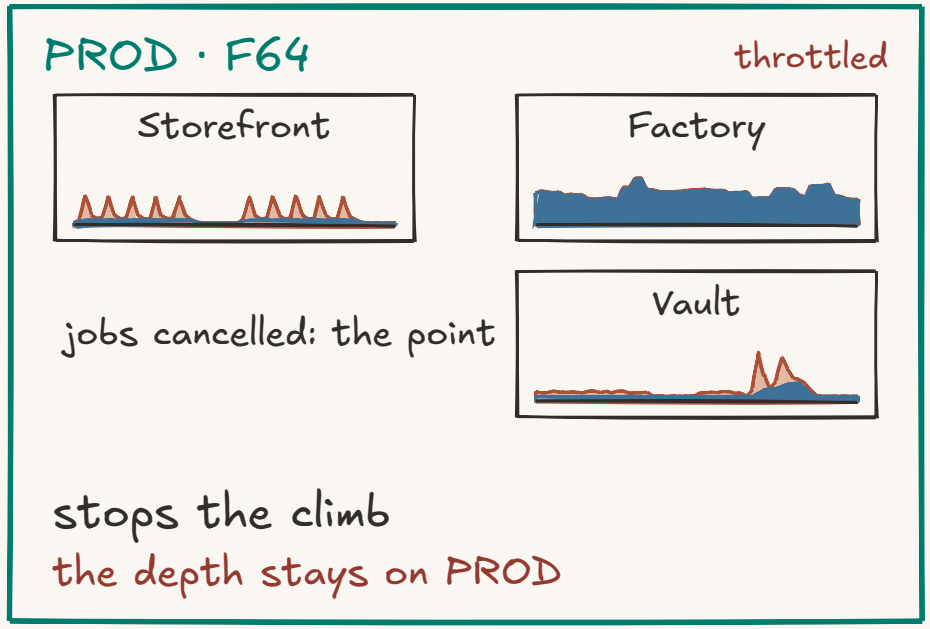


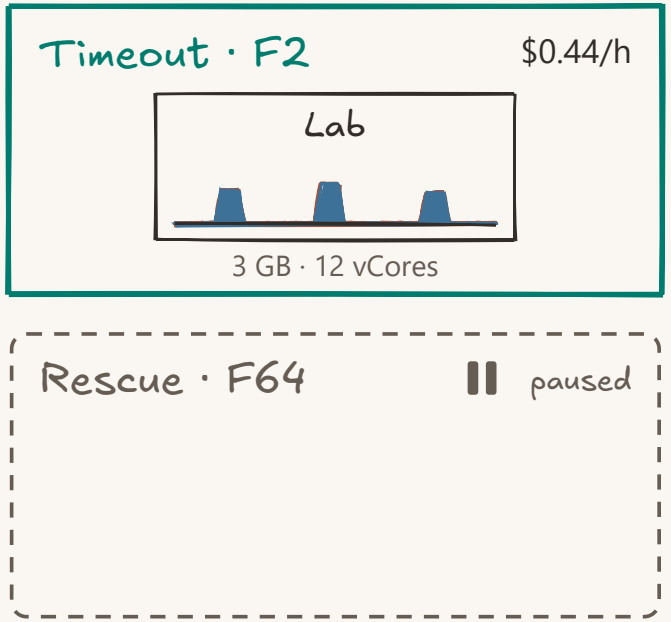
The Capacity Metrics app shows how deep a capacity is in percentages, each against its own allowance, never in hours. On the Throttling tab, the Interactive rejection graph's 100% is sixty minutes of future capacity, so its percentage divided by 100 is hours: 600% is 6 hours. (The same six hours against Interactive delay's ten-minute window would read 3,600%.) On the Overages tab, Cumulative % compares the carryforward with one 30-second timepoint of the whole capacity, so one hour is 12,000% and 72,000% is the same 6 hours. Hours × the SKU's CU gives CU-hours (384 on an F64); × the pay-as-you-go price

09:15 on the meter **$0.44/h**

Stop it: the timeout.

A timeout, not a sandbox.

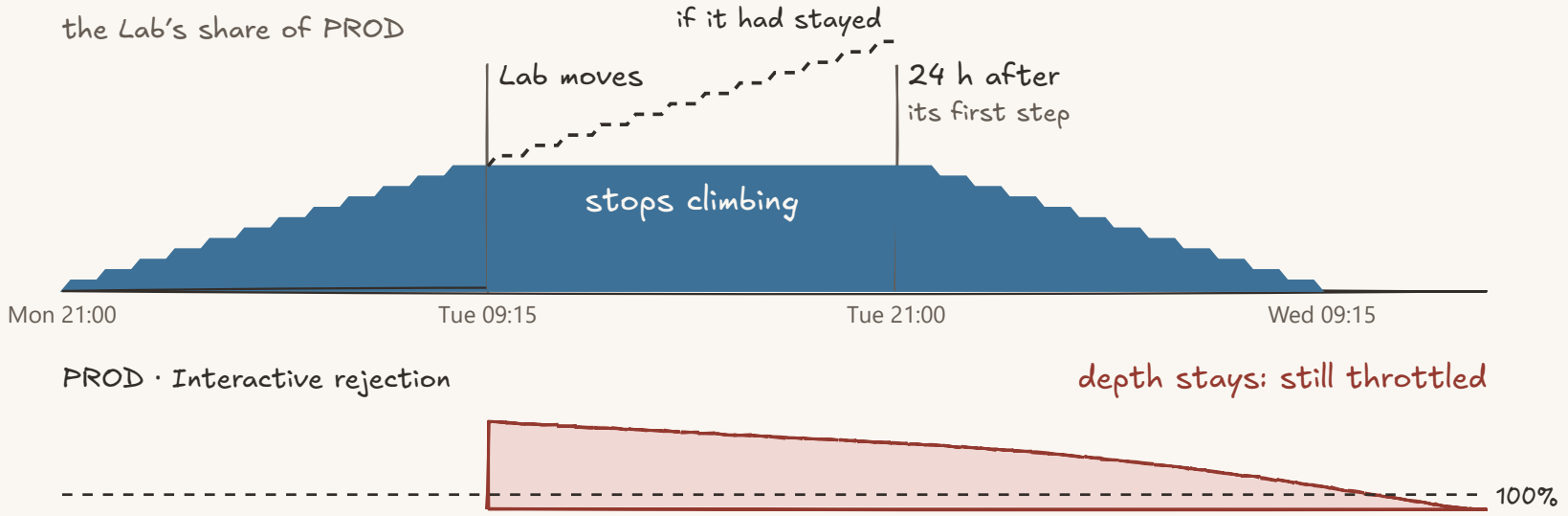




Moving the runaway workspace to a small spare capacity stops it: every job in a workspace is cancelled when it moves, and throttling never stops an operation that is already running. Both capacities must be active for the move. The move stops new depth from building up; it does not reduce the depth already owed, which stays with the source capacity, so the source stays throttled. The moved workspace now runs within an F2's limits — 3 GB of model memory and 12 Spark vCores — so it is a timeout, not a development environment. If the job alone can be cancelled, cancelling it is simpler.

09:15 on the meter **$0.44/h**

It left. Its debt didn't.

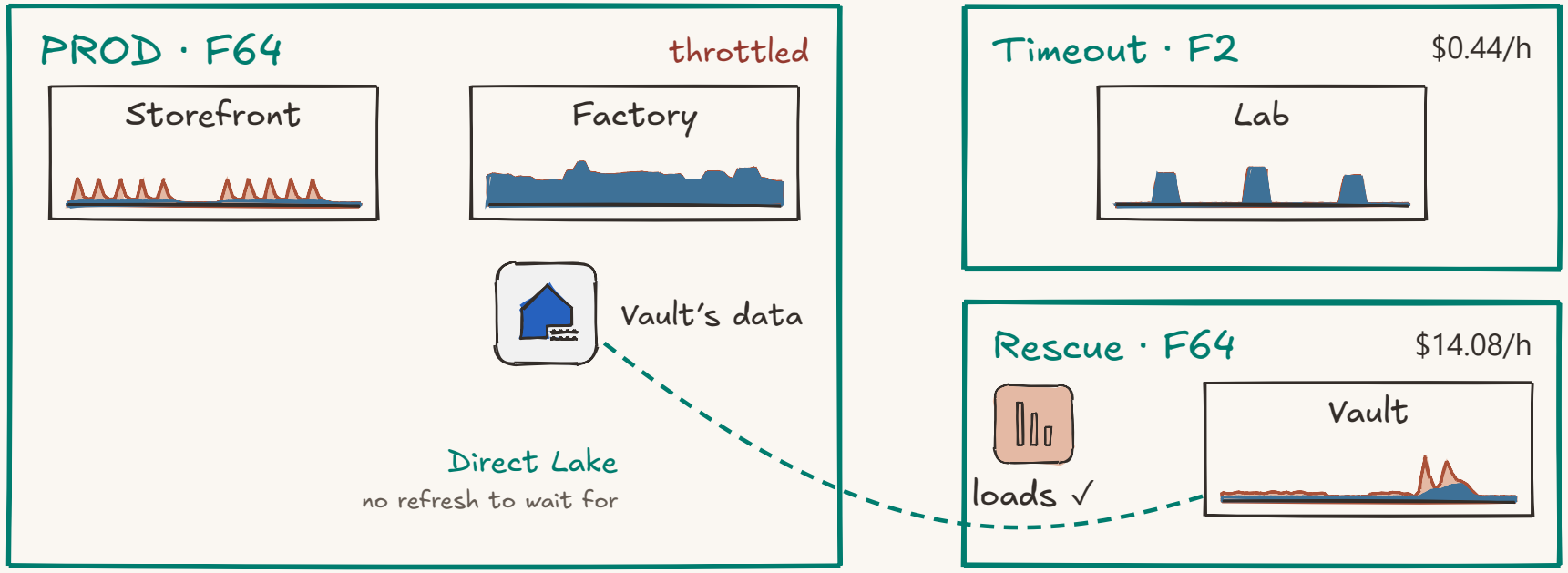


Moving a workspace does not move what it has already consumed. Background operations are smoothed over 24 hours, so a long-running notebook builds a staircase on its capacity: each chunk of work it reports adds a step. Moving it stops the staircase from climbing, but every step already laid keeps landing on the source capacity until its 24 hours are up: the load stays flat until 24 hours after the first step, then eases off, and is gone 24 hours after the move. The depth already owed also stays with the source, which therefore stays throttled. In the example, waiting alone would clear it in 27 h 55 min, because the spare

09:20 on the meter **$14.52/h**

Shelter it: the rescue.

The capacity that asks decides.



A workspace that cannot wait can move to an empty spare capacity. Its data can stay where it is: when one capacity reads OneLake data another capacity produced, the throttling state of the consuming capacity decides whether the call is throttled, and the consumer's capacity is charged for the read; the read even works while the producing capacity is paused. A report on a Direct Lake model (or over a shortcut) therefore works on the rescue capacity even though its data still sits on the throttled one, with no refresh to restart. An Import model also moves, but any refresh running during the move is cancelled. Direct Lake

09:20 on the meter **$14.52/h**

Is it worth it?

Pay what the hour is worth.

the way out	the optimal way	price	worth it while
Timeout F2	the smallest box holds the Lab	$3.52 8 h	frees $3.52/h of PROD for $0.44/h
Rescue F64	the smallest box that keeps viewers free	$31.68 2 h 15 min	an hour of the Vault down costs more than $14.08
Scale up F256	the biggest box that cancels nothing	$81.10 1 h 55 min	an hour of throttling costs more than $1.76

Every way out is a value question: is the hour it buys back worth what it costs? A timeout capacity should be the smallest that holds the workspace (F2); it costs $3.52 for 8 h and frees $3.52 an hour of the source capacity's room. A rescue capacity should be the smallest that keeps free viewers working, F64 or larger; it costs $31.68 for 2 h 15 min and pays while an hour of the sheltered report being down costs more than $14.08. A scale-up buys time back at one rate, the room the capacity already has times the price, $1.76 per hour saved; the optimal size is the biggest that does not cross the F256 and F512 boundary, which

09:22 on the meter **$14.52/h**

One debt, three prices.



Wait

27 h 55 min

$0

Scale up

F128 · 5 h 20 min · $75.09

F256

1 h 55 min

$81.10

Pause

minutes

$168.96

12 h booked all offline

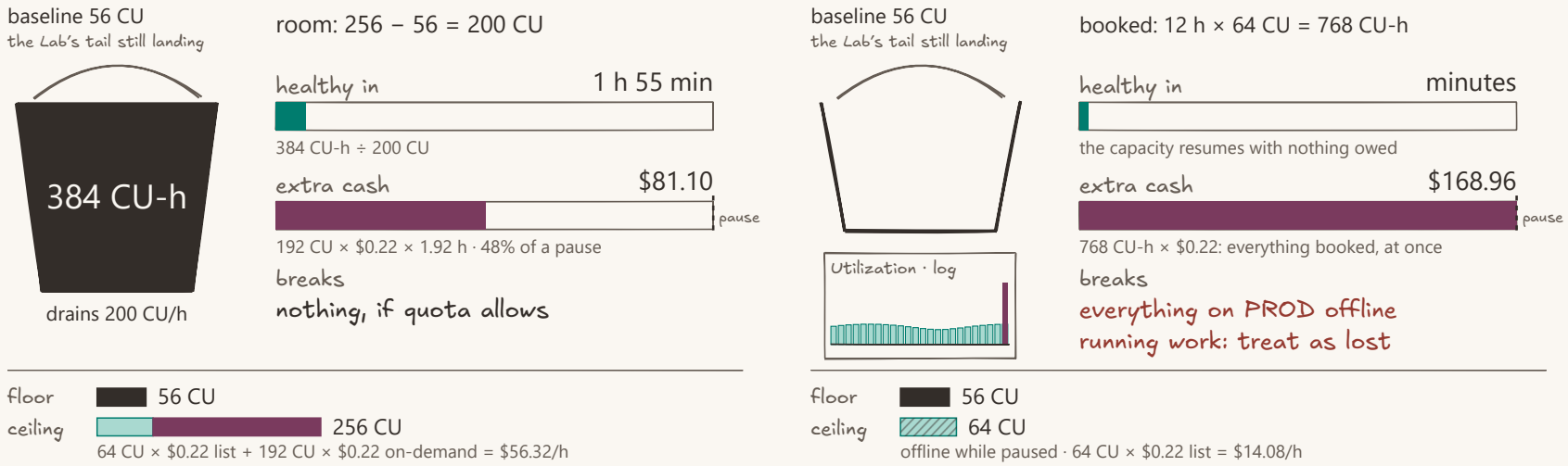
Overage

Protect's switch **carry-forward at 3×**

With 384 CU-hours owed and the moved notebook's work still landing (56 CU until it ages out), a production F64 has three basic options. Waiting costs nothing extra and takes 27 h 55 min. Scaling up to F128 takes 5 h 20 min and costs $75.09 extra; to F256, 1 h 55 min and $81.10. Pausing and resuming clears it in minutes but bills everything already booked for the next 24 hours, not only the depth: read it on the Background rejection graph, where 50% means 12 hours of the capacity, 768 CU-hours or $168.96, with everything on the capacity offline. Turning on capacity overage mid-incident bills the accumulated carryforward

09:22 on the meter **$14.52/h**

Watch it happen.



Every option can be worked out the same way. Start from the baseline load still arriving. A target size leaves room of size minus baseline; the depth divided by that room is how long recovery takes; the extra size times the price times that time is what it costs. While the baseline fits the original capacity, the extra cost of scaling up never exceeds the depth's own price, and it equals it when the baseline exactly fills the capacity. The compute strip shows the floor (the baseline load) and the ceiling (the size you run at) on one scale, with the hourly price of each part: the CU above PROD's own are on-demand, at least the reserved rate per

The way back, and the bill.

Every button costs less than the meeting about which one.

- 09:25 F256
- 11:20 cleared → back to F64
- 11:30 Vault home · banner checked
  rescue paused · Lab stays

F256 · 1 h 55 min	$81.10
rescue F64 · 2 h 15 min	$31.68
timeout F2 · 8 h	$3.52
Tuesday	**$116.30**

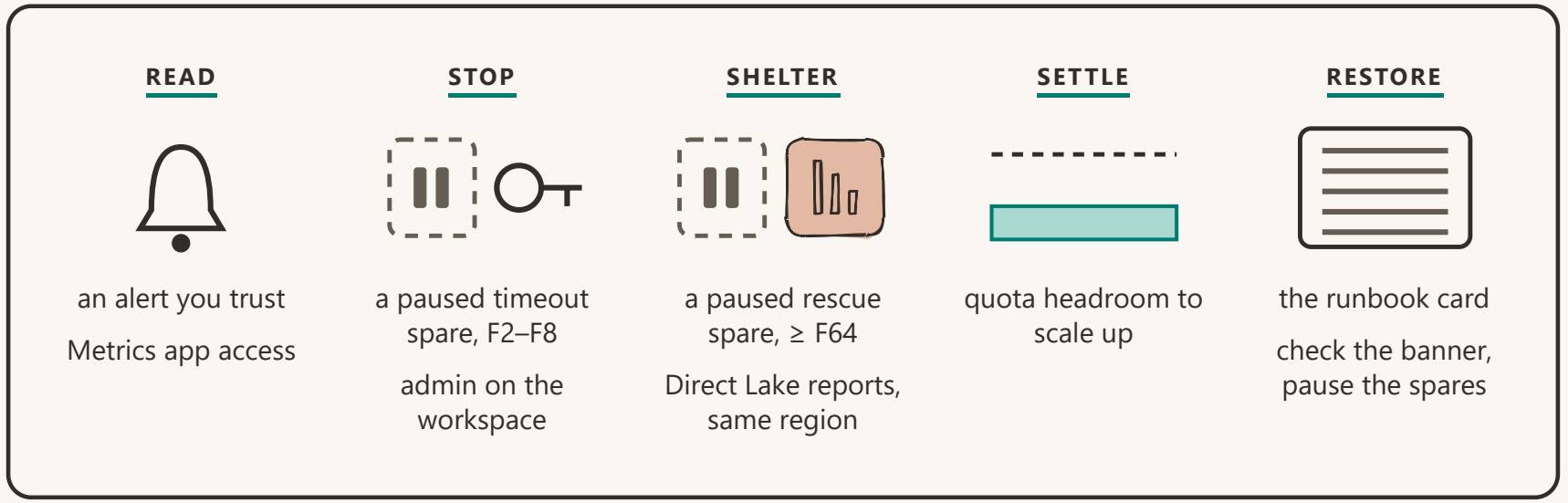
pause alone: 12 h booked = $168.96 + an outage

Recovery ends with three more moves: scale back down only once the remaining overage fits the smaller capacity, move sheltered workspaces home in a quiet window (moving cancels jobs again), and pause the spares once nothing is attached to them. In the example the day cost $116.30: F256 · 1 h 55 min $81.10, rescue F64 · 2 h 15 min $31.68, timeout F2 · 8 h $3.52. Pausing alone would have billed everything booked — 768 CU-hours, $168.96 — and taken the capacity offline. The root cause — a workspace allowed to run unbounded — is a protection question, not a rescue one.

11:30 Tuesday $116.30

Buy the kit before Tuesday.

Paused, it costs nothing.



Each step of the runbook needs something in place beforehand. Read: an alert and access to the Capacity Metrics app. Stop: a paused pay-as-you-go timeout capacity (F2 to F8) and workspace-admin rights on anything that might need moving. Shelter: a paused rescue capacity of at least F64 if its viewers use free licences, in the same region, and important reports on Direct Lake or shortcuts so that moving them moves little and needs no refresh. Settle: quota headroom to scale up. Restore: the runbook written down, ending with checking for items still attached to the source and pausing the spares. A paused capacity is not

Tuesday, paid for.

Read Stop Shelter Settle Restore

Tuesday **$116.30**

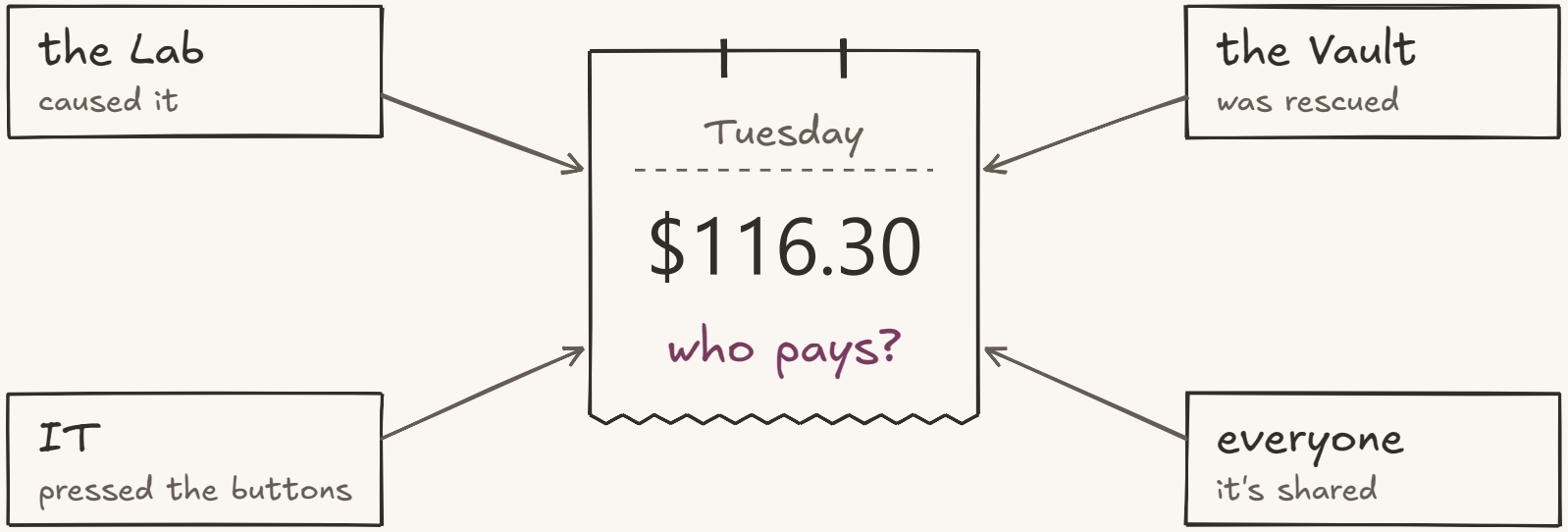


Next: who pays for that?

The runbook for an incident on a Fabric capacity: read the damage, stop what makes it worse, shelter what cannot wait, settle what the capacity owes by the option whose side effects you can accept, and restore in order. The example day cost $116.30. Who should pay for it is the next question.

Rescue: storyline

Who pays for Tuesday?

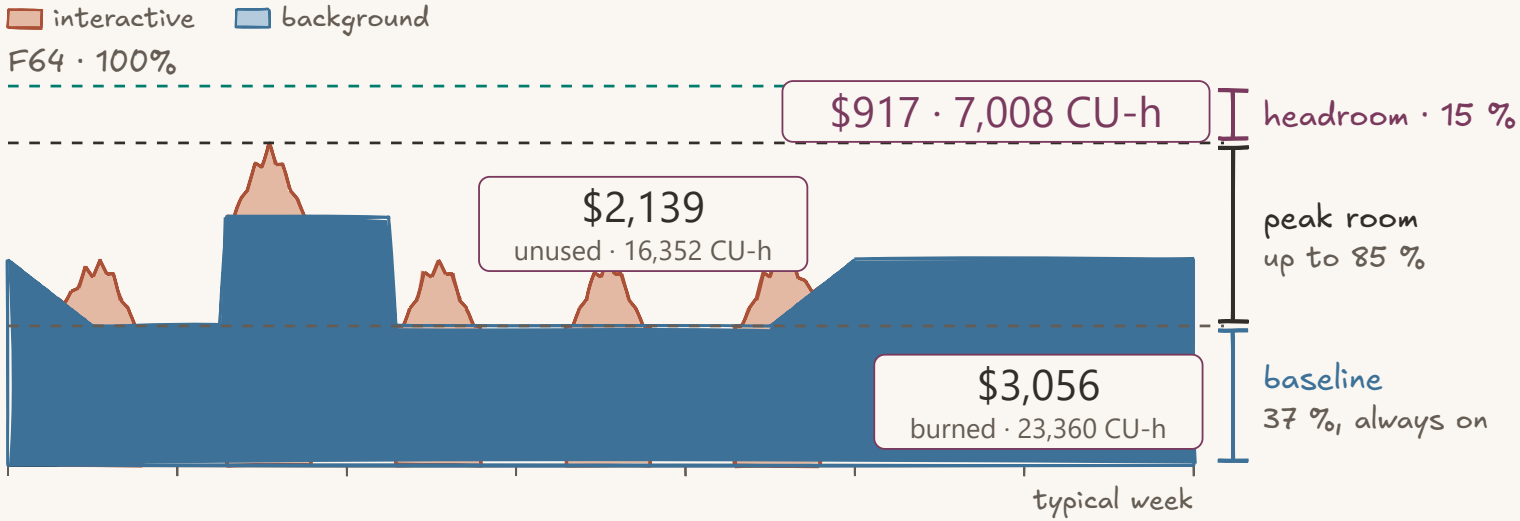


Rescue ended on a receipt: $116.30 for Tuesday's incident. The Lab caused it, the Vault was saved by it, IT made the changes — and every capacity user shares the bill. Each of those answers to “who pays?” is a different chargeback model, and this chapter compares them.

The money talk: storyline

1 month, 1 bill.

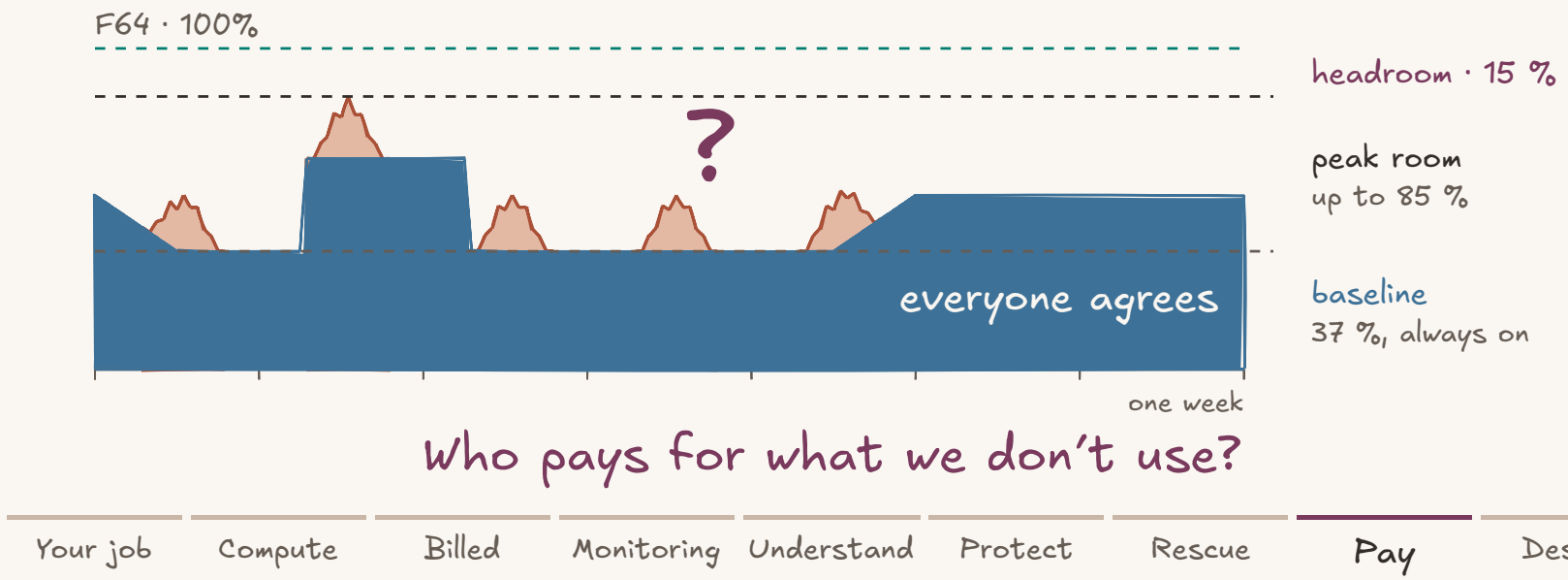
F64 · reserved · West Europe · one month = **$6,112**



You don't pay for what you use. You pay for what we keep.

A reserved F64 in West Europe costs $6,112 a month at list price: 46,720 CU-hours at $0.1308 each. Drawn as a typical week of smoothed consumption — a background floor with interactive spikes on top — three levels matter: the baseline the capacity never drops below (37 %), the peak (85 %) and the SKU line. The area under the curve was burned: $3,056 a month, 23,360 CU-hours, at a 50 % average. The white space up to the peak was paid for and unused ($2,139, 16,352 CU-hours); above the peak is headroom ($917, 7,008 CU-hours). The reservation bills all of it.

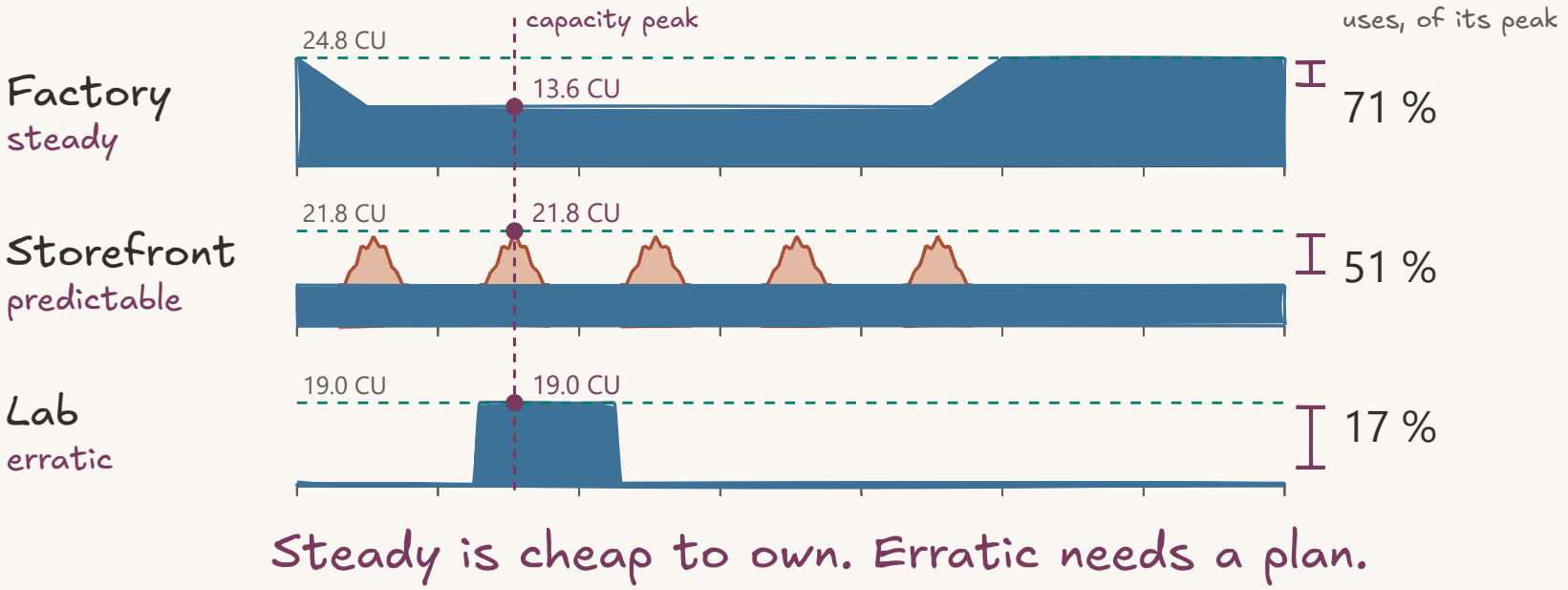
08 · The money talk



Burned CU can be attributed to a workspace, item, domain or person, and every way of charging for a capacity splits it the same way. The unused part — the room the peaks need plus the headroom above them — belongs to nobody, so every charging method is really an answer to one question: who pays for what we don't use?

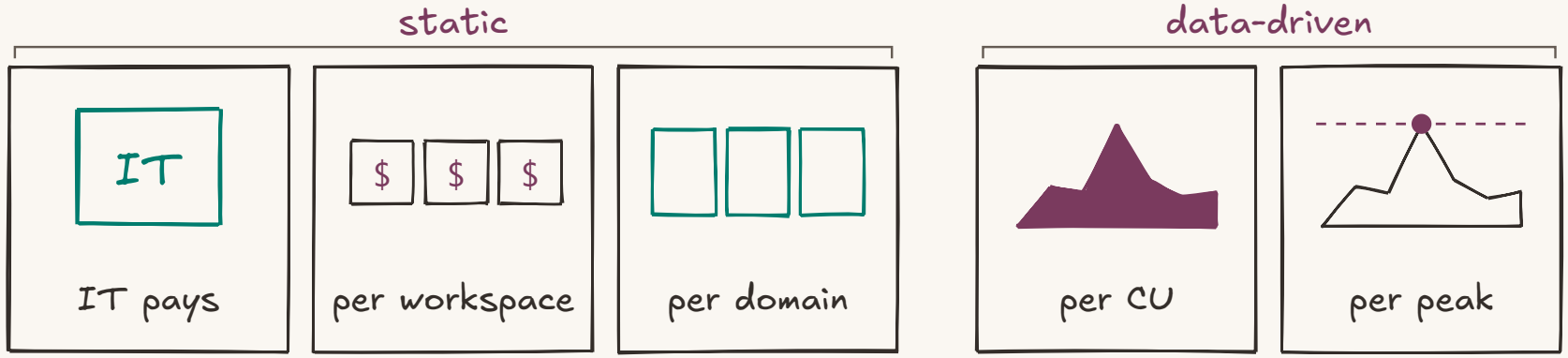
The money talk: storyline

Stable or erratic.



Split by team, the same week shows why the room exists. Against a straight line at its own peak, the Factory (steady background) uses 71 % of what it needs, the Storefront (a predictable weekday hump) 51 %, and the Lab (one run, smoothed into a 24-hour shadow) 17 %. Stable workloads are cheap to own; erratic ones need planning. At the capacity's peak, Tuesday 13:00, the three bring 13.6, 21.8, 19.0 CU. Because the teams peak at different times, sharing one capacity needs less room (54.4 CU at the shared peak) than the three would apart (65.6 CU).

Five ways to split it.

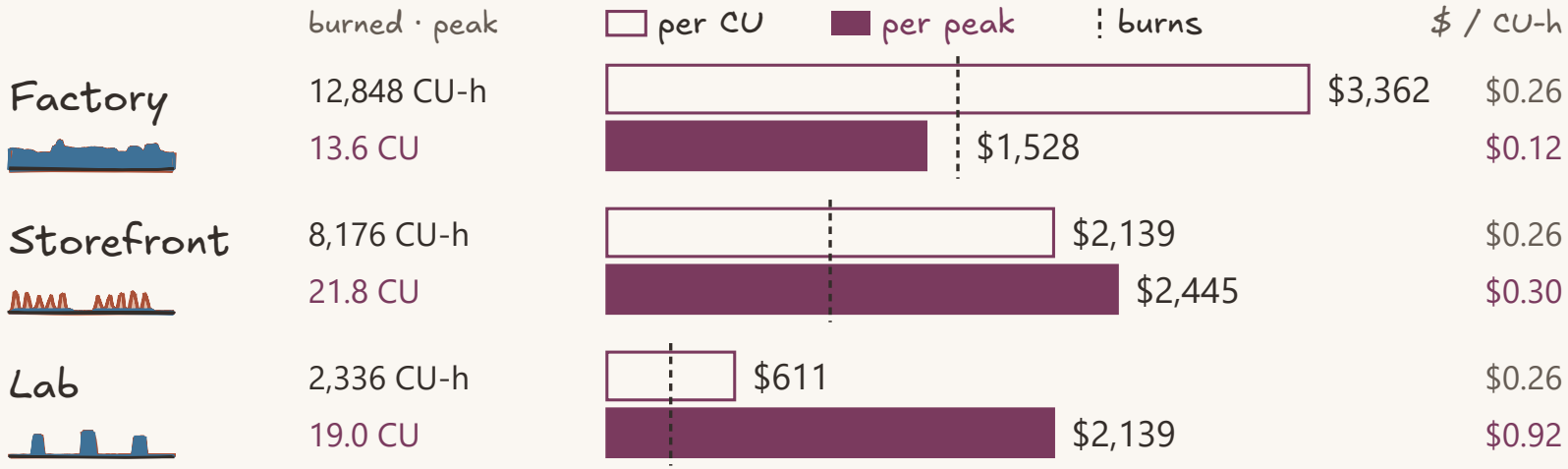


Have you talked about it?

Capacity costs are split in two families of ways. Static methods ignore consumption: IT pays centrally; teams pay a fixed price per workspace; or each domain gets a capacity of its own. Data-driven methods read it: a share of the CU burned, or a share of the capacity's peak. Static is simpler, but a capacity per domain buys the room again for every domain — in the running example, three F32s at $9,168 a month against one F64 at $6,112, 50 % more. Which one an organisation uses decides what its teams learn about the room — and in many organisations IT pays by default, without the choice ever being discussed. Even without

Per CU, or per peak?

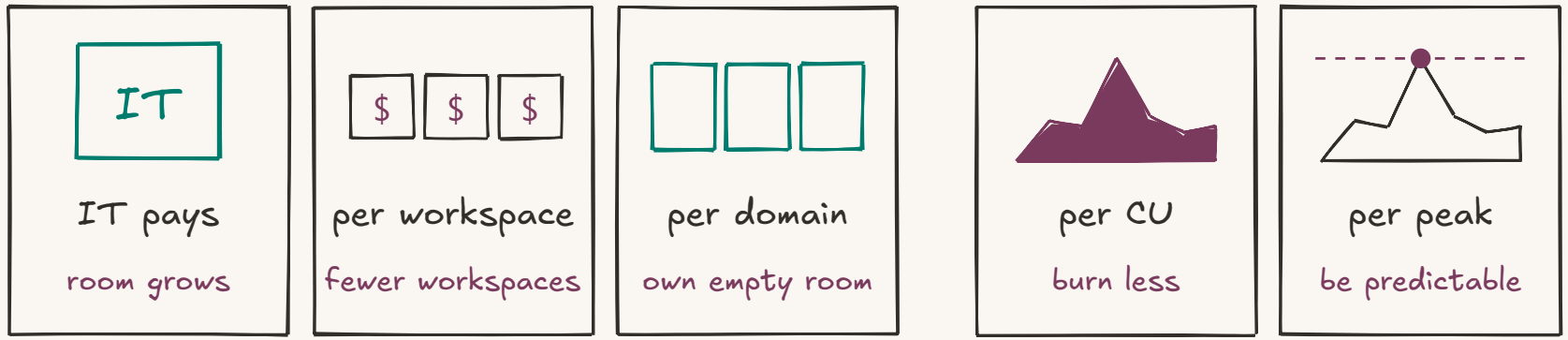
same bill, $6,112, fully allocated both ways



Per peak, you pay for the room you cause.

Two data-driven ways to split the same $6,112 bill, each from a CU measure: the CU-hours each team burns in a month (Factory 12,848, Storefront 8,176, Lab 2,336) or the CU each brings to the capacity's peak (Factory 13.6, Storefront 21.8, Lab 19.0). Per CU — Microsoft's example — charges each team its share of CU burned: everyone pays $0.26 per CU-hour, so the steady Factory pays $3,362 and the spiky Lab $611. Per peak charges each team its share of the capacity's peak: the Factory $1,528 ($0.12 per CU-hour, less than the $1,681 it burns at cost), the Storefront $2,445, the Lab $2,139 ($0.92, 7× what it burns). Per CU makes the

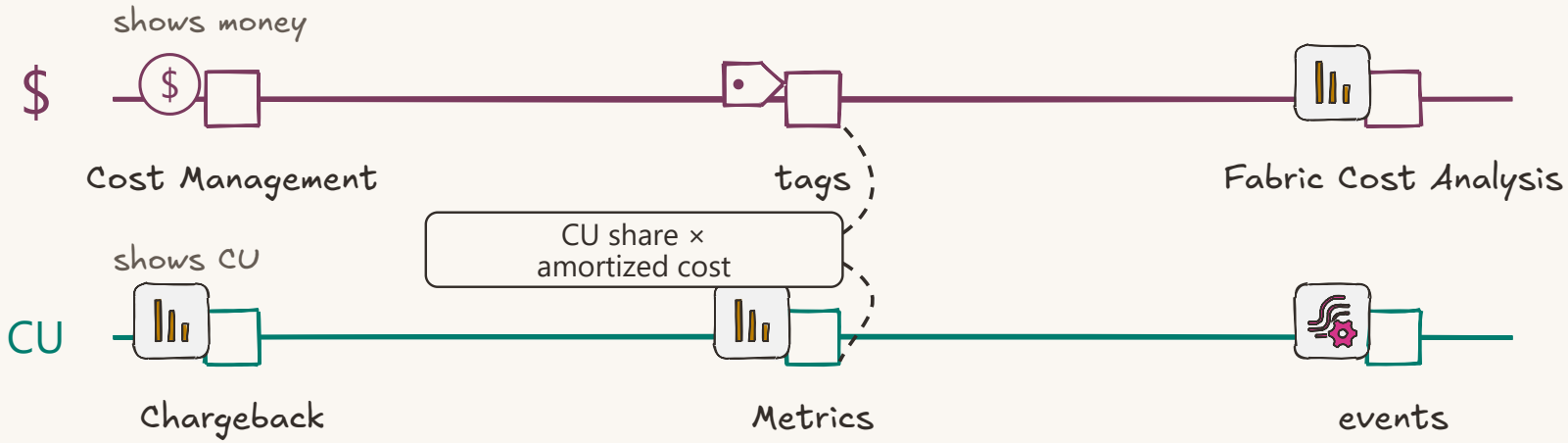
What it teaches.



An incentive system with a spreadsheet attached.

Each way of charging is an incentive. When IT pays, nobody sees headroom, so it grows. A price per workspace rewards fewer, larger workspaces. A capacity per domain makes each domain own its box — including all of its unused room. Charging per CU rewards burning less but says nothing about the shape of load; charging per peak rewards predictable load: moving work off-peak, smoothing it, or moving it off the capacity. Prices need enforcement behind them: workspace limits, timeouts and — per Microsoft's governance guidance — moving repeat offenders to a quarantine capacity.

Two floors, one gap.

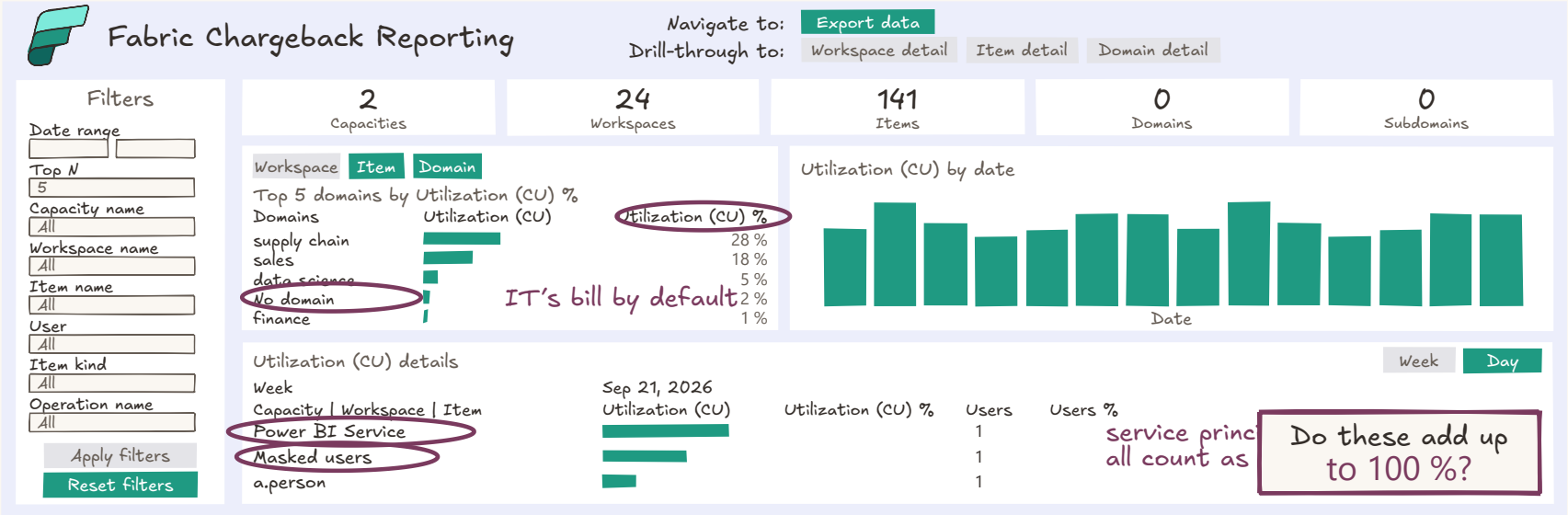


Show it in CU. Charge it in money.

Fabric's cost tools sit on two floors. Below the capacity, the Microsoft Fabric Chargeback app (daily), the Capacity Metrics App (30-second timepoints, 14 days) and Fabric capacity operation events (one row per operation) show CU by team — enough for showback, not yet for a chargeback in money. Above it, Azure Cost Management, capacity tags and Fabric Cost Analysis — a community solution accelerator in microsoft/fabric-toolbox, not an official product — show money but stop at the capacity. An internal invoice is the join: a CU share multiplied by amortized cost

Where the money gets blurry.

Microsoft Fabric Chargeback app



The Microsoft Fabric Chargeback app has Workspace, Item and Domain tabs, a daily Utilization (CU) by date chart and a Utilization (CU) details matrix with users. It shows each workspace's, item's, domain's and user's share of capacity usage, refreshed daily; Microsoft recommends installing it in a Pro workspace. Three things blur the money: anything a service principal runs is attributed to “Power BI Service”, all masked users count as a single user, and workspaces without a domain fall into “No domain”. Before invoicing, check whether the percentages sum to 100 % or to the capacity's utilization — the answer decides

Above the line.

tag

The capacity's cost owner — one of three things ARM lets you set.

allocation rule

Split evenly, by cost, or by a fixed %. Not reservations: use amortized cost.

UnusedReservation

Room one level up — its own charge type.

Copilot capacity

The one native way to charge by people.

F64

Above the capacity: tag each capacity with its cost owner — tags are one of the three properties the ARM resource exposes. Cost Management allocation rules can split a shared cost evenly, in proportion to cost or by fixed percentages, but only under EA or MCA agreements, without changing the invoice, and not for purchases such as reservations; for a reserved capacity, charge back amortized cost. Unused reservation hours appear as their own charge type. Fabric Copilot capacity bills a named group's Copilot usage to one capacity.

Only operation events see the peak.

	per CU	per peak
Chargeback app	✓ today	✗ daily
Metrics App		✗ 14 d
operation events + notebook		✓ per timepoint
no backfill: start now	shares ✓	peak level ≈

Fairer method, less help from the tools.

Charging per CU can run on the Microsoft Fabric Chargeback app's daily percentages. Charging per peak needs each workspace's share of the capacity's peak timepoints, which neither the daily Chargeback app nor the 14-day Metrics App provides; Fabric capacity operation events processed in a notebook can. The shares are reliable; the reconstructed peak level will not exactly match the Metrics App. Operation events have no backfill, so start retaining them before committing to it. In practice: write the rule down, put every workspace in a domain, publish it for two months before charging, and keep it for a

Show it first.

Factory



$1,528

12,848 CU-h

↳ Storefront, Lab

the supply plan

Storefront



$2,445

8,176 CU-h

↳ sales, every day

the sales numbers

Lab



$2,139

2,336 CU-h

↳ the Factory

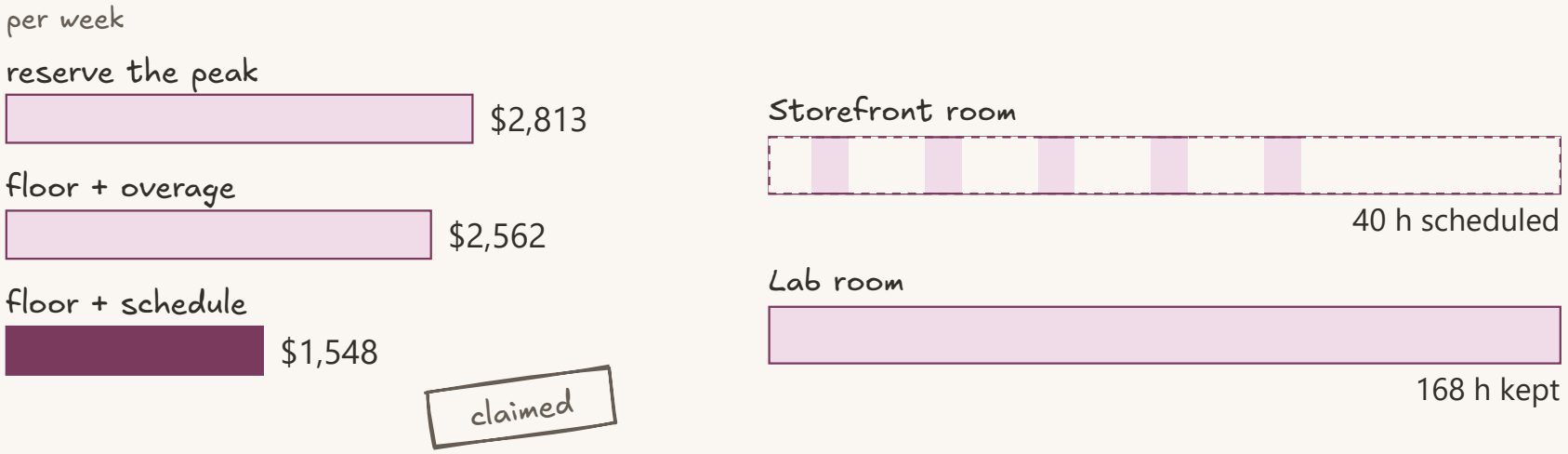
the demand forecast

What does it cost? What does it bring?

Showback sends each team a statement before, or instead of, an invoice: what its work costs including the room it needs (here per peak: Factory $1,528, Storefront $2,445, Lab $2,139 a month, with the CU-hours behind each), what uses it downstream, and what it brings. In the example the work is a chain — the Lab's forecast feeds the Factory's supply plan, whose tables feed the Storefront's sales numbers — so a cost that looks high on its own can carry value elsewhere. Showback does not decide whether a cost is justified; it makes it concrete so the organisation can ask whether the capacity is being spent the right way

Shrink the room first.

reserve the floor, schedule the peak: 45 % less



Predictable room can be scheduled. Unpredictable room has to be kept.

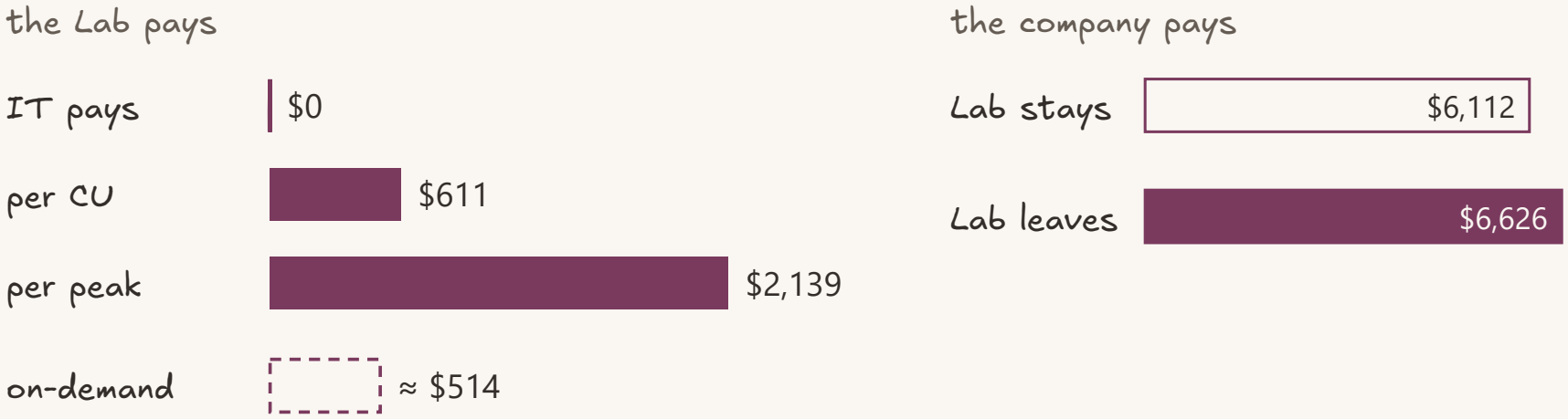
Shrinking the room beats arguing about it. Optimize and protect priced three ways to cover a peak: reserving the peak costs $2,813 a week; a reserved floor with a scheduled resize costs $1,548, 45 % less. Predictable peaks, like the Storefront's business-hours hump, can be scheduled; unpredictable ones, like the Lab's, have to be kept all week. Per peak, that difference is priced in.

The money talk: storyline

F64 · reserved · $6,112 / month

Cheapest for whom?

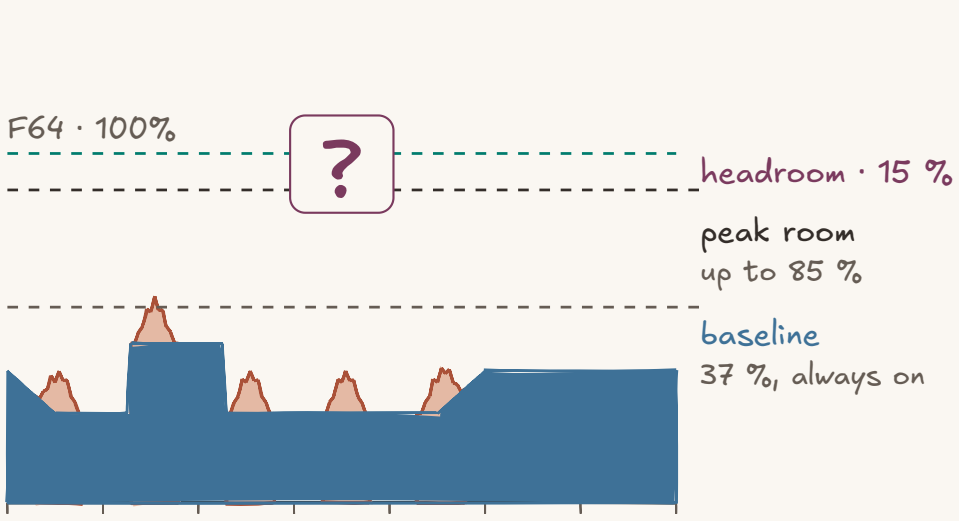
a reservation is cheapest — as long as you use it



Who decides what's cheapest?

A reservation is the cheapest way to buy capacity only while it is used; for bursty work such as a data-science Lab, pay-as-you-go or on-demand billing for Spark — serverless Spark compute, outside the capacity, billed only while jobs run — can be cheaper. What looks cheapest depends on who pays. In the example the Lab pays nothing if IT pays, $611 a month per CU and $2,139 per peak, against about $514 on demand (illustrative; the rate basis is still being verified). But if the Lab leaves, the remaining work still peaks at 35.4 CU, which does not fit an F32 at the 85 % target, so the F64 stays and the company's bill rises

A design question.

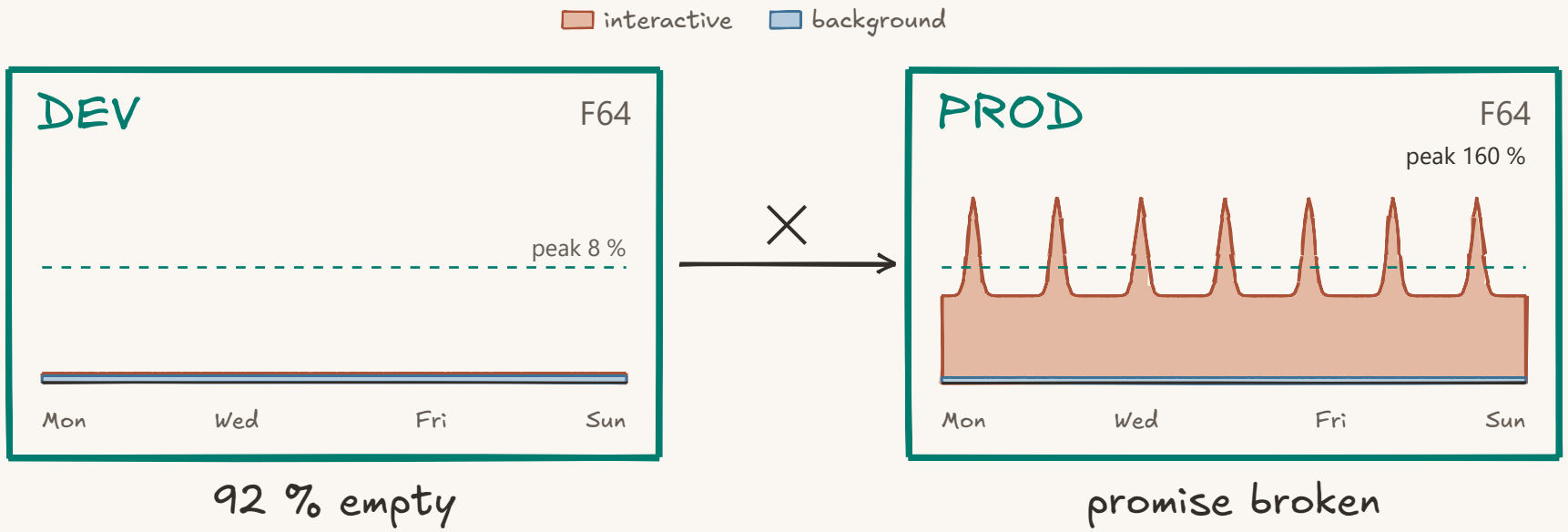




Every chargeback model ends at the same question for each team: what capacity do we need to keep for you? With the cost, the split and the cheapest option on the table, the answer is a design decision rather than a billing one — grow the capacity, or split work out onto capacities of its own — and it is where Design picks up, for the whole estate at once.

The money talk: storyline

Where does the spare compute go?

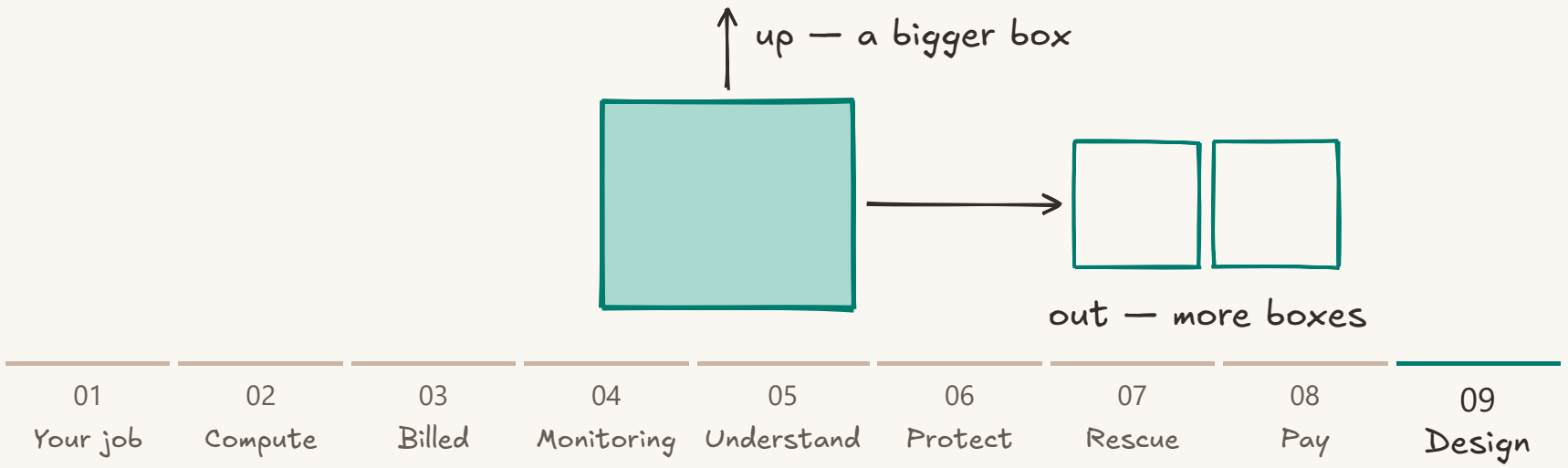


You already scaled out.

Most of the talk looked at one workspace on one capacity. Real estates schedule and balance many workloads across several capacities — at minimum production and development. Compute cannot cross a capacity boundary, so one capacity can throttle while its neighbour sits almost empty. Design decides where those boundaries go, and how they change over time.

Design: agreed storyline / Fabric throttling and the free buffer

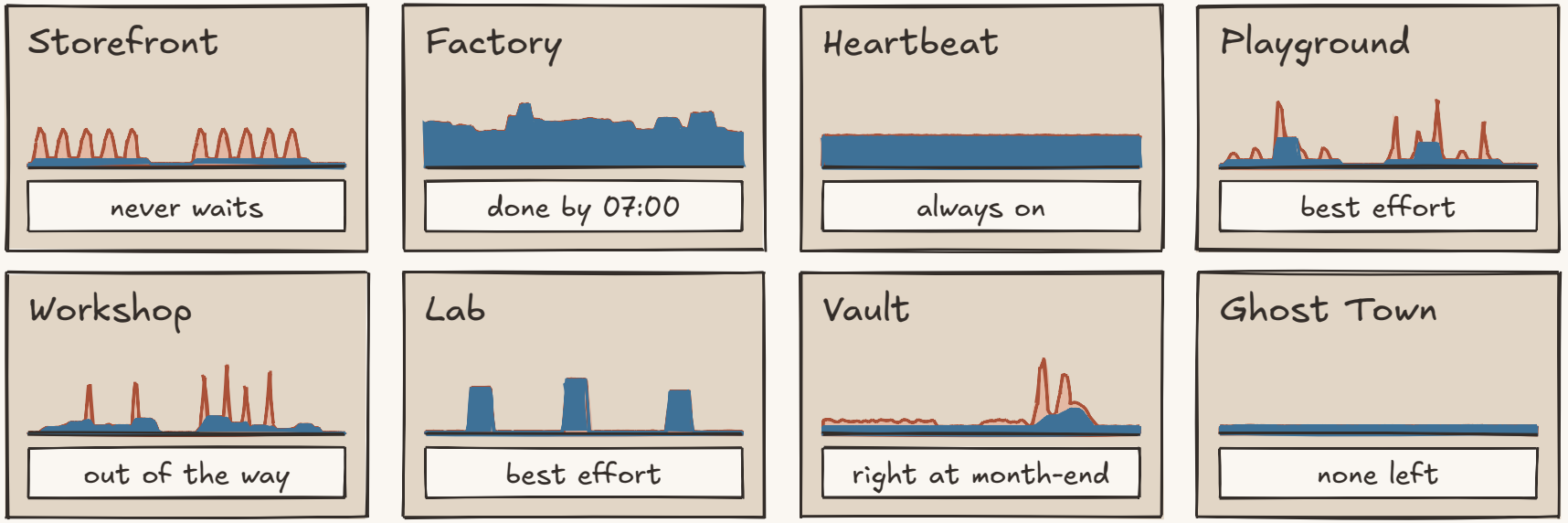
09 · Design: allocating the pool



Two moves describe every capacity design. Scale up means a bigger capacity; scale out means more capacities. The rest of this chapter is about when each one keeps a promise and when it only costs one.

Design: agreed storyline

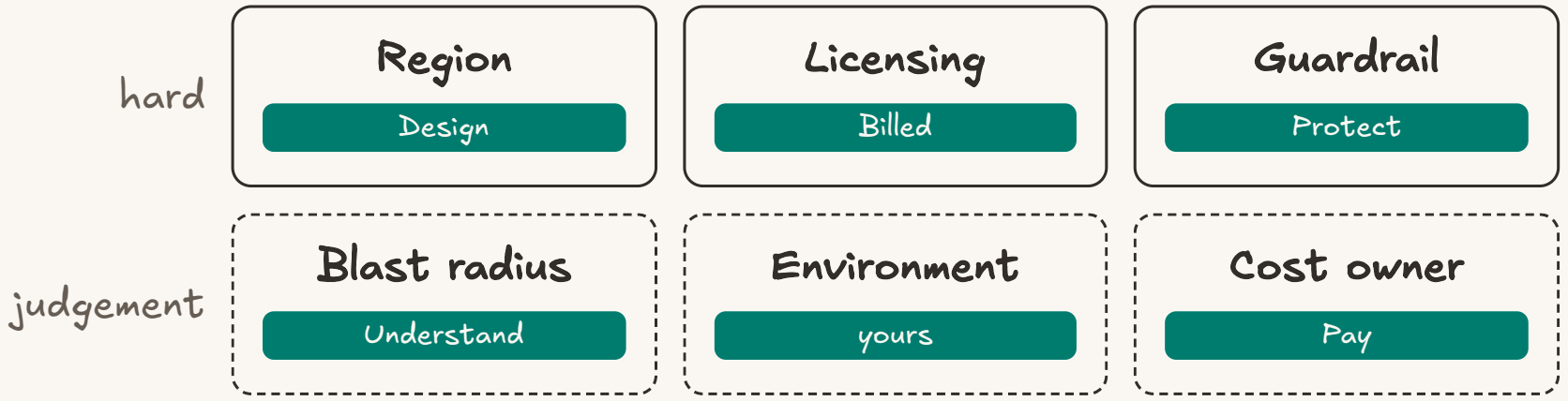
Every workspace has a promise.



Every workspace already has an SLA.

Each of the eight workspace patterns from Understand carries an implied service level: the Storefront never waits, the Factory is done by 07:00, the Heartbeat is always on, the Vault is right at the month-end close, the Workshop stays out of production's way, and Labs and Playgrounds are best effort. A Ghost Town has no readers left to promise anything to. A capacity cannot make a query faster; it can only keep it from waiting behind throttling, so a responsiveness promise is kept with headroom, not with a bigger SKU. Neighbours and per-SKU ceilings are the two things that break these promises. The tags are illustrative

Six reasons for a thick line.



Three you can't argue with. Three you decide.

Workspaces are given; design decides which workspace runs on which capacity. Six things justify a separate capacity. Three are hard constraints: the region the data and compute must live in (a design decision in its own right — Fabric items cannot be moved across regions), licensing (free viewers need F64 or larger), and a per-SKU ceiling a smaller capacity lacks. Three are judgements about behaviour: blast radius between workloads, environment (production kept apart from development or self-service), and a distinct cost owner. Hard reasons always force a boundary; judgement reasons force one only as far as the

Compute scales linearly. Guardrails don't.

	F32	F64	F128
CU	32	64	128
Model memory	10 GB	25 GB	50 GB
Direct Lake rows	300 M	1,500 M	3,000 M
OneLake model size	40 GB	unlimited	unlimited
Free viewers	✗	✓	✓

Capacity units scale linearly with the SKU — F32, F64 and F128 have 32, 64 and 128 CU — but the per-SKU guardrails do not: semantic model memory, Direct Lake row limits and the OneLake model size step unevenly, and free viewers need F64 or larger. A workload that needs a larger ceiling forces a capacity boundary (or a size) that its CU use alone would never justify.

Semantic model limits per SKU / Direct Lake overview and guardrails / Microsoft Fabric licenses and capacity SKUs

Two goals.

Make your SLAs

- separate by environment
- spread over capacities

Reduce waste

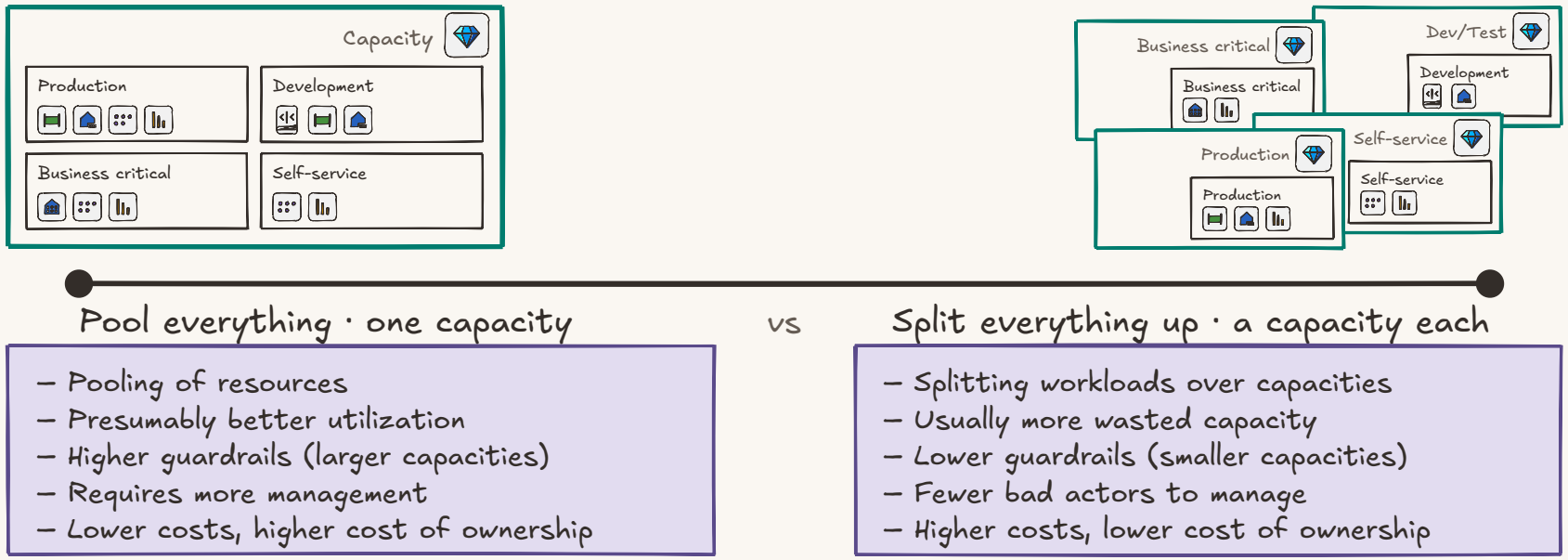
- pool capacities
- capacity overage
- the one-third maths

Every design trades one against the other.

Capacity design has two goals that pull against each other: meet each workload's service level, and reduce waste by using the purchased compute efficiently. Service levels are protected by separation — by environment, or across several capacities. Waste is reduced by pooling: sharing capacities, relying on capacity overage for peaks, and checking when paying for occasional overage is cheaper than a larger SKU. Every design is a trade between the two.

Design: agreed storyline / Fabric throttling and the free buffer

Where on the axis?

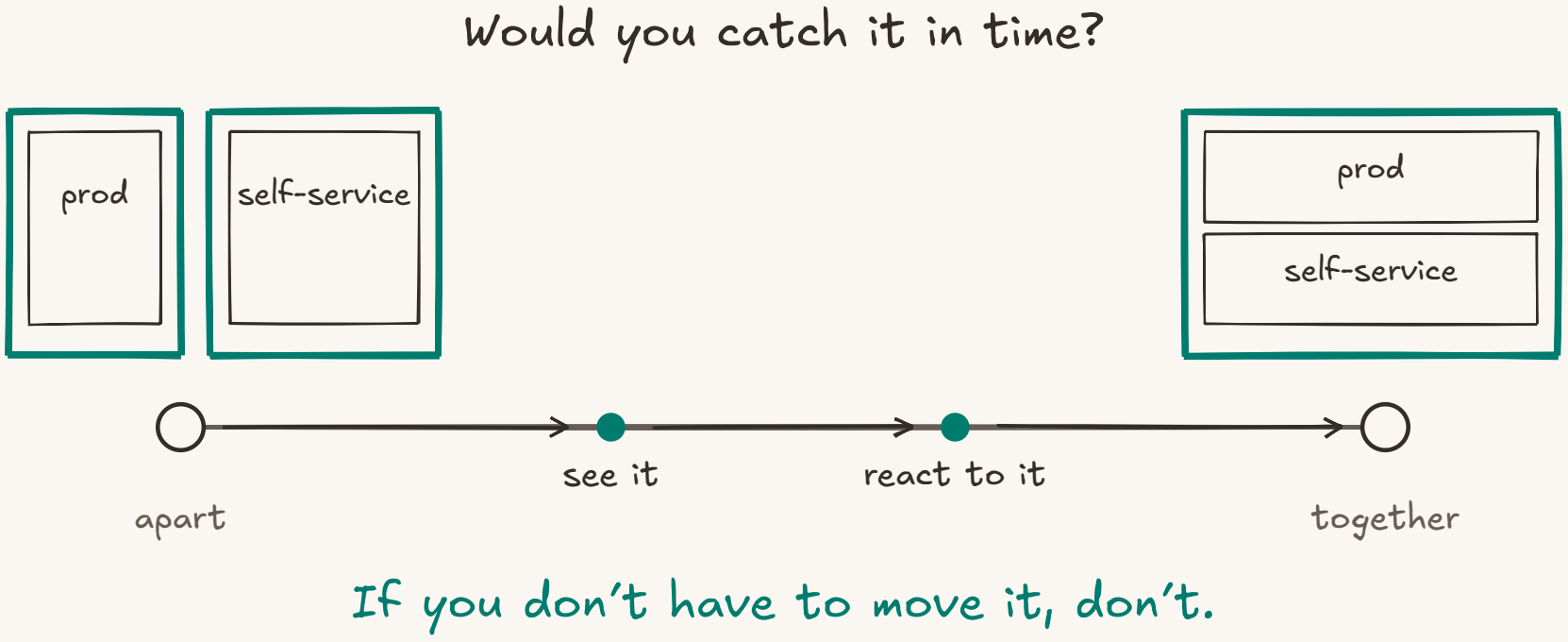


After the hard lines: somewhere in between.

The axis runs from pooling everything on one capacity (presumably better utilisation, higher guardrails because the capacity is larger, lower cost, but more to manage and a higher cost of ownership) to splitting everything up, a capacity each (usually more wasted capacity, lower guardrails because each capacity is smaller, fewer bad actors to manage, a lower cost of ownership). After the hard constraints are applied, the right point is somewhere in between.

Design: agreed storyline / Scale a Fabric capacity

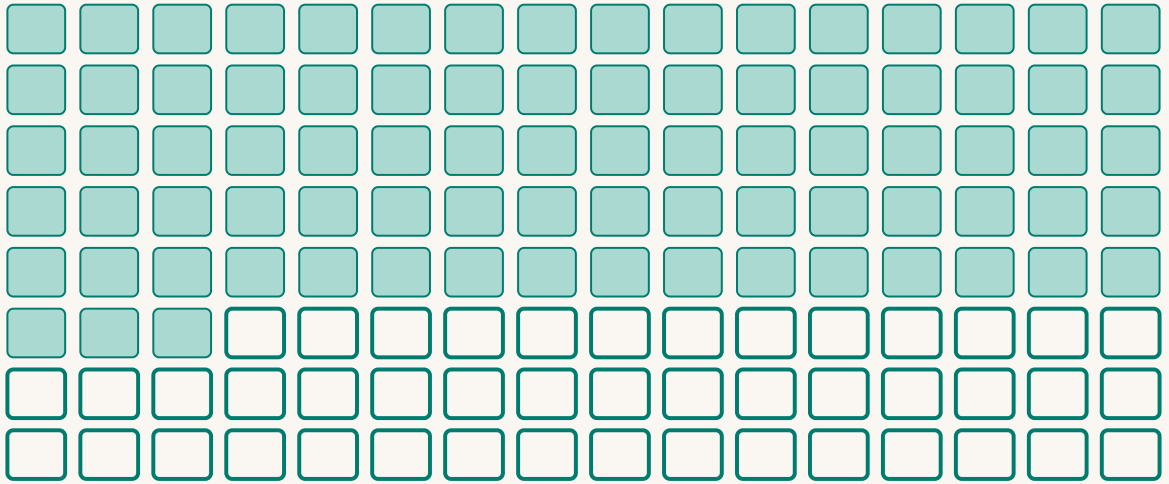
How far apart?



If you don't have to move it, don't.

Where a judgement reason applies, how far apart to keep workloads depends on maturity: how confident the admin team is that it would detect and contain a problem before it spreads, and how well users understand what their work costs. A common, sensible starting point is to keep self-service workloads on a separate capacity from production. As monitoring (capacity operation events) and alerting improve, workloads can be consolidated to use compute better. Avoid moving workspaces without a reason: every move interrupts running work.

Stop drawing boxes.



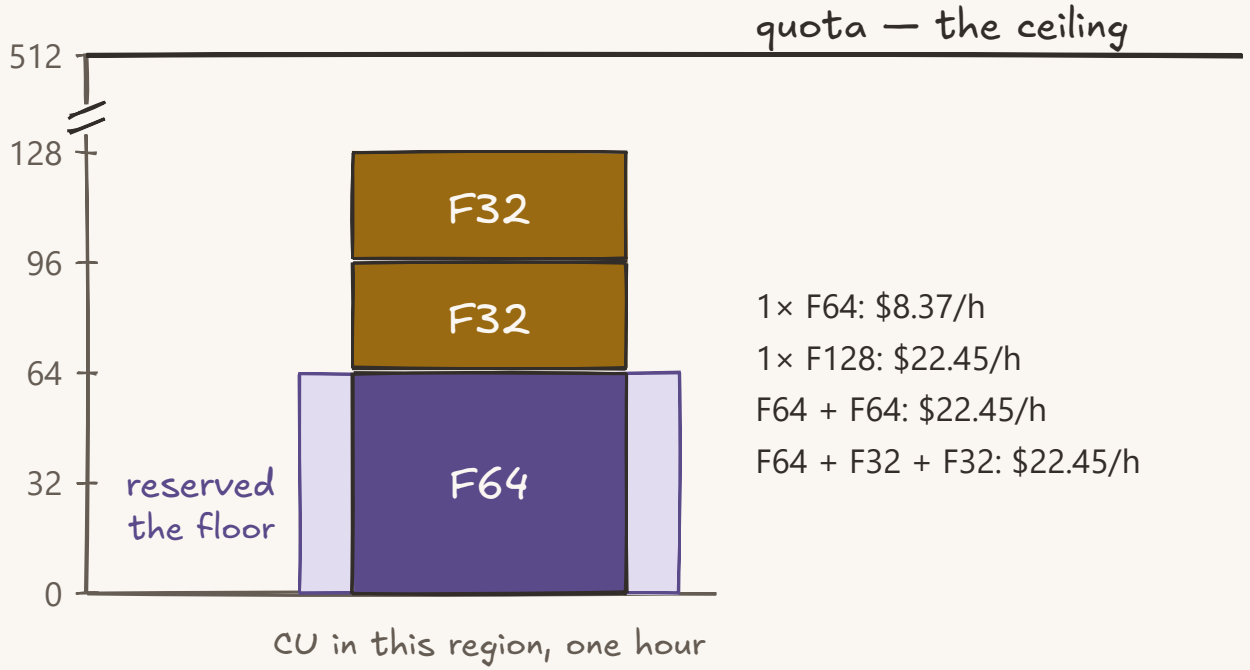
45 idle CU: what the promises cost

The capacities aren't the design. The pool is.

Once the service levels are protected, the design question changes from "which boxes?" to "how much of the pool is wasted?" In the example, about 45 of 128 CU sit idle on average: that idle capacity is the cost of the promises, and the rest of the chapter is about reducing it.

Design: the arithmetic behind every number

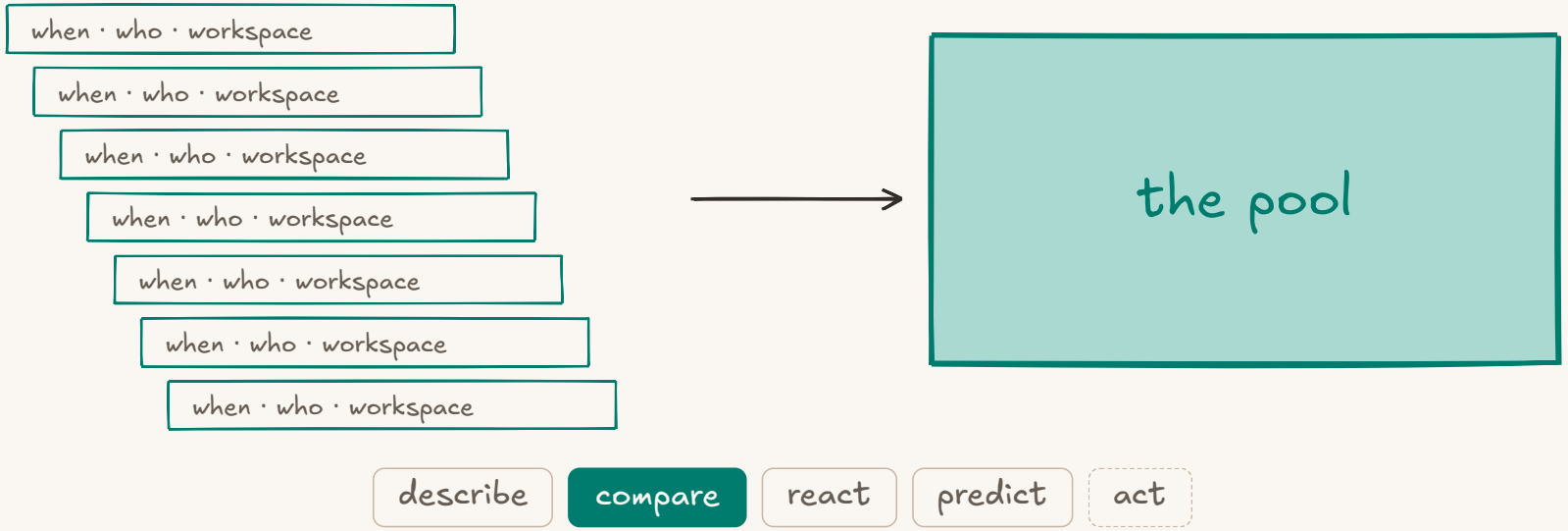
A floor, a ceiling, and a band.



A reservation sets the floor of the pool and quota (or budget) sets the ceiling; the drawing is the same column as in How you're billed. Between the two, the same CU can be carved into different capacities freely, because a reservation applies to CU in a region rather than to a specific capacity: the reserved 64 CU are discounted whichever capacities stand on them, and anything above is billed pay-as-you-go. The 512 CU quota is illustrative.

Fabric capacity reservations / Fabric capacity quotas

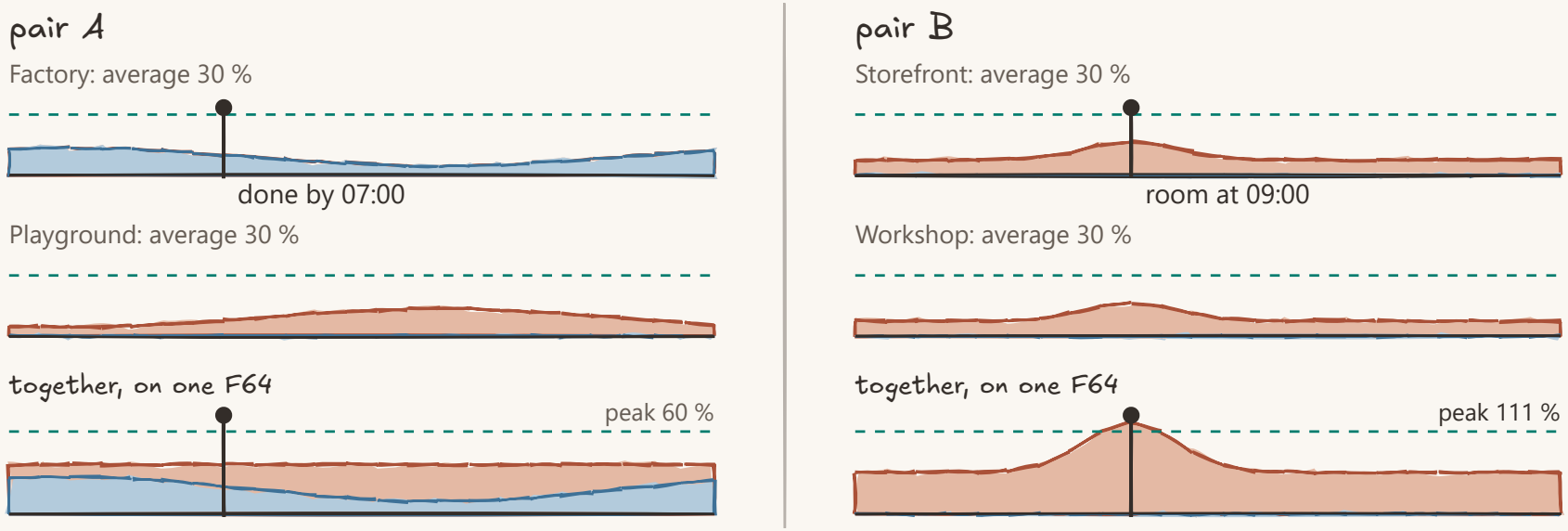
See the whole pool.



Fabric capacity operation events deliver one row per operation across every capacity, retained for as long as you choose. They make it possible to compare workspaces across capacities over time, which the Capacity Metrics app cannot.

Fabric capacity operation events in Real-Time hub

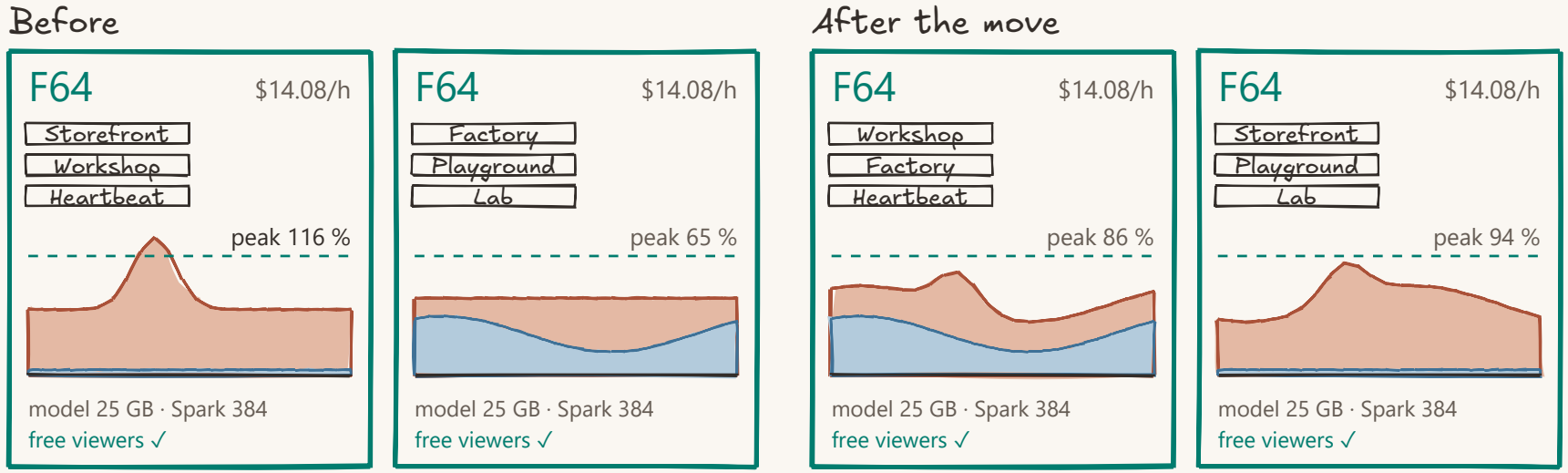
Thread the deadlines.



Same average. Different when.

Each workload's promise has a time when it is tested — the Factory must be done by 07:00, the Storefront must not wait at the 09:00 peak. Two pairs of workspaces with identical averages behave very differently together: one pair sums to a flat 60% of an F64, the other passes 100% at 09:00, exactly when the Storefront needs headroom. Smoothing means a capacity need not be sized for its peak, but a pool is filled well only by planning where peaks fall. Headroom keeps interactive work from waiting; it does not make queries faster. As in Understand, the workspace to move is the one whose removal alone brings the peak under

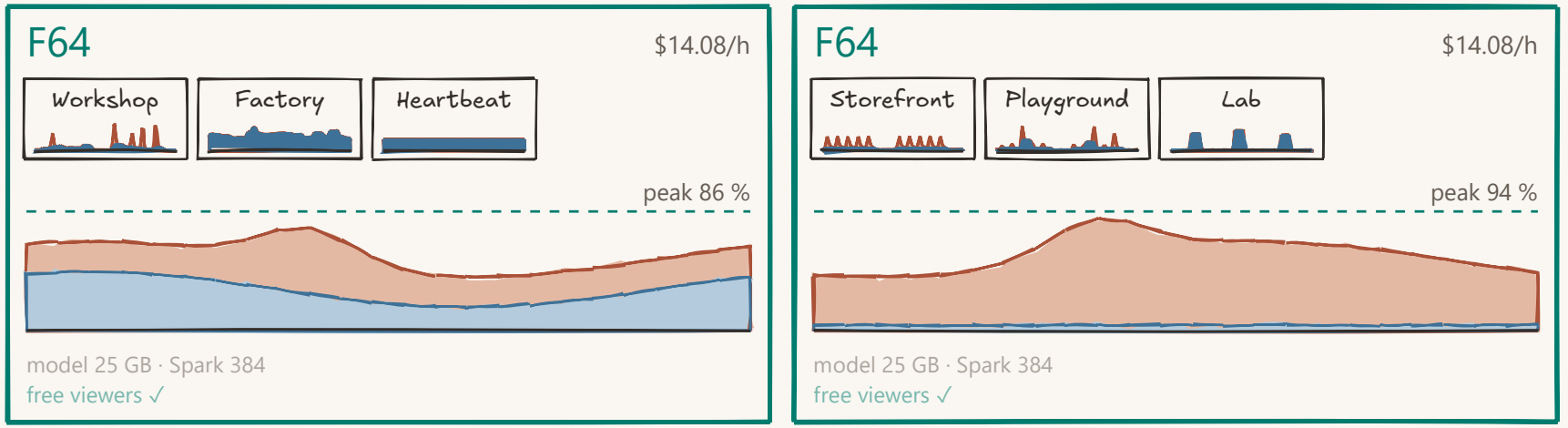
Move by when.



Two F64s with the same total load: before the move one throttles at 09:00 while the other idles; after moving one interactive workspace away from a coinciding peak, both stay under the line without adding CU. Balance background work for volume and interactive work for timing.

Reassign a workspace to another capacity / Design: the arithmetic behind every number

What the events can't see.



Every CU. Not one guardrail.

Operation events record CU consumption, but not guardrail pressure such as model memory or Spark vCores. A design based only on CU can still hit a ceiling, so per-SKU limits must be checked separately.

Fabric capacity operation events in Real-Time hub / Semantic model limits per SKU

Four moves.

Resize	Split	Merge	Temporary
up by day down by night	F128 → F64 + 2× F32	2× F32 at 40 % → 1× F32 at 80 %	onboarding timeout rescue

Moving is inside three of them.

Every change to a pool is one of four moves: resize a capacity (up for the peak, down for the quiet hours); split one capacity into several so a promise gets its own room; merge capacities that are each under-used (two F32s at 40% fit one F32 at 80%, at half the cost); or add a temporary capacity — used to onboard a workload, to isolate one that misbehaves, or to rescue a throttled capacity. Moving workspaces is part of splitting, merging and temporary capacities, not a separate move.

Scale a Fabric capacity / Reassign a workspace to another capacity

What a move costs.



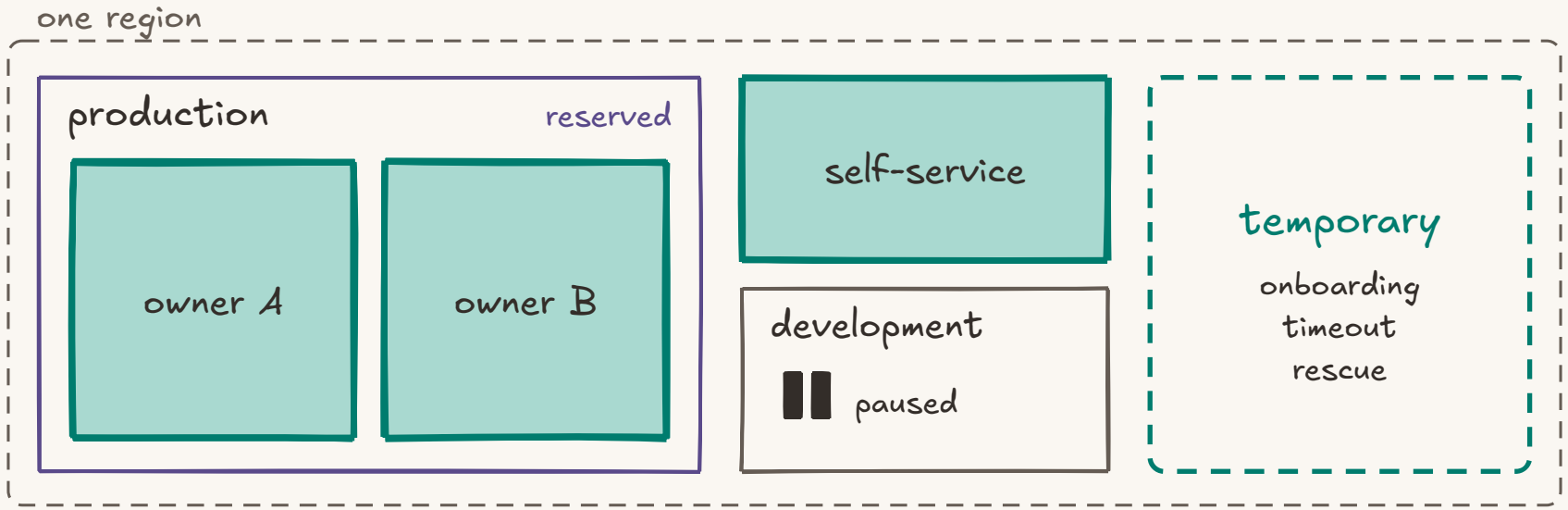
resize F256 ↔ F512: can cancel jobs
cross F64: licence takes up to a day

Flexible, not dynamic.

Moving a workspace between capacities interrupts running jobs, can take up to an hour, may complete partially, requires admin rights on both capacities, and needs both to be active and in the same region. Resizing between F256 and F512 can cancel jobs, and crossing the F64 threshold can take up to a day to update licensing. Moves are flexible, but not dynamic.

Reassign a workspace to another capacity / Scale a Fabric capacity

A complete capacity system.



Plan for region, owner, environment — and a spare slot.

A complete starting design within one region: a production pool covered by a reservation and divided into capacities by owner; a separate capacity for self-service work; a development pool without a reservation so it can be paused; and room for a temporary capacity, used to onboard a new workload, to isolate one that misbehaves, or to rescue a throttled capacity. Region, ownership and environment should all be planned for, even if the environment split changes over time.

Fabric capacity reservations / Pause and resume a capacity / Fabric region availability

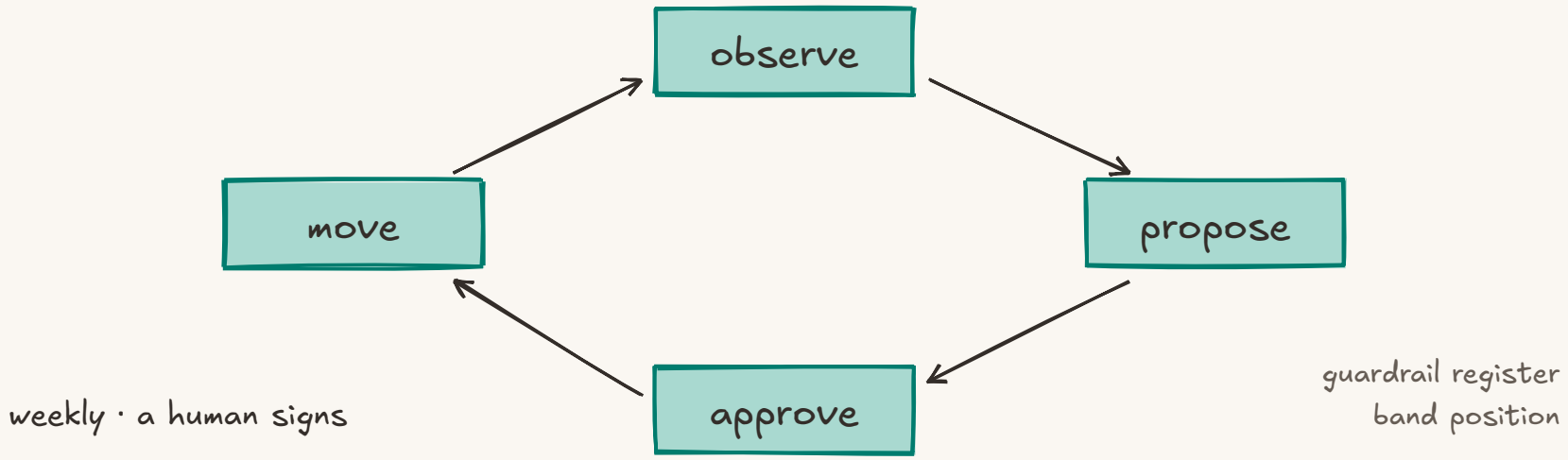
The design card.

1	Promise	every workspace: never waits, on time, always on, right at month-end, best effort	Understand
2	Draw a thick line	hard reason: always; judgement: as far as your maturity needs	Design
3	Size each box	the largest of CU, ceiling and licence	Protect
4	Let the shape move	split for the peaks, merge for the quiet hours	Design
5	Set the band	reserve the floor, quota the ceiling	Billed
6	Fill it by when	not by how much	Understand · Monitoring
7	Pick the lever	for the peak, and schedule what you can predict	Protect · Pay
8	Keep a spare	in the drawer, and a name on every box	Rescue · Pay

Every lever buys CU. None buys a ceiling.

The design, in eight steps: write each workspace's promise; always draw a capacity boundary for a hard reason (region, licence, a ceiling), and for a judgement reason (blast radius, environment, cost owner) only as far as your maturity needs; size each capacity to the largest of CU, ceiling and licence; choose the shape on ceilings and blast radius, since price is equal; reserve the floor and set a quota ceiling; fill by timing, not volume; pick a lever for the peak and schedule the predictable; keep a spare capacity and name an owner for every one.

An ongoing process.



A pool is run as an ongoing loop: observe operation events, propose a move, have a person approve it, and make the move in a quiet window. A guardrail register covers what the events cannot see. As the team comes to understand what is happening and why, the design can follow the workload — and consolidation becomes safe.

Fabric capacity operation events in Real-Time hub / Reassign a workspace to another capacity

Your Thursday.

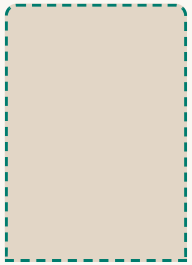
Thursday	See before you move.	
09:00	Keep the operation events	Monitoring
10:00	Write the promises	Understand
11:00	Read your ceilings	Protect
13:00	A spare in the drawer	Rescue
14:00	Name the cost owner	Pay

Five first steps, about an hour each, none needing approval: start retaining capacity operation events, since history begins only when you do; write down each workspace's promise and owner; check your largest models and Spark pools against their SKU's limits; provision a small spare capacity, paused, in the same region; and tag every capacity with a cost owner. None of them moves a workspace.

Fabric capacity operation events in Real-Time hub / Pause and resume a capacity / Fabric Chargeback app

What do you know now?

Throttled last month?



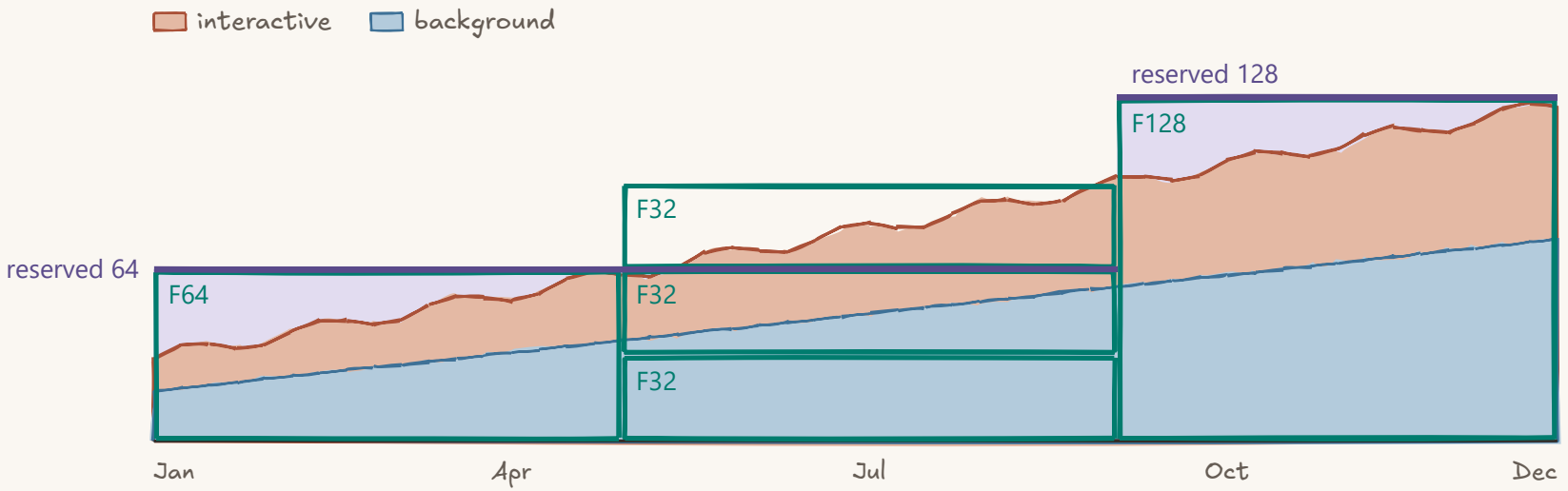
minute 2

What do you know now that you didn't?

The opening poll asked how many attendees had been throttled in the past month. The answer does not change during a session; what changes is what the audience can now see, explain and do about it.

Design: agreed storyline

Invisible is not idle.



A well-run capacity is invisible — and invisible is not idle.

Over a year, an illustrative pool changes shape twice — one capacity, then three, then one larger one — while its reservation floor steps up with it. The load fills each shape without crossing its limit. The goal of capacity design is a pool that runs full without anyone noticing: high utilisation, and no throttling.

Design: agreed storyline / Fabric capacity reservations

Thank you.

Daan Humblé

Cloud Solution Architect · Data & AI

LinkedIn /in/daanhumble
GitHub /HumbleDaan

Slides will be published at
github.com/HumbleDaan/conference-presentations

Evaluation QR code (capture)



Daan Humblé, Cloud Solution Architect, Data & AI. LinkedIn: /in/daanhumble. GitHub: /HumbleDaan. Scan the QR code to evaluate the session. The slides will be published at github.com/HumbleDaan/conference-presentations.

Design: agreed storyline